Introducción a Git y GitHub

Estructuras de Datos - DCIC - UNS

Qué es Git



Git es un sistema de control de versiones distribuido, libre y de código abierto, optimizado en cuanto a velocidad y performance lo que lo vuelve apto tanto para sistemas de pequeña como gran escala.

Fue creado originalmente (2005) por Linus Torvalds para el control de versiones del Kernel de Linux.

Es el sistema de control de versiones más utilizado en la actualidad por sobre otras alternativas antiguas/obsoletas o menos utilizadas como por ejemplo CVS, SVN, o Mercurial.

Se puede descargar e instalar desde: https://git-scm.com/

Qué es GitHub



GitHub es un servicio online de repositorios Git.

Provee una interface web y hosting para gestionar repositorios remotos y para facilitar la clonación, conexión y administración de los mismos y los cambios que éstos trackean.

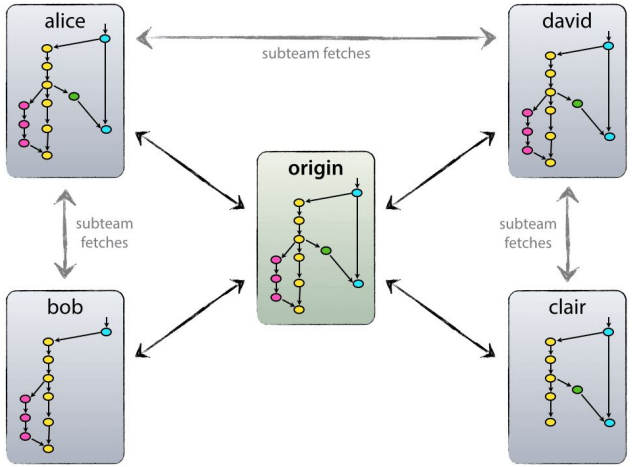
Por sobre Git, incorpora un conjunto de facilidades para la gestión de proyectos de desarrollo: Pull Requests (PR), gestión de Issues y tareas, protección de ramas importantes, mecanismos de Integración y Despliegue Contínuo (CI/CD), fork y templates de repositorios, etc.

Es el servicio más utilizado. Otras alternativas a GitHub son GitLab, Bitbucket, etc. Sitio web: https://github.com/

Git - Conceptos Básicos

Git - Conceptos Básicos

Git es una utilidad por línea de comandos (CLI) que provee una serie de **comandos** para gestionar el **historial de cambios** de proyectos mediante ramificaciones y transferencias entre nodos.



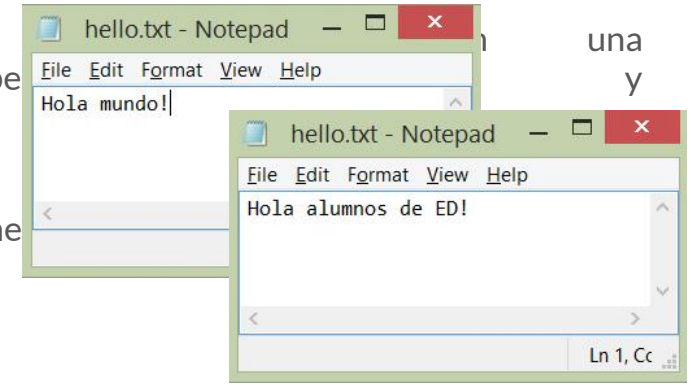


Git - Cambios y Repositorios

En un sistema, los archivos se organizan en carpetas o directorios. Cuando un archivo cambia y se guardan esos cambios, su contenido en el disco se modifica y la versión anterior en general no puede recuperarse.

Git permite trackear esos cambios una estructura que permite recupe y gestionarlos. A esta estructura se la denomina **repositorio**.

Un **repositorio git** es un conjunto que son administrados por esta he versiones y que presentan un historial de cambios común.

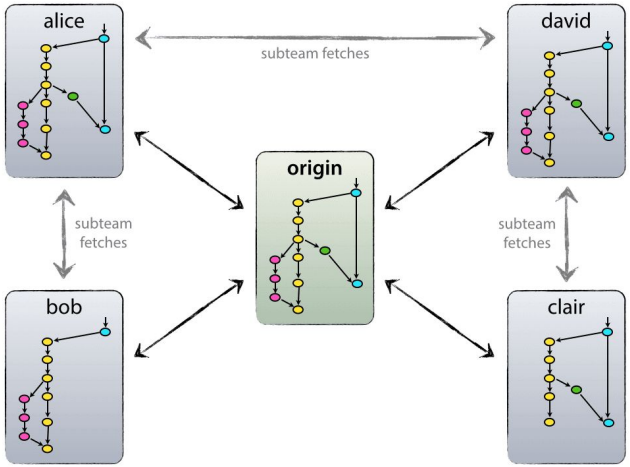


Git - Repositorios y Nodos

Un repositorio puede estar replicado (clonado) en varios sitios o nodos (ej. servidores, PCs, o incluso diferentes carpetas en una misma PC) y Git gestiona las vinculaciones entre ellos.

Se trata del **mismo** repositorio distribuido en diferentes locaciones.

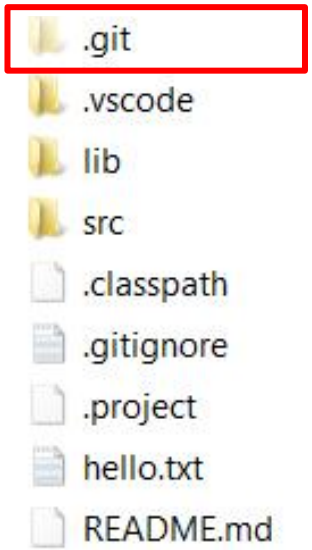
Estas réplicas se pueden conectar y sincronizar su historial de cambios.



Git - Repositorios y Carpetas Locales

En nuestra computadora podemos identificar cuando una carpeta **es repositorio Git** por la existencia de una subcarpeta oculta llamada **.git** en la misma:

- En la subcarpeta **.git**, Git guarda toda la información sobre los cambios realizados. Uno como usuario **NO** la manipula de manera directa.
- A Git no le importa dónde se ubica la carpeta repositorio, si la movemos o copiamos a otro lado, ni tampoco qué nombre tiene. Mientras tenga la subcarpeta **.git**, Git la considera un repositorio y permite gestionarla.
- Cualquier cambio que hagamos a los archivos dentro esta carpeta y las demás subcarpetas distintas de **.git**, será detectado y gestionado por la herramienta.



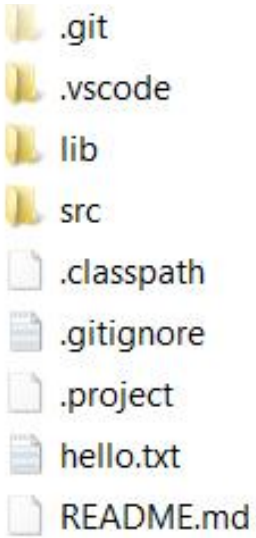
Git - Repositorios y Archivos Locales

Los archivos en la carpeta del repositorio pueden estar en dos situaciones:

- **Tracked**: se encuentran bajo la gestión de Git y su contenido puede ser modificado por la herramienta al navegar el historial de cambios
- **Untracked**: son archivos que no forman parte del repositorio. Coexisten con los archivos trackeados pero Git no los toca (sólo los reporta)

Git muestra los archivos untracked como tales. Para los archivos tracked indica si son nuevos, si han cambiado o no, o si han sido eliminados, respecto del último contenido confirmado en el repositorio.

Git sólo trackea archivos y lo hace por su ruta desde la carpeta base del repositorio. Por eso Git ignora carpetas vacías.

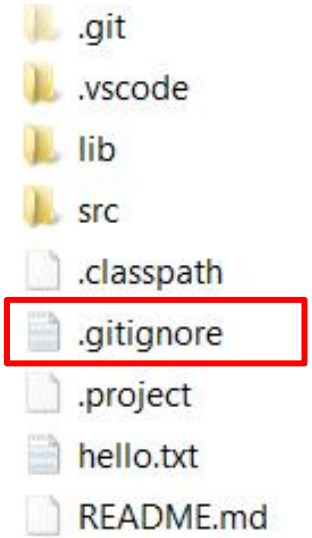


Git - Repositorios y Archivos Ignorados

Git permite ignorar archivos (no aparecen como tracked ni como untracked). Los archivos ignorados se especifican en el archivo **.gitignore** a partir de patrones coincidentes con sus nombres de archivo o rutas.

Se suelen incluir como archivos a ignorar:

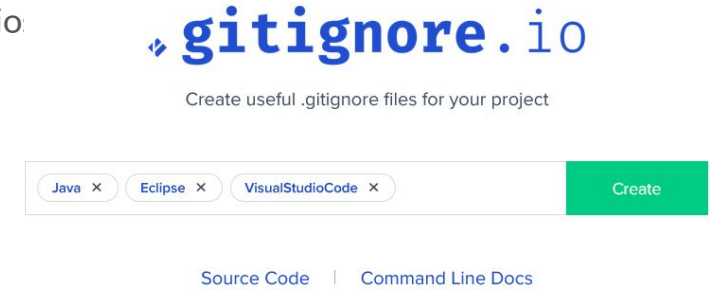
- Archivos binarios generados por el compilador
- Archivos generables por las herramientas utilizadas
- Logs y volcados de diagnóstico de procesos y herramientas
- Librerías y paquetes que se pueden descargar de otras fuentes
- Todo archivo que deseamos evitar compartir u omitir del historial del proyecto



Git - Repositorios y Archivos Ignorados

Ejemplo del contenido de un archivo **.gitignore**. El formato admite comentarios, caracteres comodín, y cada línea no comentada establece un patrón de archivos a ignorar.

Existen sitio archivos función de herramientas cada https://gitignore.io



.gitignore

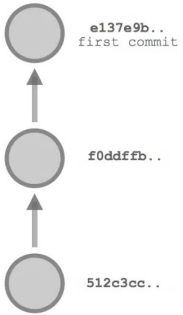
```
1  # Compiled class file
2  *.class
3
4  # Log file
5  *.log
6
7  # BlueJ files
8  *.ctxt
9
10 # Mobile Tools for Java (J2ME)
11 .mtj.tmp/
12
13 # Package Files #
14 *.jar
15 *.war
16 *.nar
17 *.ear
18 *.zip
19 *.tar.gz
20 *.rar
21
22 # virtual machine crash logs, see http://
23 hs_err_pid*
24 replay_pid*
25
```

Git - Cambios y Commits

Cuando uno o más archivos cambian (se añaden, modifican y/o eliminan) Git detecta esas diferencias con respecto al último punto en el historial del repositorio y nos permite incluirlos para formar parte del próximo punto en el historial de desarrollo.

A cada punto en el historial se lo denomina **commit**. Cada **commit**:

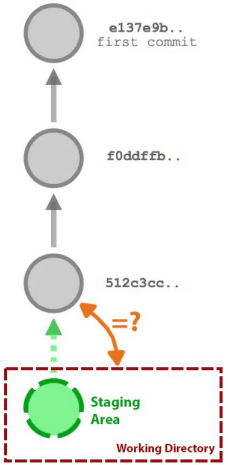
- se identifica con un código (un **hash**, concepto que veremos más adelante en la materia)
- almacena los cambios confirmados en el repositorio desde el commit previo,
- tiene un mensaje descriptivo de los cambios almacenados
- se enlaza al commit anterior, formando una estructura que representa una sucesión de cambios



Git - Working Directory vs Staging Area

De todos los cambios hechos respecto del último **commit**, Git nos permite elegir un subconjunto para incluir en el próximo. Esto da lugar a tres áreas o estados de trabajo:

- **Working Directory**: contiene los archivos con los cambios que hemos realizado. Git marca las diferencias respecto del último commit incluyendo archivos untracked.
- **Staging Area**: incluye los cambios que formarán parte del próximo commit (sobre archivos tracked). Git marca las diferencias respecto del último commit.
- **Commit**s: Son los cambios una vez confirmados. Pasan a ser permanentes en el historial.



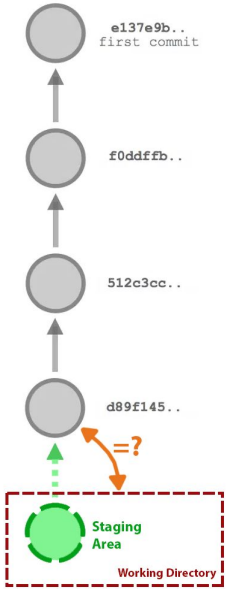
Git - Working Directory vs Staging Area

Al cambiar uno o más archivos (sin importar el editor o IDE usado), estos se encuentran en el **Working Directory** (las carpetas del repositorio).

Al añadir dichos cambios al **Staging Area** (**git add**), los archivos que no estaban trackeados pasan a estarlo (se incorporan al index de Git) y los cambios quedan listos para formar parte del próximo commit.

Al confirmar los cambios (**git commit**), el contenido del Staging Area pasa a ser el próximo commit que se añade al historial. Y el ciclo continúa con nuevos cambios en el Working Directory, algunos de los cuales los pasamos al Staging Area, y luego confirmamos como un nuevo commit.

También es posible descartar cambios y volver a la versión del último commit.

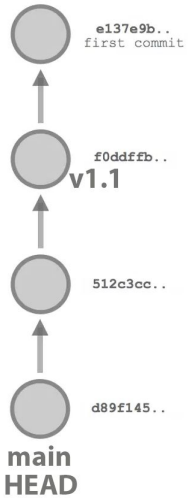


Git - Tags y Branches

Git permite identificar commits con etiquetas más amigables para el humano, (por ej. para indicar versiones específicas). En Git se llaman **tag**s.

A una sucesión de commits se lo denomina rama o **branch**. Git las representa con una etiqueta al final de la misma, que se va desplazando a medida que se agregan commits.

- La rama principal de todo repositorio normalmente se denomina **main**.
- La rama de trabajo actual se denomina **HEAD (**con mayúsculas). En el ej. **main** y **HEAD** coinciden indicando que la rama de trabajo actual es **main**.
- Se puede mover HEAD a commits que no sean extremos de rama (detached mode) para resolver ciertas tareas.

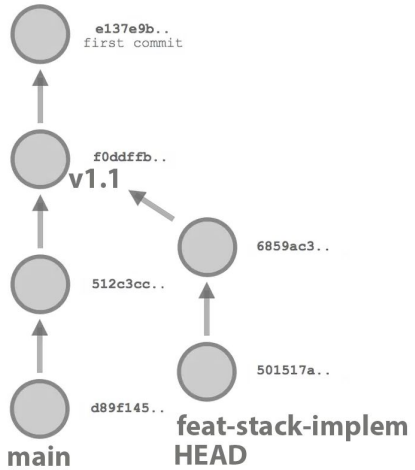


Git - Branches y Checkouts

La potencia de las ramas es que permiten bifurcar (**git branch**) la historia y progresar en sucesiones de cambios alternativas independientes.

Ej. **feat-stack-implem** es una rama que se deriva a partir de la rama main en el commit **f0ddffb..** con una sucesión de cambios independientes. **HEAD** marca que la rama de trabajo actual es **feat-stack-implem**.

El cambio de rama se denomina checkout (**git checkout**) y ocasiona que Git modifique los archivos trackeados de nuestra carpeta para llevarlos al estado del último commit de la rama destino (**feat-stack-implem** en el ej.). Si luego, por ej., hacemos checkout de **main**, volverá a modificar los archivos para reflejar las versiones guardadas en el último commit de **main**.

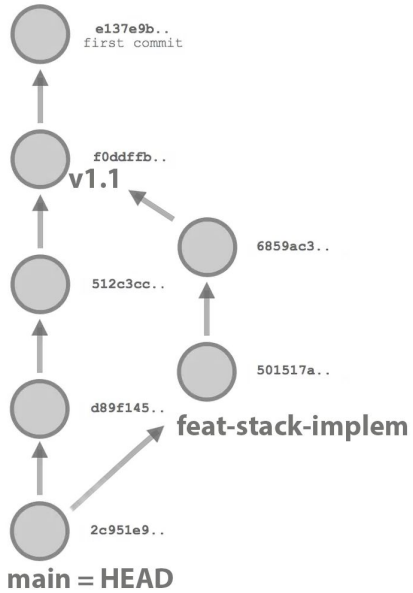


Git - Branches y Merges

Tener varias ramas en paralelo permite avanzar con el desarrollo de manera independiente.

Los cambios de varias ramas pueden combinarse mediante la operación de mezla (**git merge**). Git integra los cambios de ambas ramas en un único **commit** que los sintetiza y que unifica las secuencias de cambios hasta ese momento independientes.

Las mezclas son relativas a la rama actual. La rama elegida siempre se mezcla en **HEAD**. En el ejemplo (luego de hacer **checkout main**), los cambios de **feat-stack-implem** se han mezclado en la rama **main**. Podría darse el caso contrario y no son situaciones equivalentes. No es lo mismo mezclar la rama A en la B, que mezclar la rama B en la A.



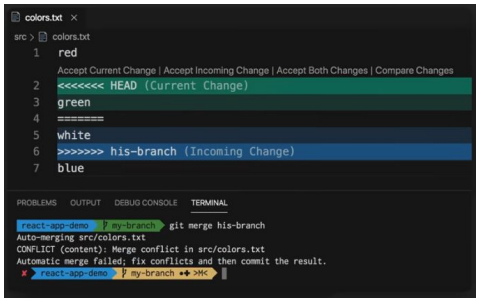
Git - Branches, Merges y conflictos

Git trabaja con los archivos en modo texto. Considera su contenido línea a línea, y caracter a caracter. Si bien se puede utilizar con archivos binarios, no es lo ideal.

Si los cambios ocurren en archivos distintos, o en líneas distintas de los mismos archivos, Git confirma las versiones más actuales de cada línea, para cada archivo de ambas ramas (una mezcla exitosa).

Si por el contrario, hay cambios diferentes sobre las mismas líneas, en uno o varios archivos, que son incomparables en el tiempo (no hay un cambio que se haya hecho antes que el otro en la misma sucesión de commits), surgen **conflictos** que el usuario debe resolver (el usuario debe determinar cómo quedarán los archivos para poder completar la mezcla).



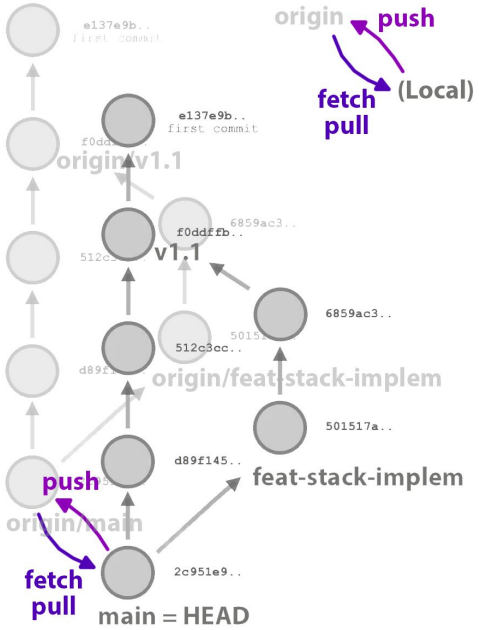


Git - Remotes

Además del historial local, git administra las copias conectadas del repositorio en otros nodos (PCs, servidores, GitHub). Esas otras locaciones se denominan **remotes**.

Si bien Git no establece ninguna regla acerca del rol que cada nodo tiene en el conjunto, es usual denominar **origin** (origen) al nodo que mantiene la copia principal del repositorio.

- En repositorios clonados de GitHub, **origin** suele designar a la réplica del repositorio que se aloja en los servidores de GitHub.
- Ciertos servicios de despliegue de aplicaciones que utilizan Git, definen **remotes especializados** que al enviar commits a ciertas ramas, construyen, testean y despliegan el software desde el repositorio.

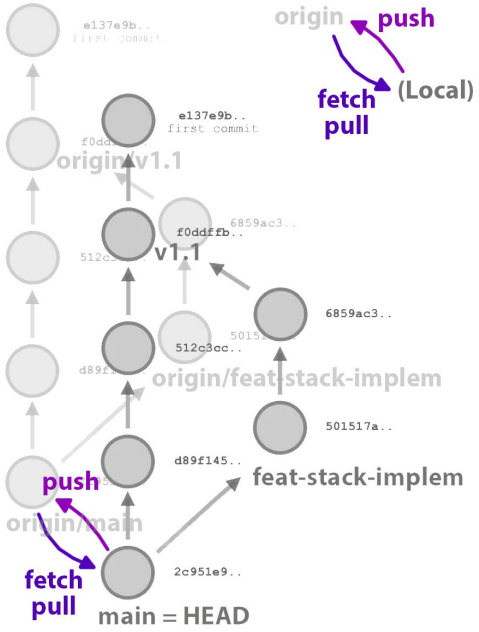


Git - Remotes: fetch, pull y push

Los nombres de los **remotes** son locales y designan cada uno de los nodos con los que se establece conexión para el mantenimiento del repositorio. Se pueden definir tantos remotes como se necesiten.

Las ramas locales normalmente se replican en los remotes, (**main** en **origin/main**, **feat-stack-implem** en **origin/feat-stack-implem**, etc.) aunque no necesariamente están sincronizadas. Git provee operaciones para actualizar cambios entre nodos:

- **git fetch**: trae los cambios desde un remote al Working Directory (el usuario decide qué hacer con esos cambios)
- **git pull**: trae los cambios desde un remote y los mezcla en la rama actual (equivale a hacer **git fetch + git merge**)
- **git push**: envía los commits locales de la rama actual al remote elegido

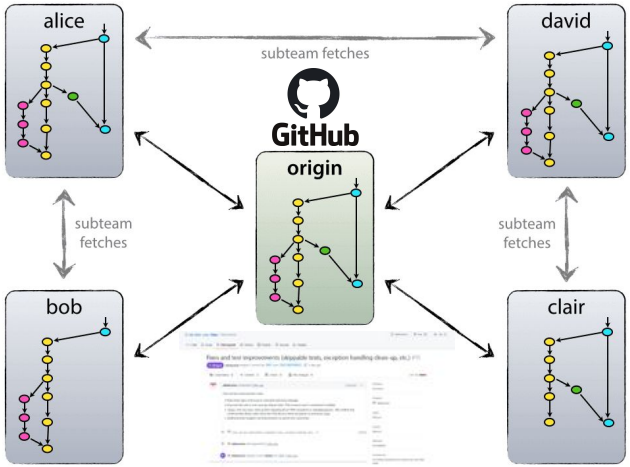


Git - Remotes y GitHub

Cuando usamos GitHub, **origin** es la réplica del repositorio mantenida por GitHub. Cada repositorio que se clona mantiene una conexión con **origin**.

Git permite conexiones directas a los otros nodos (configurando cada remote) ya que es descentralizado, pero no es lo usual. Con GitHub, el desarrollo se centraliza en la plataforma y todos los participantes bajan y suben cambios desde y hacia **origin**. Las ramas se mezclan en GitHub y no localmente (salvo en caso de conflictos).

GitHub es un nodo Git con espacio de hosting al que se le añade una interface web, y un conjunto de servicios, como por ej. issue tracker, gestión de proyectos y equipos, CI/CD, etc.



Flujo de trabajo con Git/GitHub

Flujo de trabajo con Git/GitHub



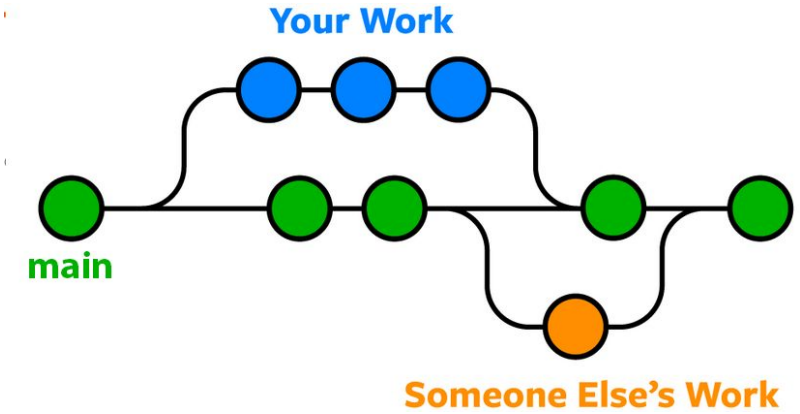
El flujo de trabajo básico es el siguiente:

1. [opcional/recomendado] Crear una rama específica (**git branch**) para el trabajo a realizar
2. Editar archivos/introducir cambios (Git es independiente del editor/IDE utilizado)
3. Añadir los cambios a confirmar al área de staging (**git add**). Agrupar cambios que tengan relación (que se puedan describir en una breve oración y usarla como descripción del commit).
4. Cuando los cambios están listos confirmarlos (**git commit**) en la **rama actual**
5. Enviar/publicar los cambios (y/o la rama) en **origin** (**git push**)
6. Al completar la secuencia de cambios: integrar y revisar los cambios (GitHub Pull Request, resolución de conflictos, etc.).

Flujo de trabajo con Git/GitHub

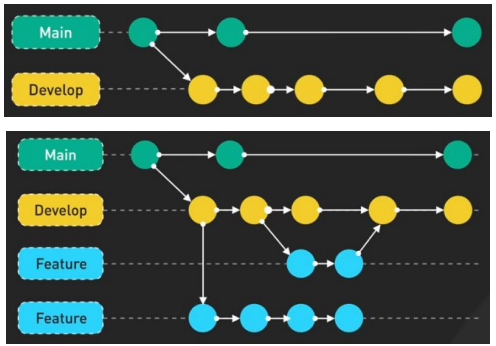
Si bien se puede trabajar haciendo commits sobre main, el uso de ramas diferentes permite:

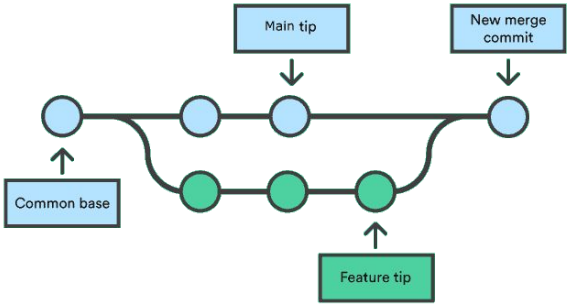
- que cada desarrollador trabaje de manera independiente de los demás
- definir ramas diferentes para cada característica o trabajo que se hace
- descartar ramas sin afectar la rama principal
- posponer la resolución de conflictos (si hubiese) hasta la integración/mezcla de esas ramas

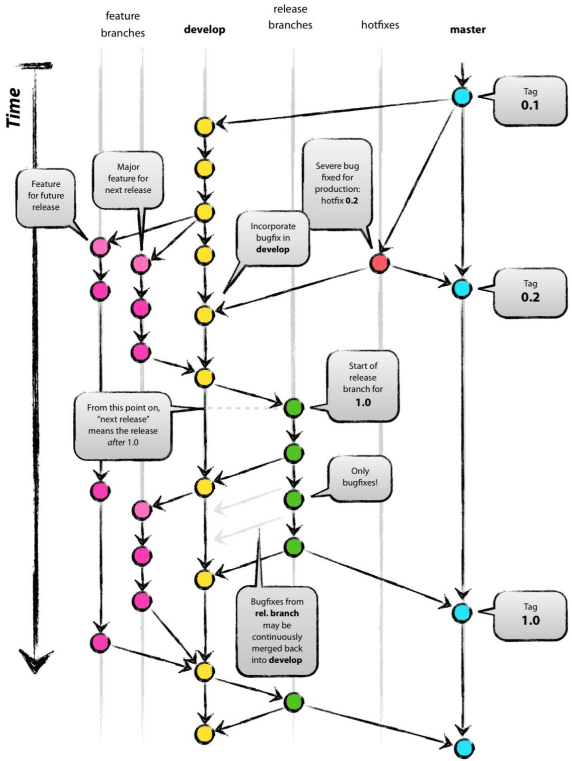


Branching models

Hay numerosas formas de organizar las ramas en un proyecto. Cada organización define su **branching model**. En la materia nos alcanza con poder gestionar los cambios de manera efectiva. Sean libres de gestionar las ramas como mejor les parezca.





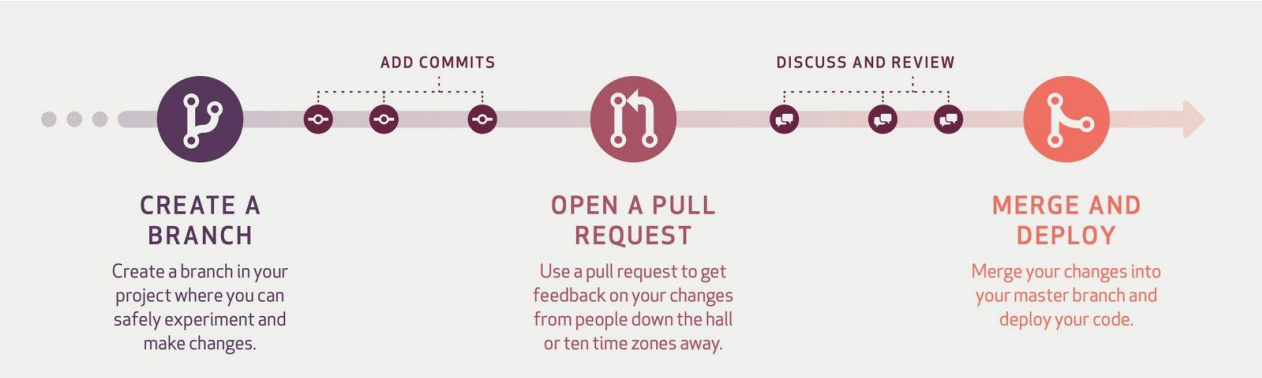


GitHub Workflow - Pull Requests



GitHub incorpora el concepto de **Pull Request (PR)**: consiste en solicitar vía la interface web que el nodo en GitHub haga un **pull** hacia la rama destino (**fetch** + **merge**) para integrar nuestros cambios.

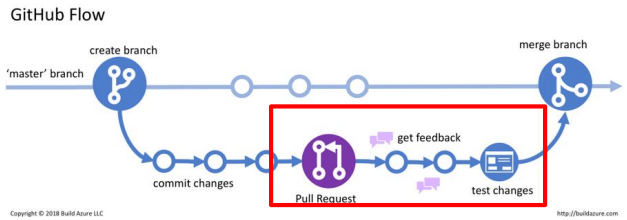
Para abrir un PR se necesita una rama específica “pusheada” a GitHub (por eso recomendamos usarlas).

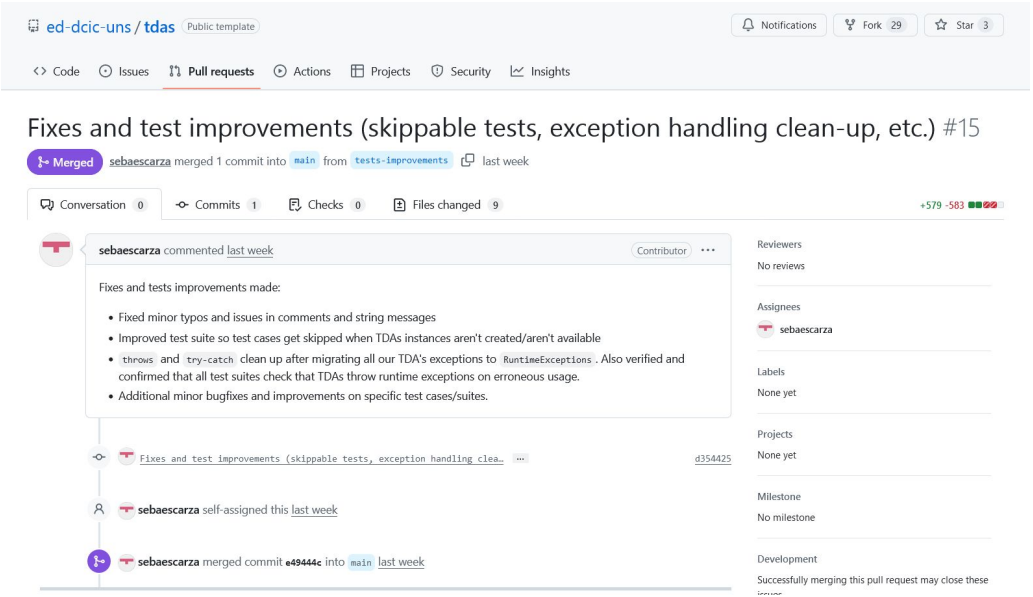


GitHub Workflow - Pull Requests

Pull Requests (PRs):

Se abre una interface para analizar los cambios, debatirlos, testearlos, hacer code review, meter nuevos cambios y ajustes, y cuando todo está listo, mezclarlos en la rama destino.





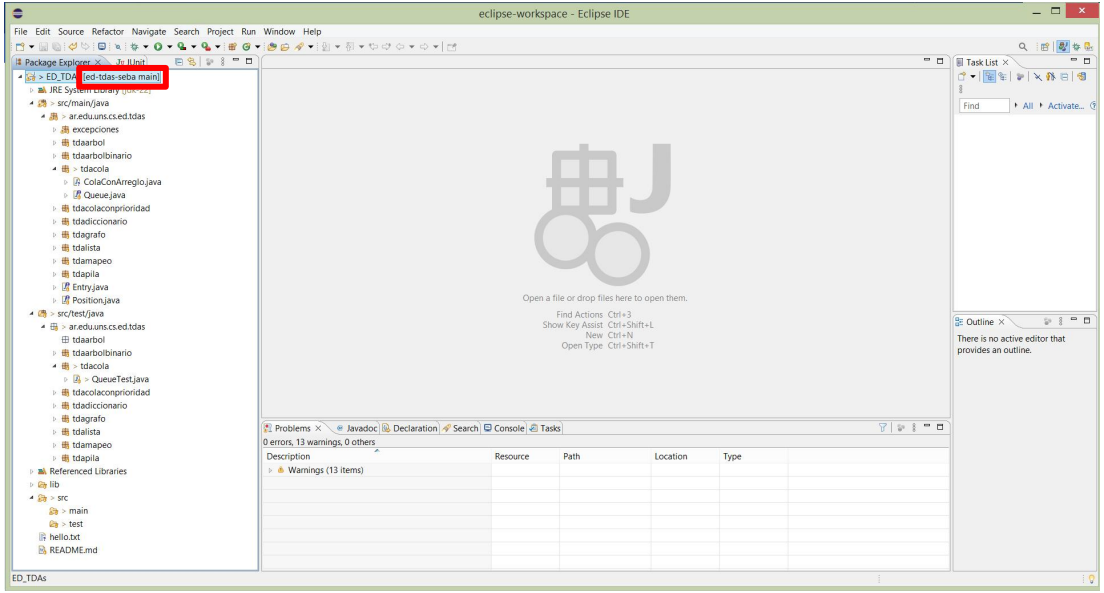
Usando Git y GitHub en Eclipse

Usando Git y GitHub en Eclipse

Para usar Git desde Eclipse hay que abrir un proyecto (Import… > Existing project into workspace) que sea repositorio Git.

Eclipse no nos muestra la subcarpeta (oculta) **.git** ni ningún archivo que comience con punto.

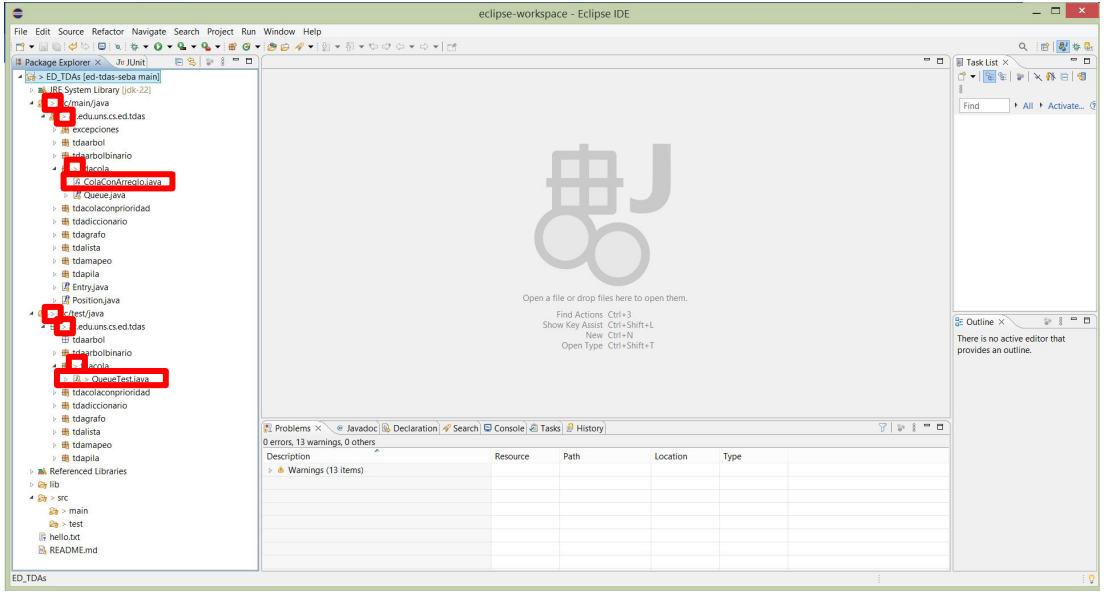
Por defecto Eclipse muestra el nombre del repositorio y la rama actual en el proyecto.



Usando Git y GitHub en Eclipse

Por defecto Eclipse también muestra los cambios mediante el signo **>** en el explorador de paquetes.

A cada archivo Untracked le añade un signo **?** en la parte inferior derecha del ícono, y a cada archivo tracked le añade un subícono de almacenamiento.

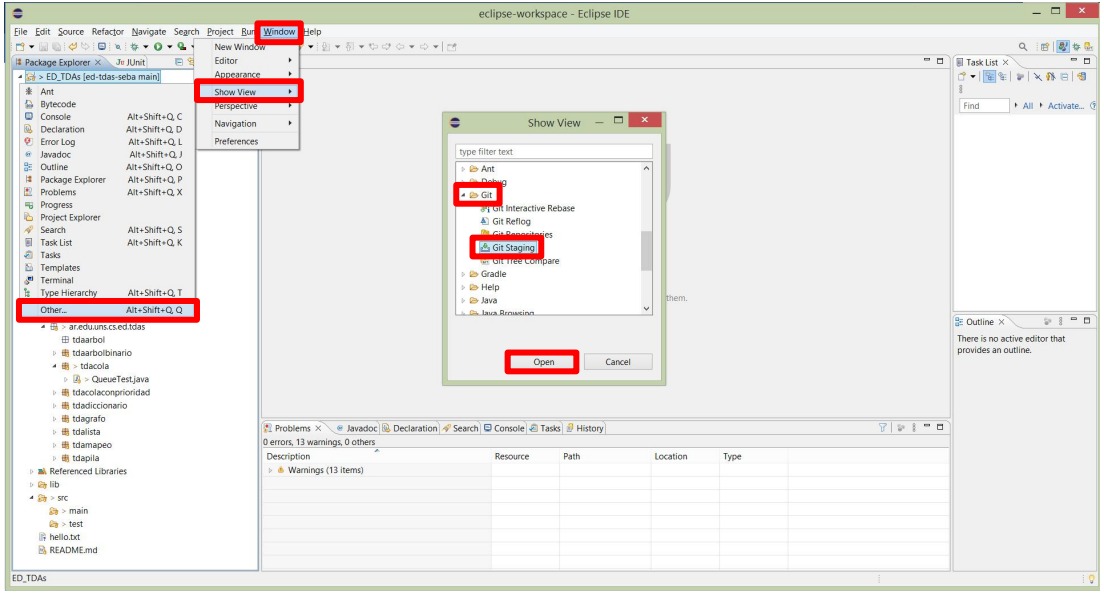


Usando Git y GitHub en Eclipse

Para trabajar con Git desde Eclipse y tener mayor claridad del estado de WD/Staging conviene habilitar la vista **Git Staging**:

1. Menú **Window > Show view > Other**.
2. Seleccionar en el recuadro de diálogo **Git > Git Staging**
3. Presionar el botón **Open**

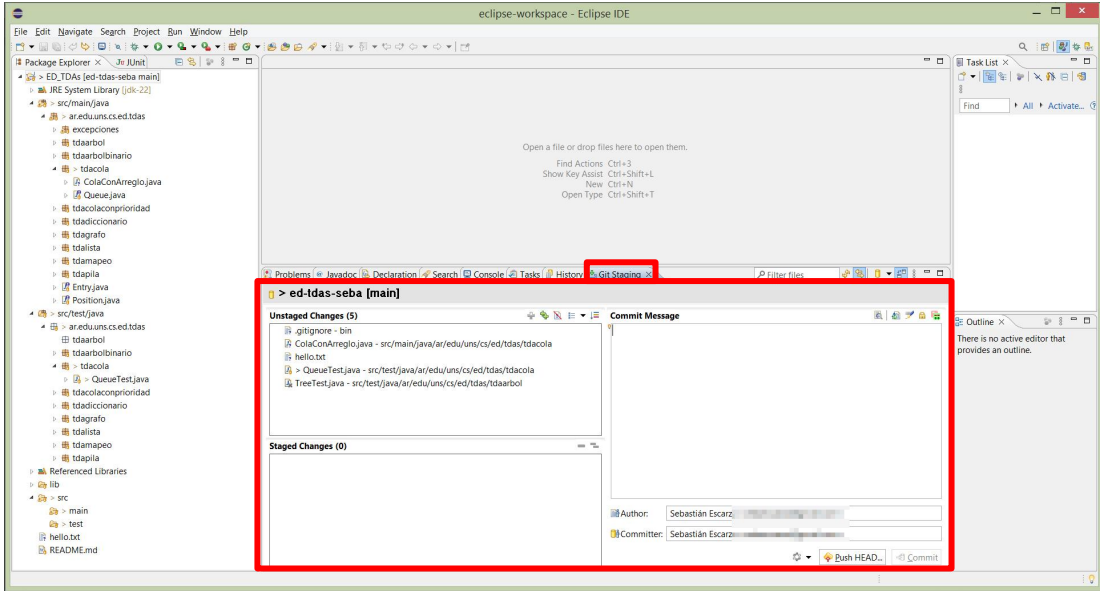
Hay otras vistas de Git disponibles pero no son esenciales para un flujo de trabajo básico.



Usando Git y GitHub en Eclipse

Esta opción habilita el panel de **Git Staging** donde se muestra:

- El Working Directory (**Unstaged Changes**)
- El Staging Area (**Staged Changes**)
- Un área para definir el mensaje para el próximo commit
- Un botón para hacer **Commit**
- Un botón para hacer Push de la rama actual (**Push HEAD**).

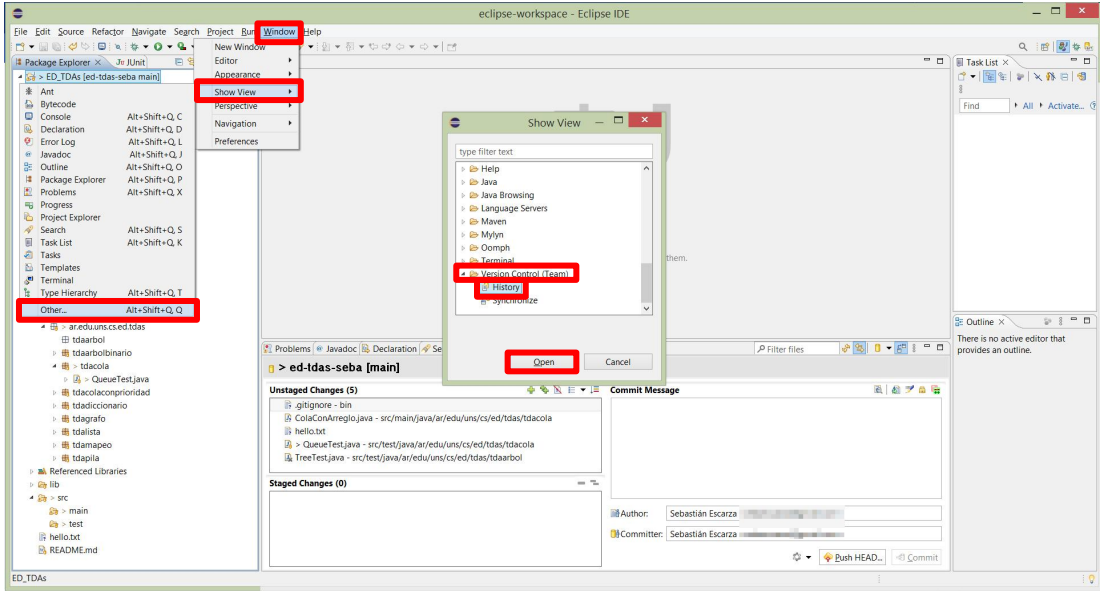


Usando Git y GitHub en Eclipse

También conviene habilitar la vista **History**:

1. Menú **Window > Show view > Other**.
2. Seleccionar en el recuadro de diálogo **Version Control (Team) > History**
3. Presionar el botón **Open**

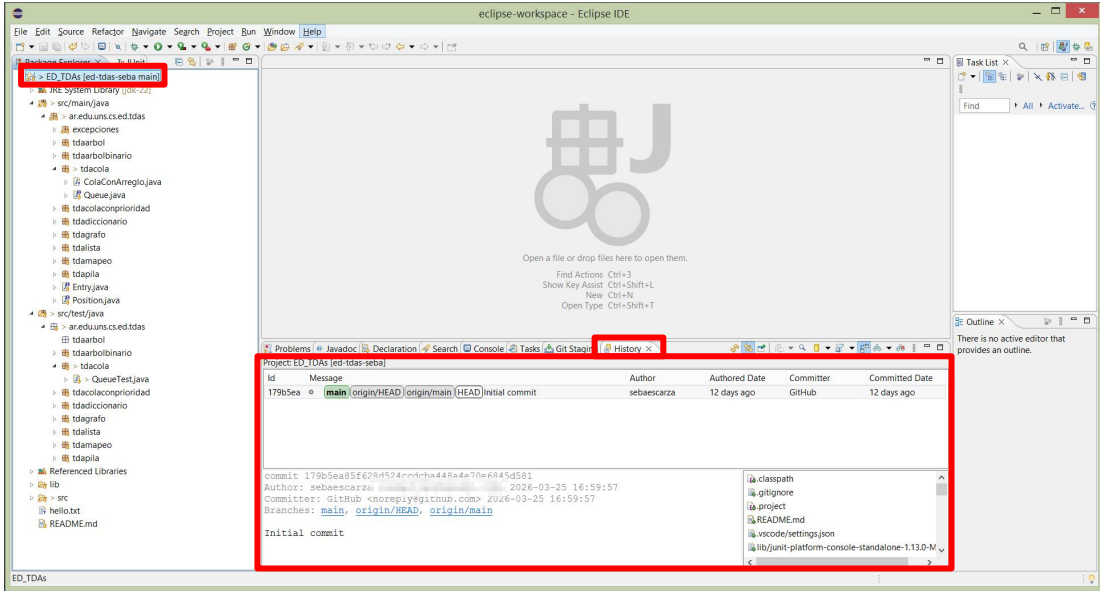
Hay otras vistas de Team disponibles pero tampoco son esenciales para un flujo de trabajo básico.



Usando Git y GitHub en Eclipse

Esta opción habilita el panel **History** donde se puede observar el historial de cambios a partir del elemento seleccionado en el Package Explorer.

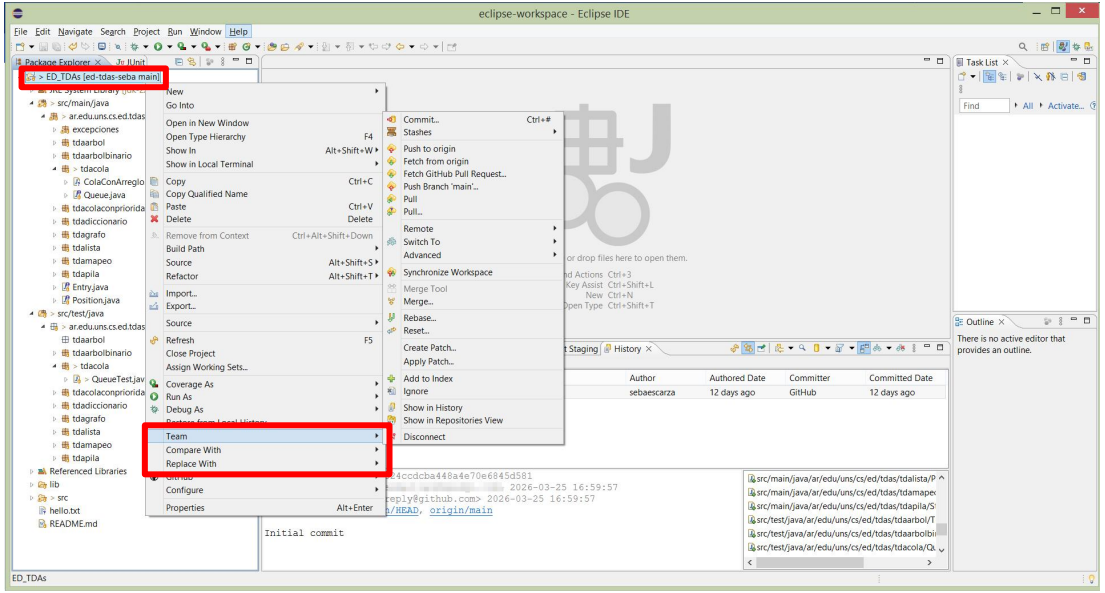
En el ejemplo se muestra el historial de todo el proyecto.



Usando Git y GitHub en Eclipse

También, haciendo click derecho sobre el proyecto, en el menú **Team** se encuentran numerosas opciones para ejecutar comandos de Git.

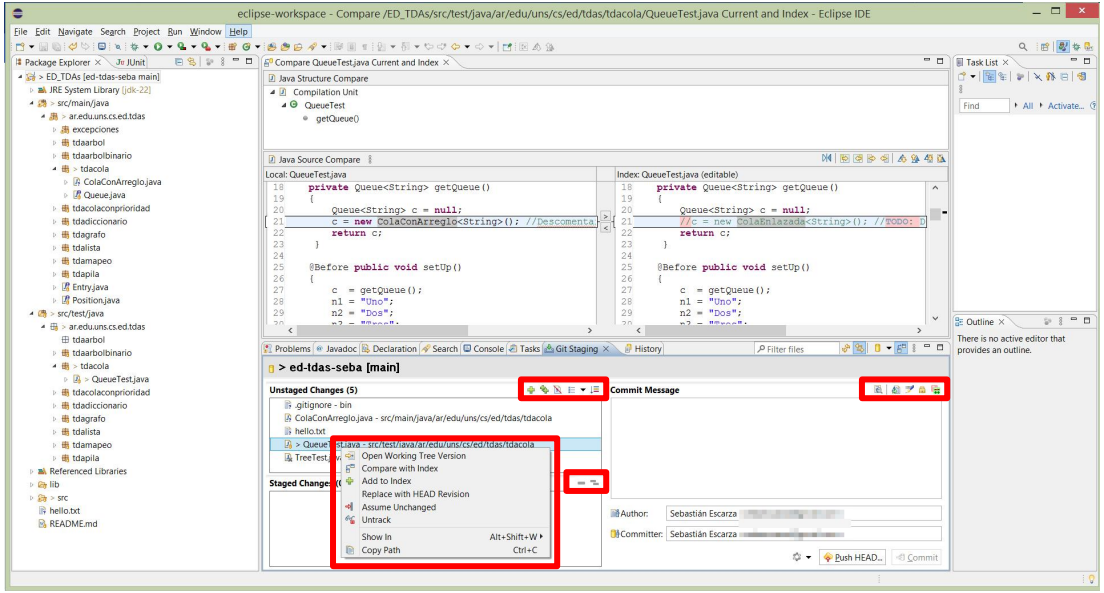
Las opciones **Compare With** y **Replace With** permiten hacer operaciones para comparar los cambios actuales con otros en el historial, y también reemplazarlos.



Usando Git y GitHub en Eclipse

Volviendo al panel **Git Staging** se tienen opciones para:

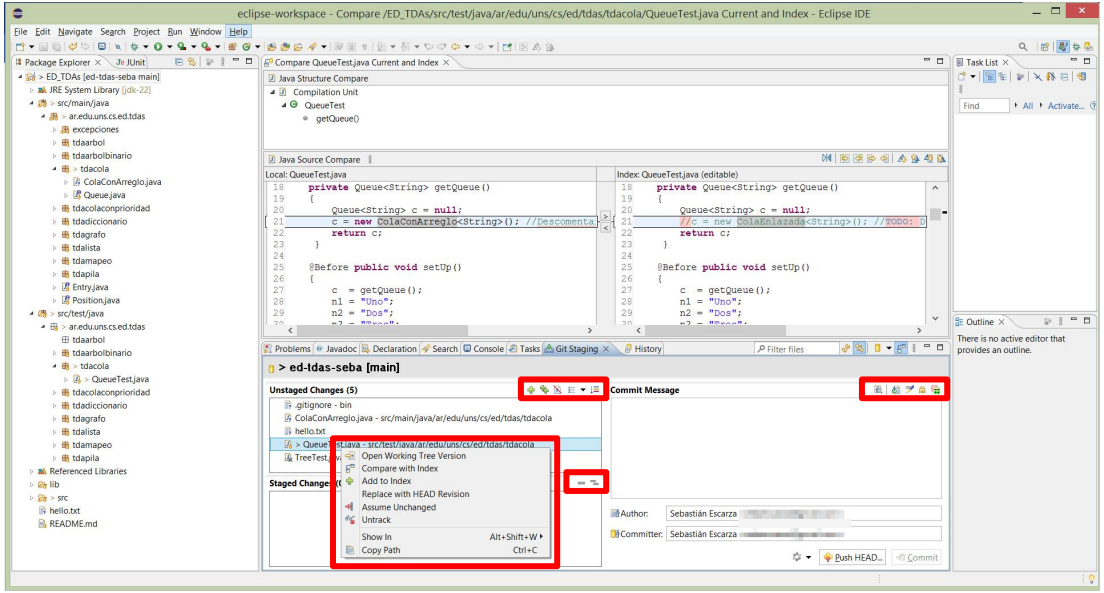
- Pasar archivos del Working Directory a Staging y visceversa
- Enemndar el mensaje del último commit
- Con click derecho en un cambio se abre un menú contextual que permite abrir el archivo, compararlo contra la versión en Index, deshacer los cambios (replace with HEAD revision), etc.



Usando Git y GitHub en Eclipse

Volviendo al panel **Git Staging** se tienen opciones para:

- Pasar archivos del Working Directory a Staging y visceversa
- Enemndar el mensaje del último commit
- Con click derecho en un cambio se abre un menú contextual que permite abrir el archivo, compararlo contra la versión en Index, deshacer los cambios (replace with HEAD revision), etc.

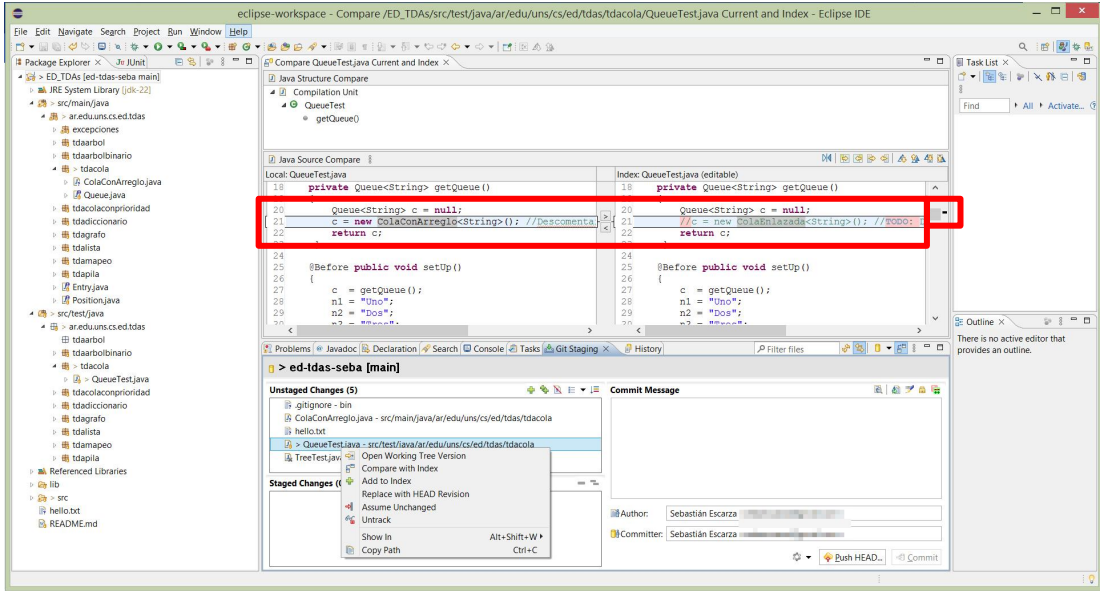


Usando Git y GitHub en Eclipse

En la vista comparativa (**git diff**) se muestran línea a línea los cambios en el archivo examinado.

El título de la vista indica qué se está comparando.

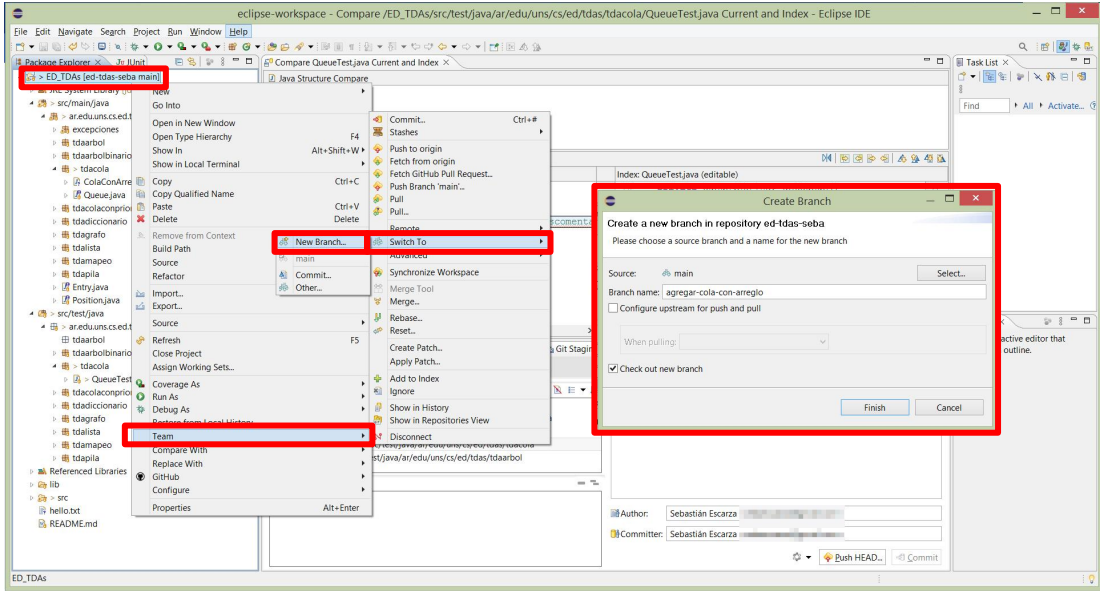
En el ejemplo se puede ver cómo se ha modificado la línea 21 en el archivo QueueTest.java.



Usando Git y GitHub en Eclipse

Antes de confirmar algunos de los cambios vamos a crear una nueva rama para los mismos.

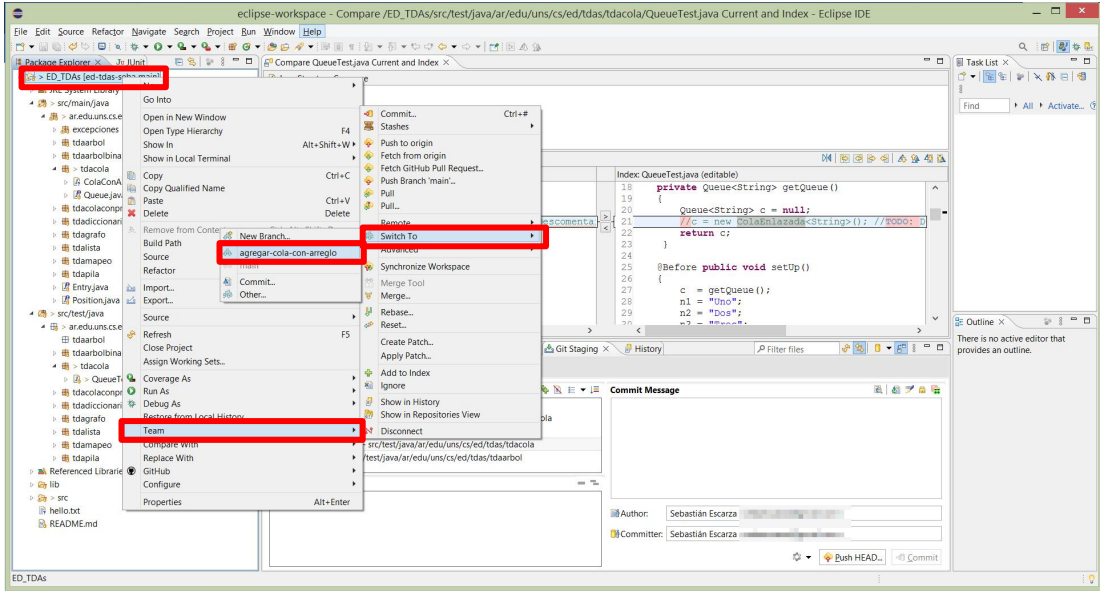
Para ello hay que hacer click derecho en el proyecto, ir al menú **Team > Switch to... > New Branch...** (**git branch**), darle el nombre, dejar tildado el check box para que el sistema automáticamente cambie (**git checkout**) a la nueva rama, y clickear **Finish**.



Usando Git y GitHub en Eclipse

Si la rama ya existe y queremos retomar el trabajo, desde este mismo menú podemos seleccionarla.

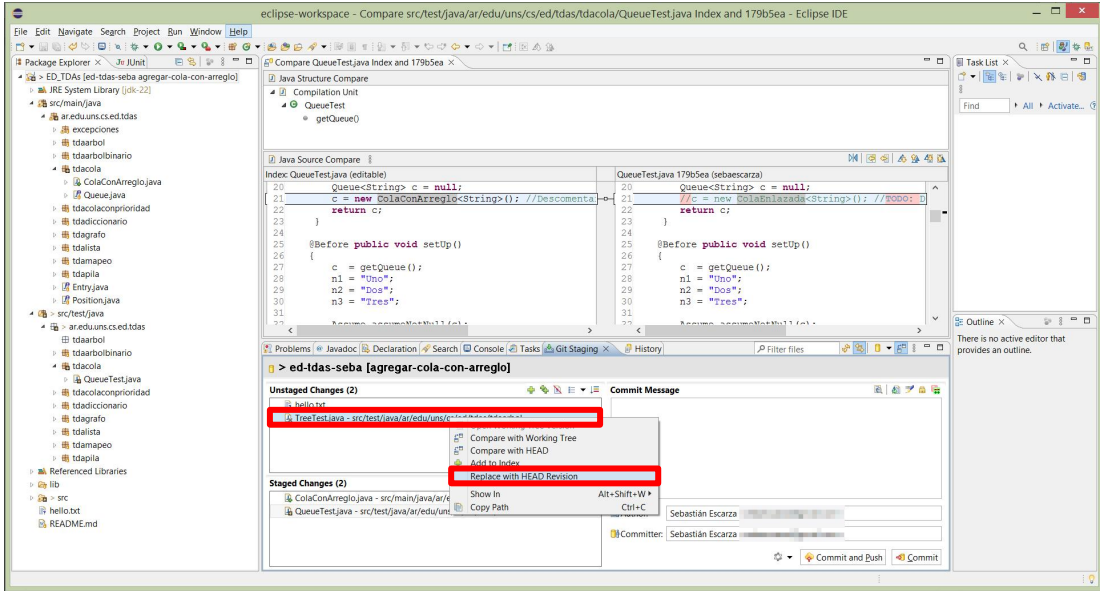
Se recomienda hacer **fetch from origin** desde el menú para actualizar el listado de ramas, y una vez hecho el checkout a la rama elegida, hacer **pull** para obtener los últimos commits de la misma. Esto es similar a utilizar la opción **Synchronize Workspace** del menú **Team**.



Usando Git y GitHub en Eclipse

Una vez en la rama correcta (y actualizada), luego de hacer cambios y antes del próximo commit, se analizan los cambios hechos (vista de cambios - **git diff**) para:

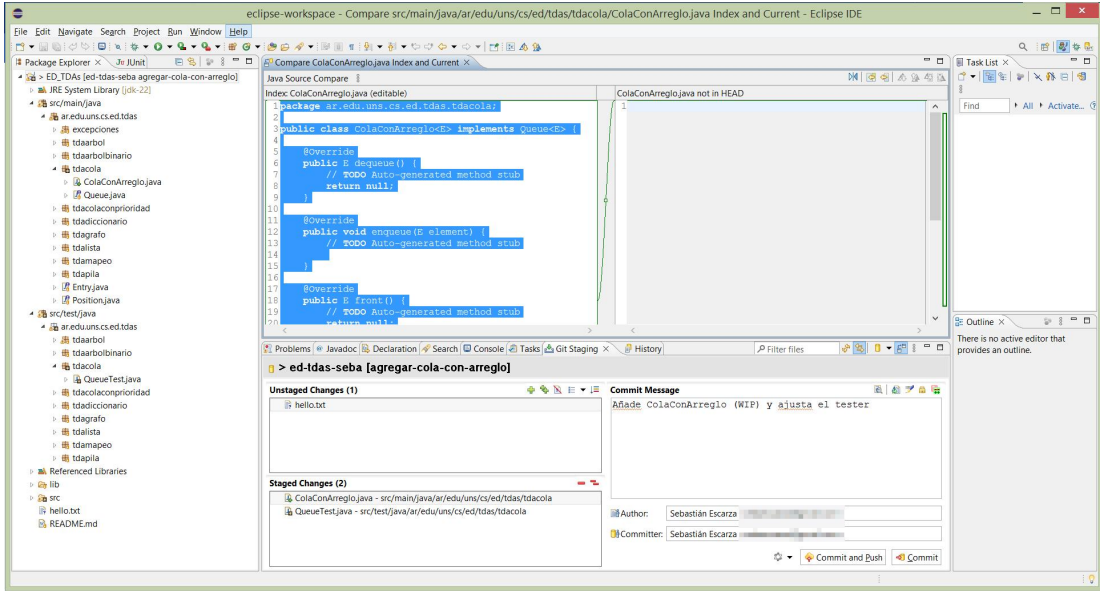
- Determinar si son aplicables (si no hay cambios que no hicimos/no deseamos aplicar)
- Se pasan a Staging sólo aquellos que formarán parte del próximo commit (los demás se dejan en Working Directory para más adelante, o si no aplican se descartan).



Usando Git y GitHub en Eclipse

En el ejemplo:

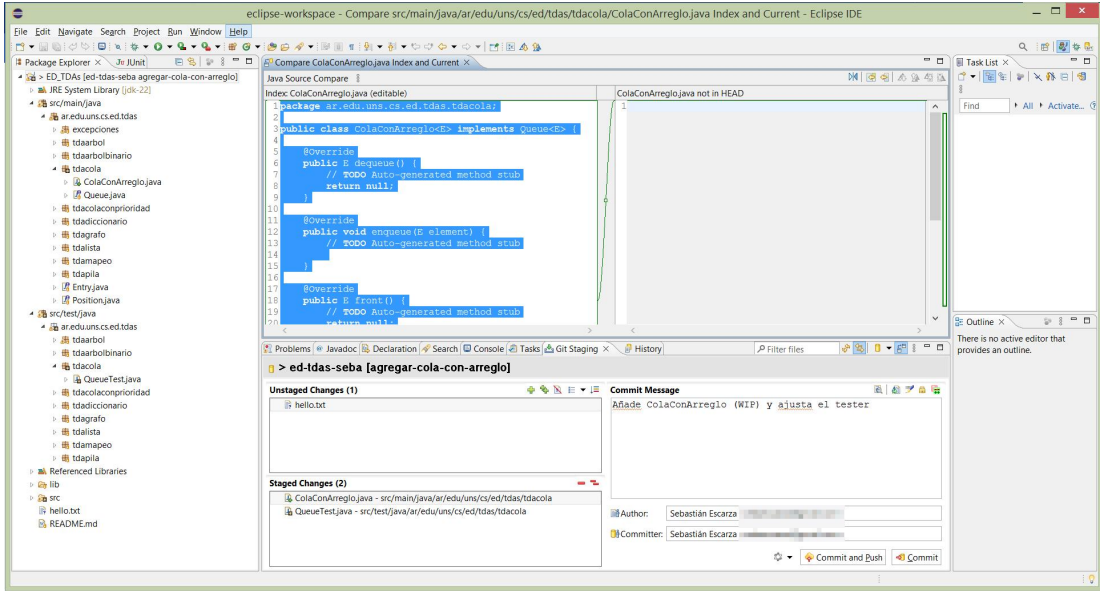
- Se añadió QueueTest.java al Staging
- Se restauraron TreeTest.java que se había eliminado “por error” y .gitignore que fue modificado por Eclipse
- Se analizaron los cambios en ColaConArreglo.java: todas las líneas son nuevas (el archivo no existía).
- Se dejó hello.txt afuera.
- Se tipeó un mensaje indicando el contenido del commit.



Usando Git y GitHub en Eclipse

El mensaje del commit tiene que ser lo más claro y breve posible y dar una idea de qué se trata (será lo que describa al commit en el historial).

En el ejemplo, el código de ColaConArreglo.java es apenas un esqueleto autocompletado para implementar la interface y no está terminado. En estos casos se puede añadir “WIP” (por Work In Progress) para indicarlo.



Usando Git y GitHub en Eclipse

En los mensajes de commit la primer línea se considera título, y las demás forman el cuerpo. Los commits más simples se describen sólo con el título, mientras que los demás pueden requerir más detalle.

Algunas recomendaciones (Eclipse controla algunas y tiene preview del mensaje):

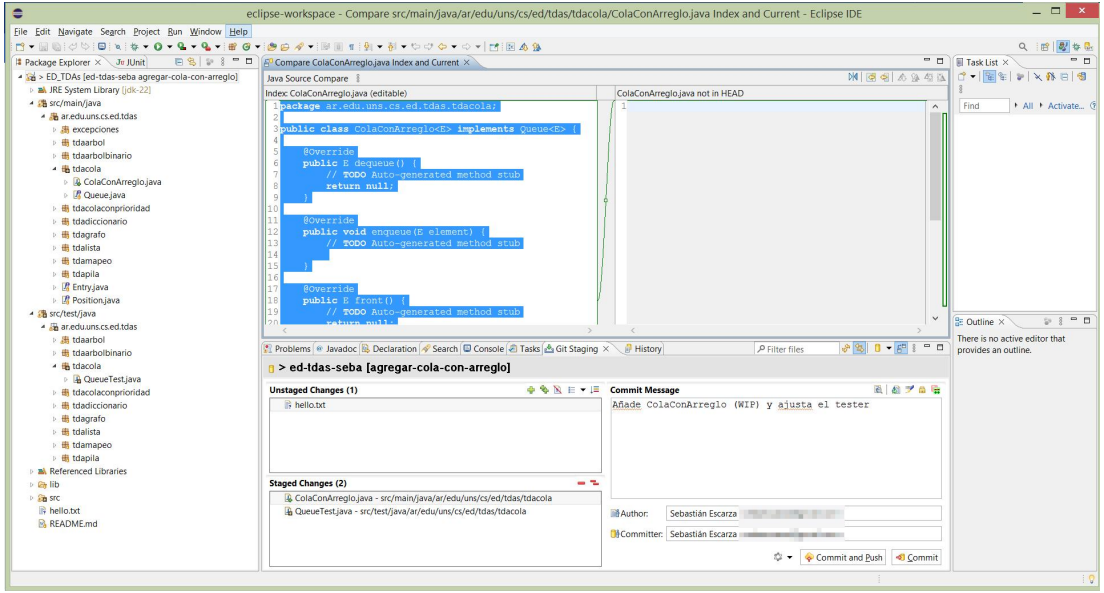
- No superar los 50 caracteres en el título
- Comenzar el título con una letra mayúscula y no usar punto final
- Dejar una línea en blanco (2da línea) para separar título de cuerpo
- No superar los 72 caracteres en cada línea del cuerpo
- En inglés se sugiere usar el modo imperativo en el título (en castellano a veces queda raro)
- Explicar qué cambia y si es necesario el por qué. No detallar qué líneas cambian (eso se puede mirar si hace falta con **git diff**)
- Respetar las convenciones del repositorio o grupo de desarrollo (si están establecidas)

Usando Git y GitHub en Eclipse

Antes de presionar Commit (**git commit**) y meter estos cambios en el historial:

- Revisar que estamos en la rama correcta
- Revisar que el mensaje refleja el contenido del commit.

Usar buenos nombres de ramas, y buenas descripciones en los commits son la clave para tener un historial de versiones práctico y fácil de gestionar.

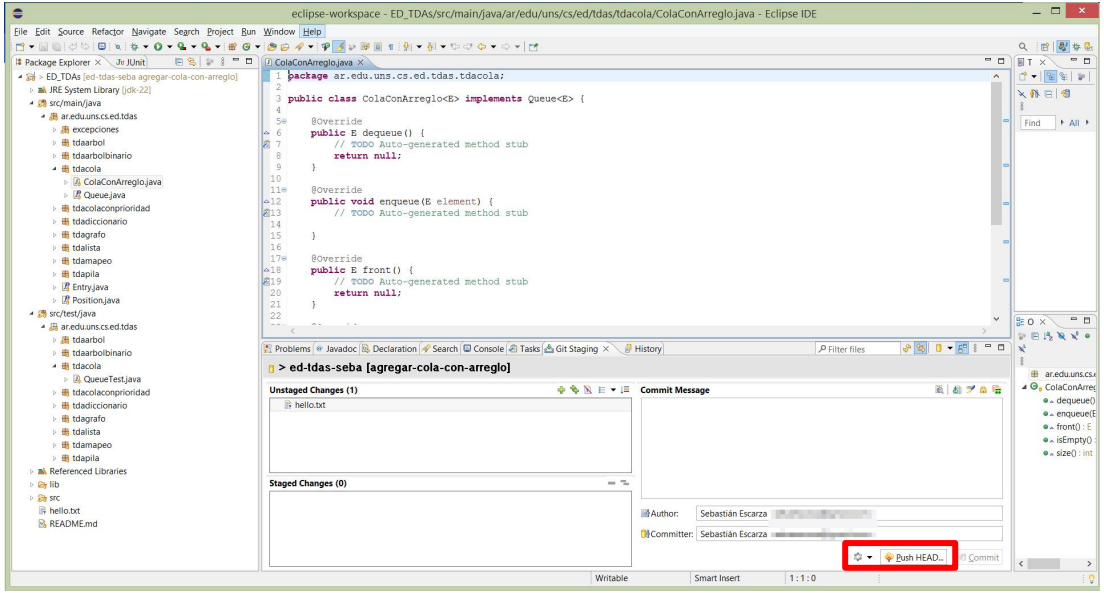


Usando Git y GitHub en Eclipse

Luego de hacer el commit, los archivos cambiados ya no muestran diferencias.

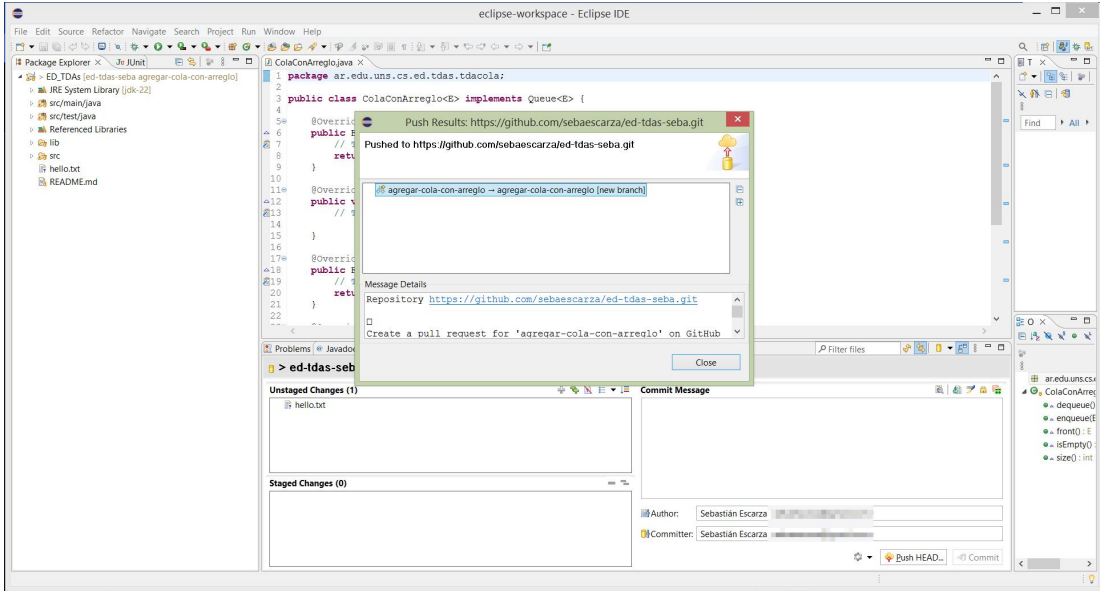
Tenemos un commit en una rama local que sólo existe en nuestra versión local del repositorio. Eclipse nos ofrece la opción de hacer **Push HEAD...** desde la solapa Git Staging, o **Push Branch...** desde el menú **Team** para subir la rama al remote con los commits realizados sobre ella.

Esto equivale a hacer **git push** (necesitaremos tener el access token de GitHub configurado para esto).



Usando Git y GitHub en Eclipse

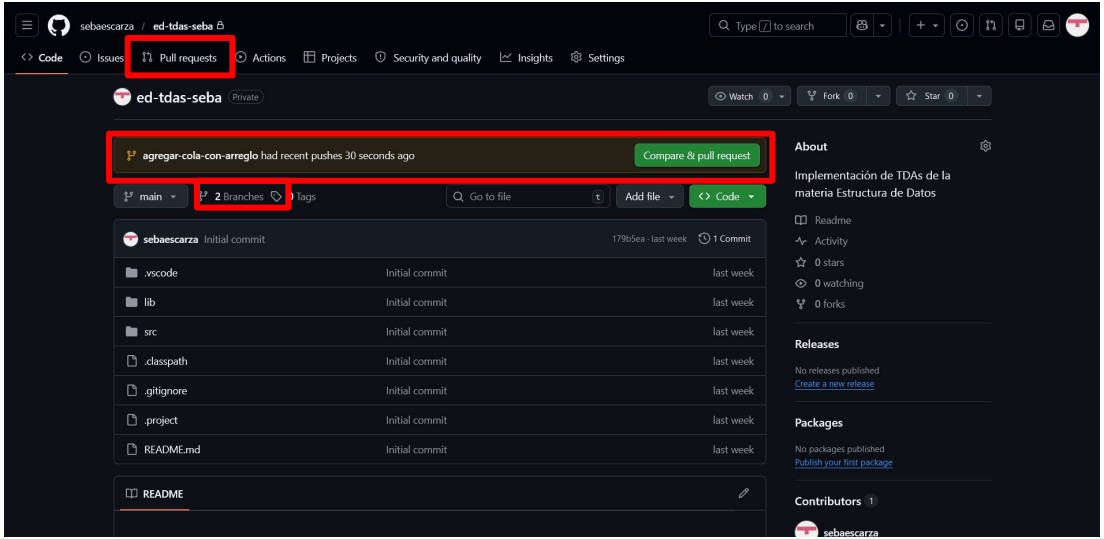
Confirmando la acción, git creará en el remote (en GitHub en este caso) una rama con igual nombre asociada a ésta. La rama remota asociada se denomina **upstream branch**, y la local se dice que trackea a la remota (es decir, que sigue sus cambios).



Usando Git y GitHub en Eclipse

La rama publicada ahora puede verse en GitHub. La plataforma nos muestra las ramas pusheadas recientemente para permitirnos abrir un Pull Request (PR) sobre ellas de manera más directa, e integrar los cambios. Sino se muestra en la solapa Code, se puede abrir el PR desde Pull requests, o desde Branches.

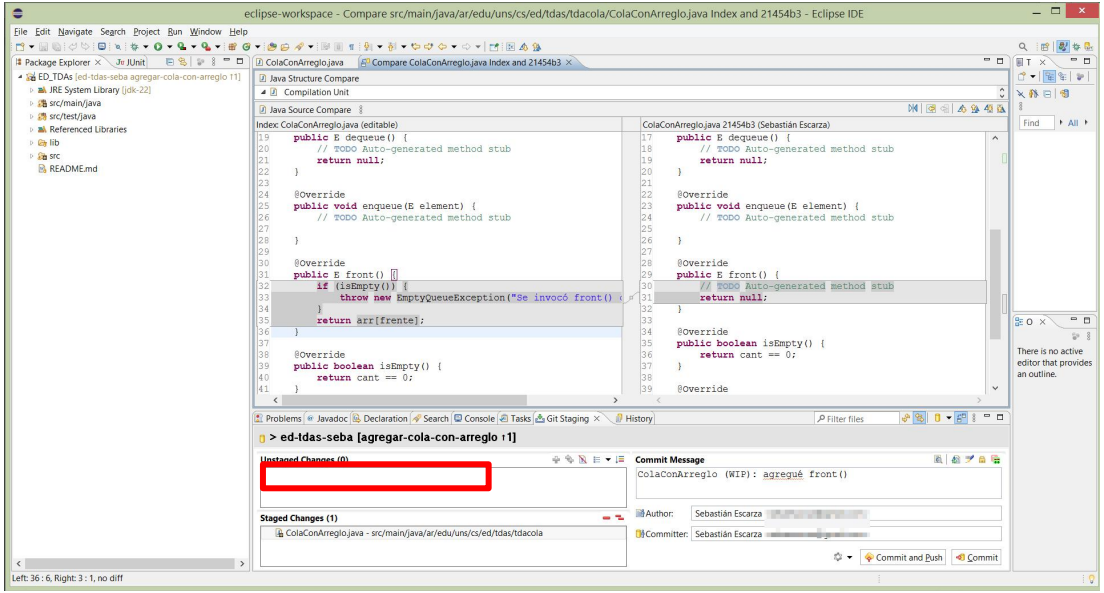
El trabajo sobre la rama se continúa haciendo cambios y commits, y cada tanto haciendo push para mantener la rama remota actualizada. Para integrar esos cambios en la rama de base (main), se puede abrir el PR.



Usando Git y GitHub en Eclipse

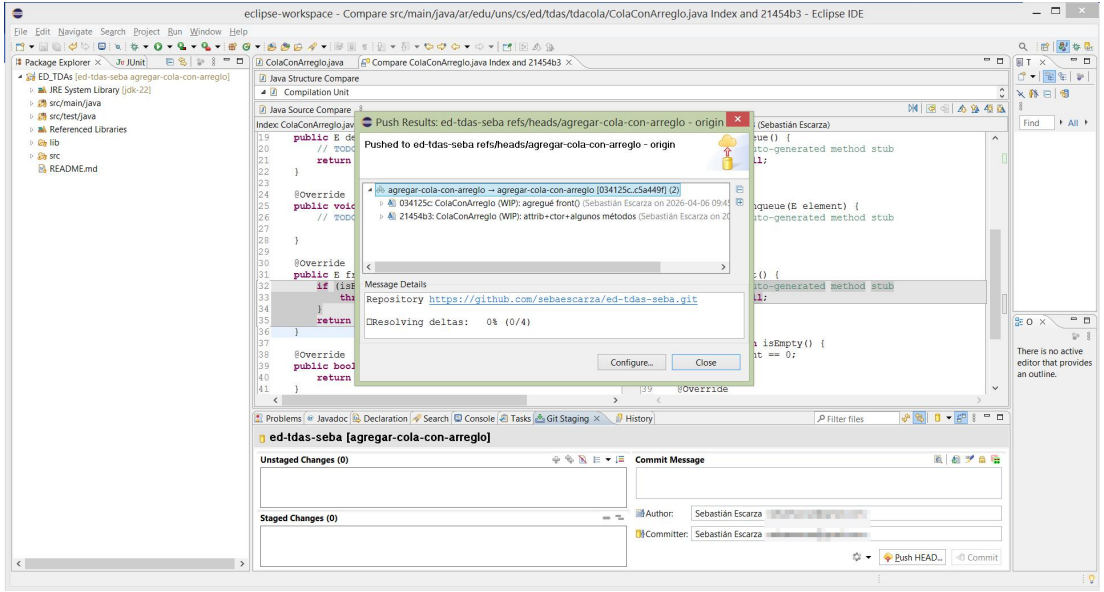
En el ejemplo, antes de abrir el PR agregamos algunos cambios y commits locales más, y limpiamos el Working Directory eliminando el archivo hello.txt de ejemplo.

Se recomienda trabajar con el Working Directory lo más limpio posible para centrarnos sólo en lo que queremos cambiar y tener menos problemas al momento de hacer **checkout**, **merge**, o **pull**. Si tenemos algún archivo con cambios sin confirmar que se vea afectado por estas operaciones, Git dará error y se negará a completarlas.



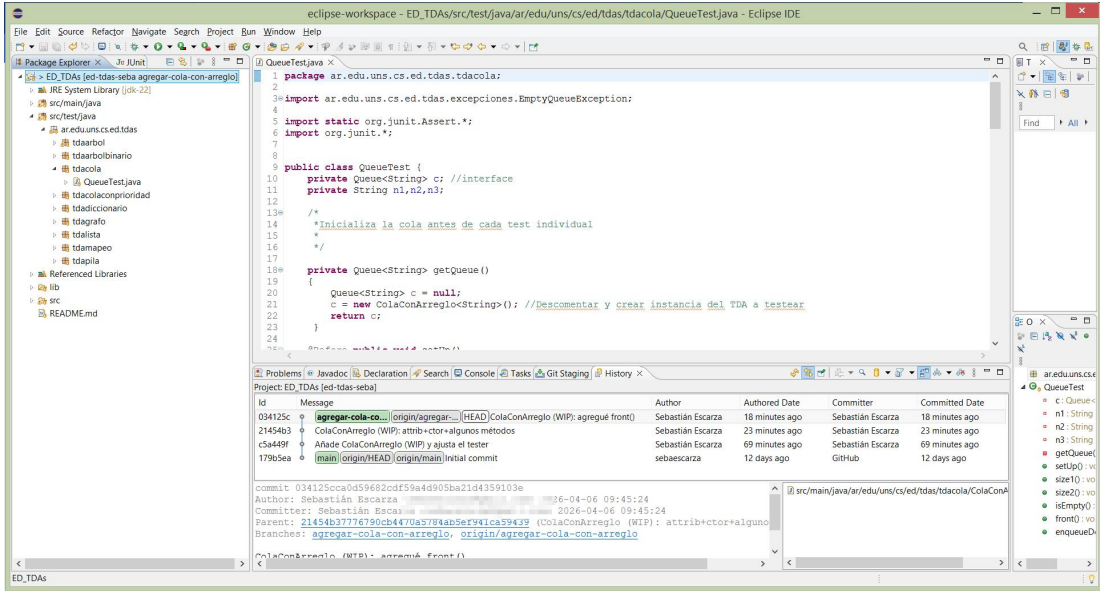
Usando Git y GitHub en Eclipse

Como próximo paso, volvemos a hacer **push** y subimos a GitHub los 2 commits añadidos (actualizando la rama remota con esos cambios).



Usando Git y GitHub en Eclipse

En la solapa History podemos tener una representación visual del historial con los commits realizados y (si bien en el ejemplo aún no se aprecia) cómo se conectan las diferentes ramas (tanto locales como remotas).



Usando Git y GitHub en Eclipse

A partir de este punto, y en general, los cambios:

- se trabajan y testean localmente,
- se añaden al Staging Area (**git add**),
- se confirman (**git commit**),
- luego se publican en el remote (**git push**),
- y finalmente, se revisan y mezclan en GitHub.

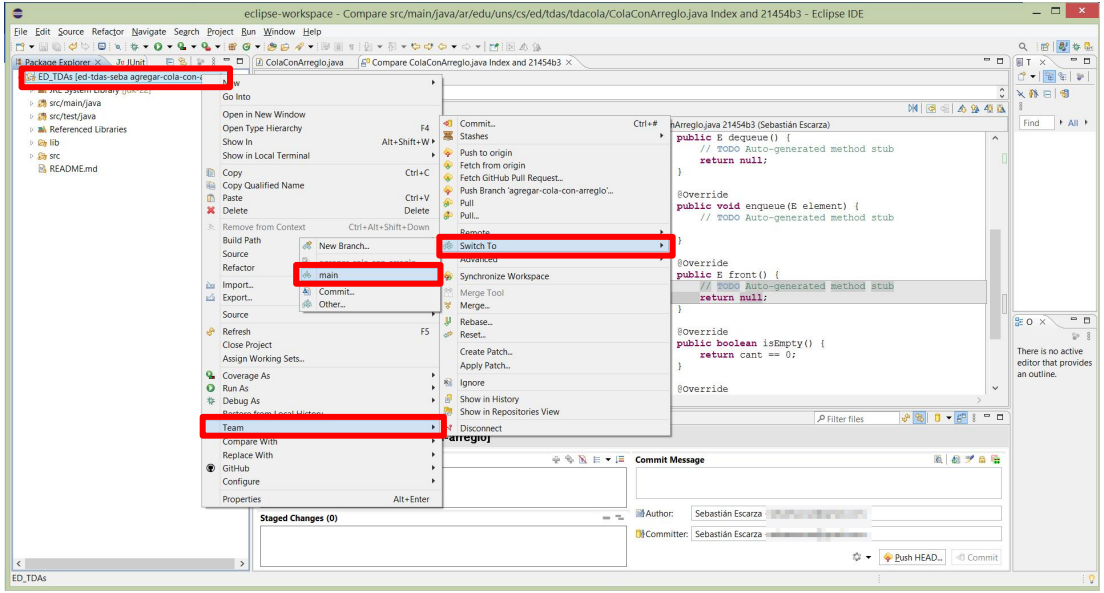
Luego de mezclar cambios en GitHub, no olvidar de actualizar (**git pull**, **git fetch**) el repositorio local con los cambios (de hecho la mejor práctica es siempre hacerlo frecuentemente, o al cambiar de rama antes de continuar trabajando).

Usando Git y GitHub en Eclipse

Una vez terminado el trabajo en una rama, podemos cambiar a otra y continuar con el desarrollo.

Git nos actualiza en nuestra carpeta local los archivos trackeados para llevarlos al estado en que estaban en el punto del historial elegido.

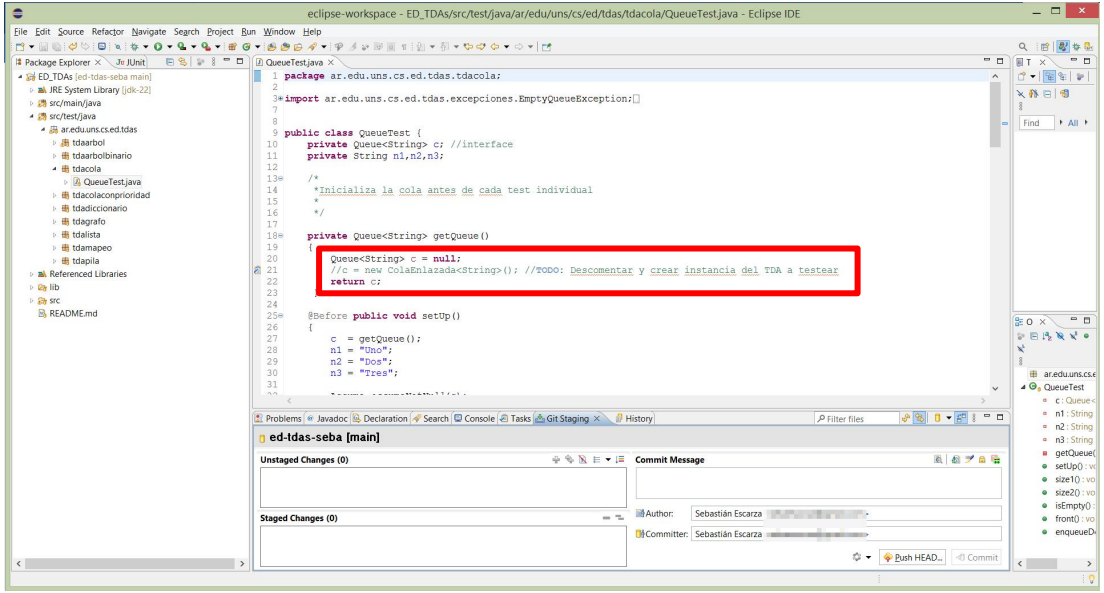
Si por ejemplo, hacemos **checkout** de la rama main…



Usando Git y GitHub en Eclipse

… el sistema nos lleva al punto del historial y nos muestra los archivos tal y como los dejamos en el último commit de main.

En repositorios compartidos, si hacemos **pull** podemos traernos la última versión de origin/main del remote y los cambios que algún otro usuario pudo estar haciendo.

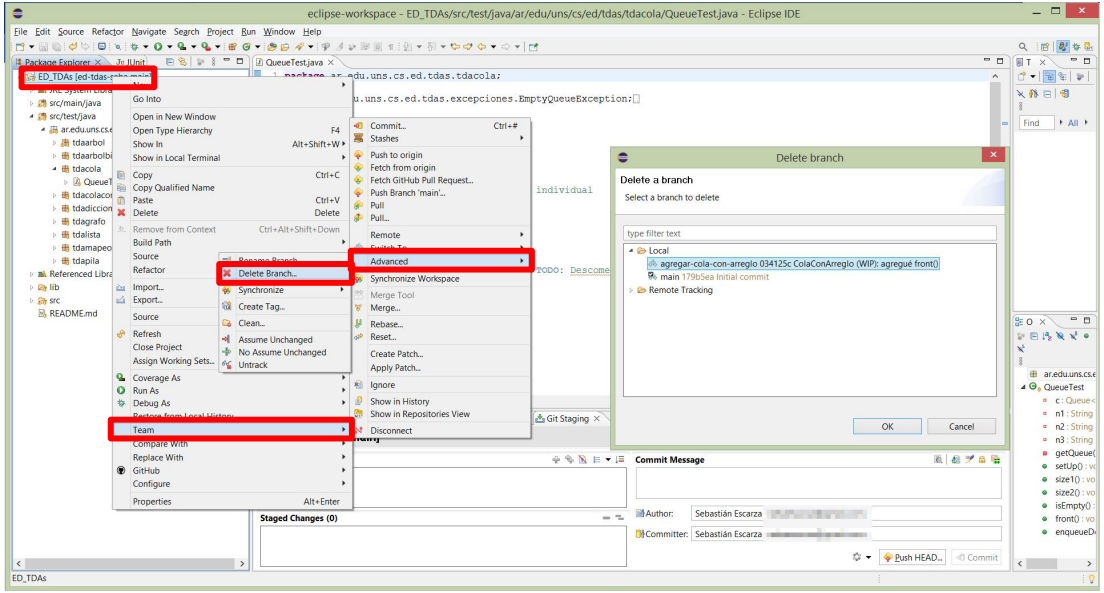


Usando Git y GitHub en Eclipse

También, una vez revisados y mezclados los cambios en GitHub, si las ramas locales han cumplido su función y ya no son necesarias pueden eliminarse.

Esto se logra yendo al menú contextual del proyecto (click derecho), eligiendo:

Team > Adanced > Delete Branch...

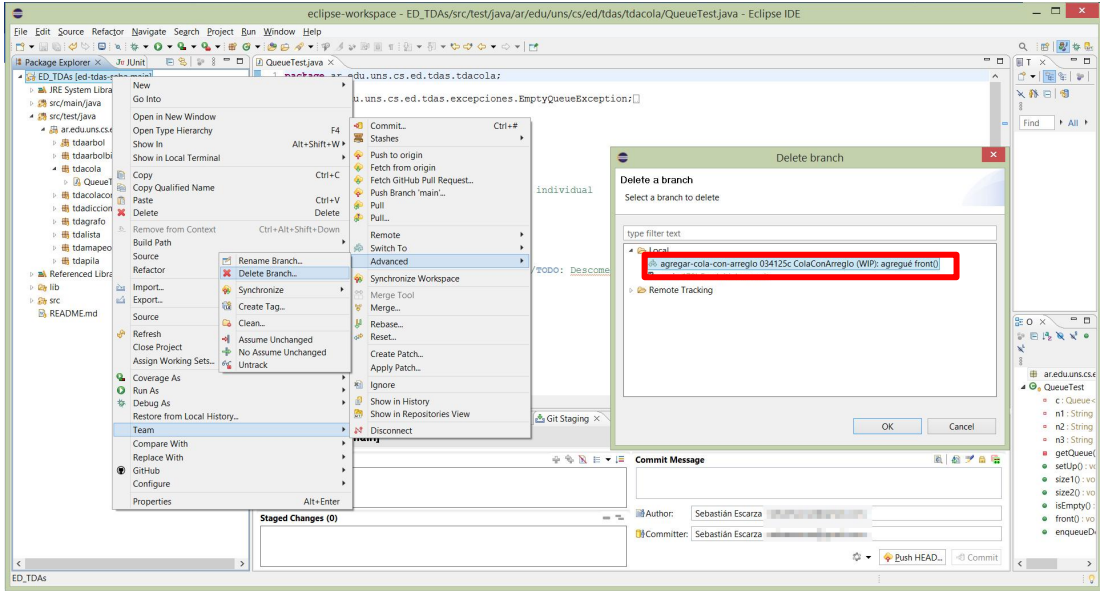


Usando Git y GitHub en Eclipse

Luego se selecciona la rama local a eliminar en el cuadro de diálogo.

En el ejemplo sólo habíamos creado una: agregar-cola-con-arreglo

La idea de las ramas es que tengan un propósito concreto y agrupen los commits relacionados con cierta funcionalidad, corrección o refactor. Una vez mezcladas se pueden eliminar.

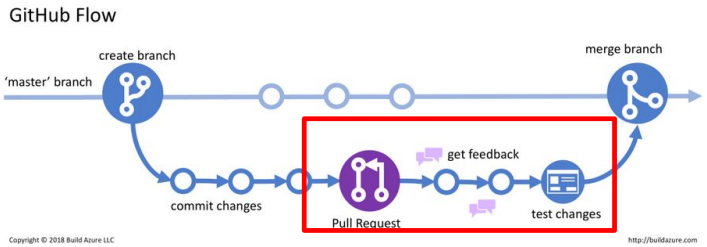


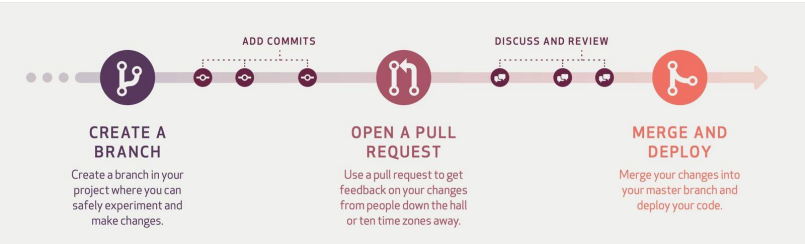
Gestionando Pull Requests y mezclando cambios en GitHub

GitHub - Pull Requests (PRs)

Una vez publicada una rama en un remote de GitHub la plataforma ofrece la posibilidad de abrir un Pull Request (solicitud de pull en el remote de GitHub) para mezclar esos cambios.

Uno puede elegir como rama target una rama del repositorio, o si el repositorio es un fork, una rama del repositorio upstream.





GitHub - Pull Requests (PRs)

Un Pull Request (PRs) es una solicitud para que GitHub intente un **pull** (**fetch** + **merge**) de los cambios de la rama en el repositorio remoto. Previo a proceder a la mezcla (**merge**) de los cambios, GitHub ofrece una interface de seguimiento para hacer revisión de código, introducir mejoras y discutir los cambios.

GitHub también tiene seguimiento de tareas (issue tracker) y normalmente los PRs están asociados y buscan resolver determinados issues (no trabajaremos con esto en la materia).

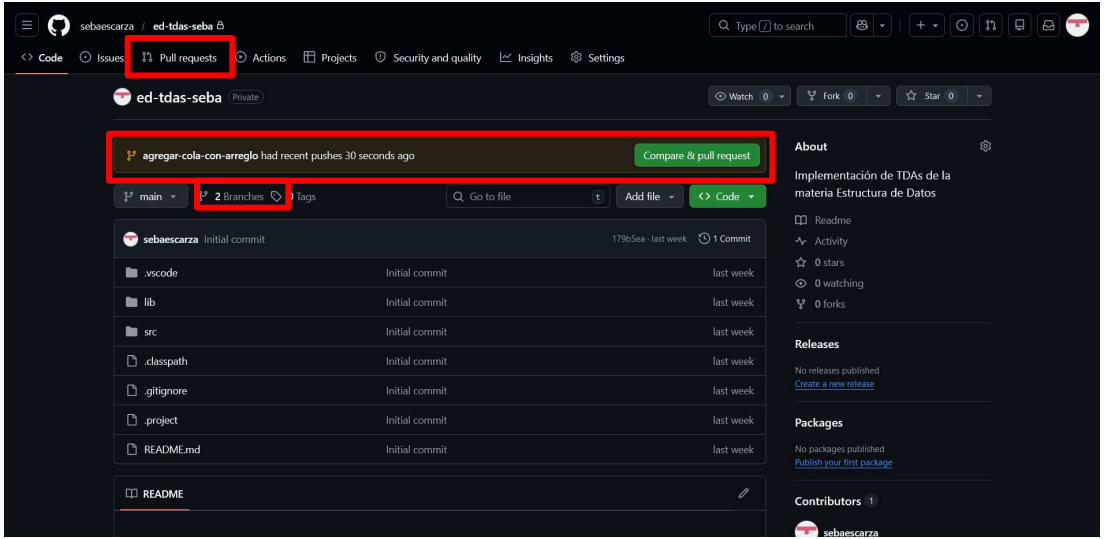
Los PRs (tanto abiertos como cerrados, independientemente si se mezclaron o no) se pueden gestionar desde la solapa Pull Requests del sitio web. Ahí se ofrece un listado completo de todos los PRs abiertos. Los PRs reciben una numeración creciente consecutiva y no se pueden eliminar ya que hacen a la historia del desarrollo del proyecto.

GitHub - Pull Requests (PRs)

Cuando una rama fue actualizada recientemente GitHub la resalta en la solapa Code.

Pulsando el botón **Compare & Pull Request** el sitio nos permite abrir un PR.

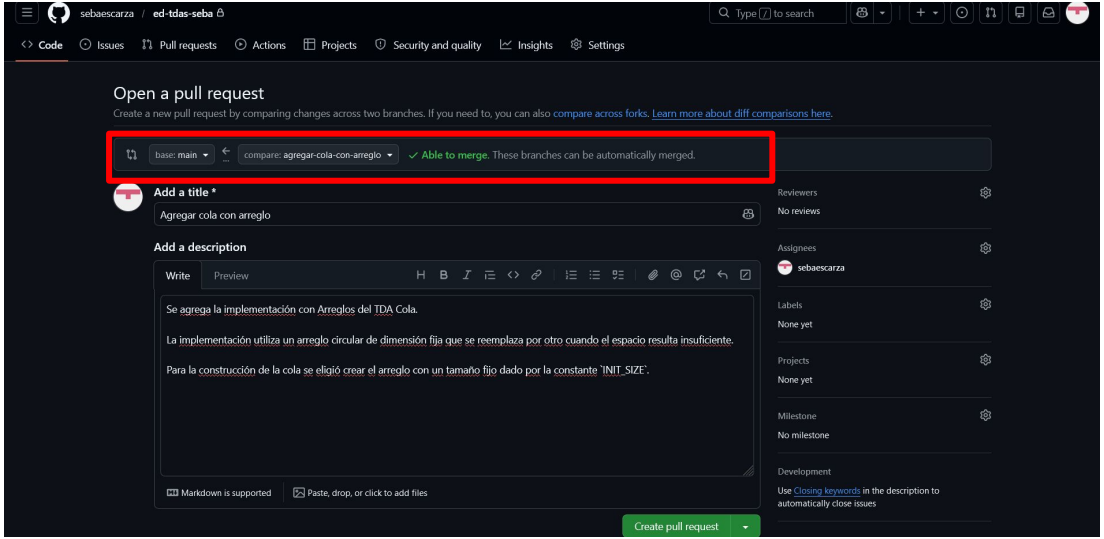
También se puede abrir un PR desde **Branches** (operando sobre la rama ahí), o desde la solapa **Pull Requests** seleccionando **New pull request**.



GitHub - Pull Requests (PRs)

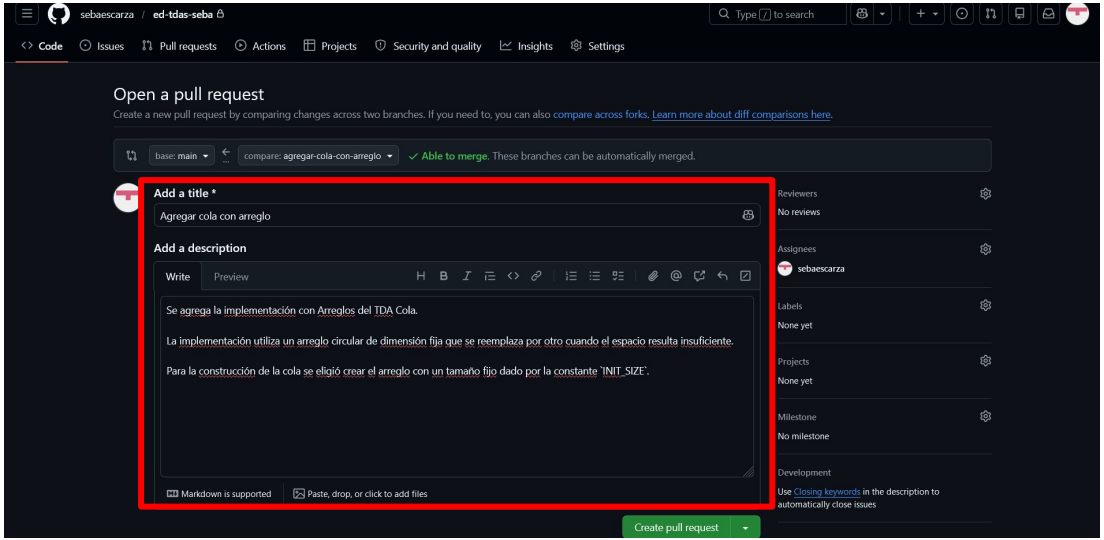
Al abrir un PR se permite elegir las ramas de origen (compare to:) y destino (base:), y se indica si GitHub puede mezclar las ramas automáticamente (**Able to merge**) o si hay conflictos a resolver.

Revisar si las ramas de origen y destino son las correctas, sobre todo en Forks ya que GitHub tiende a querer mezclar los cambios en el repositorio upstream.



GitHub - Pull Requests (PRs)

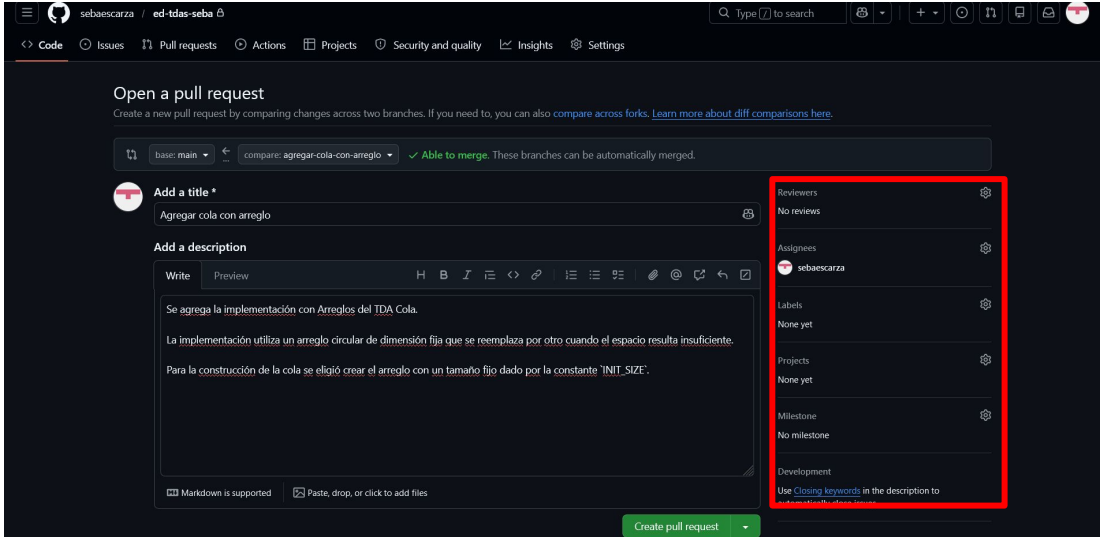
Se completa el título y la descripción (en Markdown), documentando la intención del PR, los cambios realizados, decisiones de diseño e implementación, aspectos del probelma a resolver, y/o cualquier otra información relevante.



GitHub - Pull Requests (PRs)

En el panel de la derecha se pueden configurar cuestiones organizativas del equipo de desarrollo (cuando hay alguno):

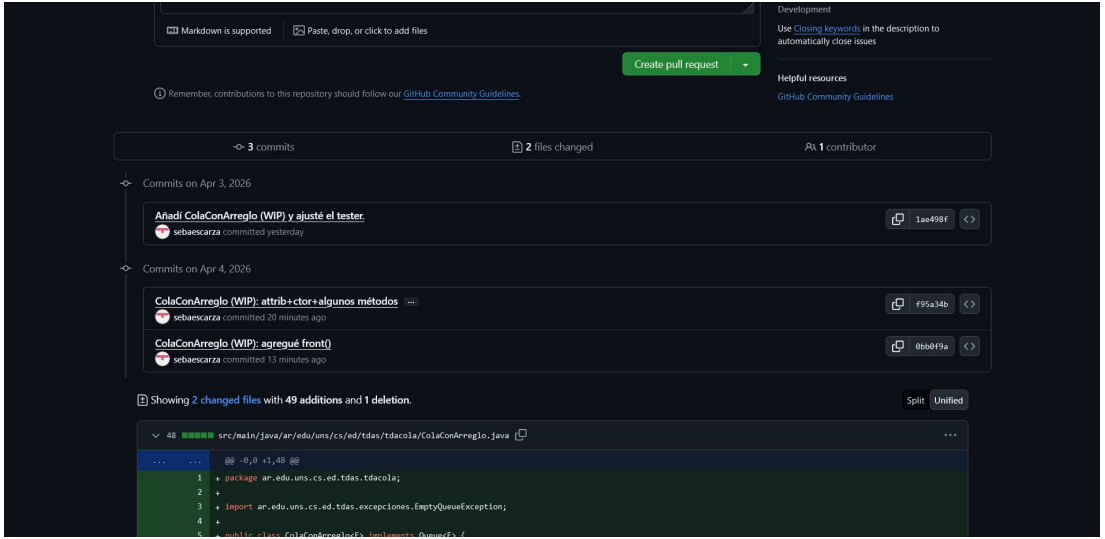
- A quién se asigna el PR
- Quién o quiénes deberían revisarlo/aprobarlo
- Etiquetas, Proyectos e Hitos de desarrollo con los que se relaciona



GitHub - Pull Requests (PRs)

Más abajo en dicha pantalla se muestran los commits y las diferencias existentes entre la rama a integrar y la base.

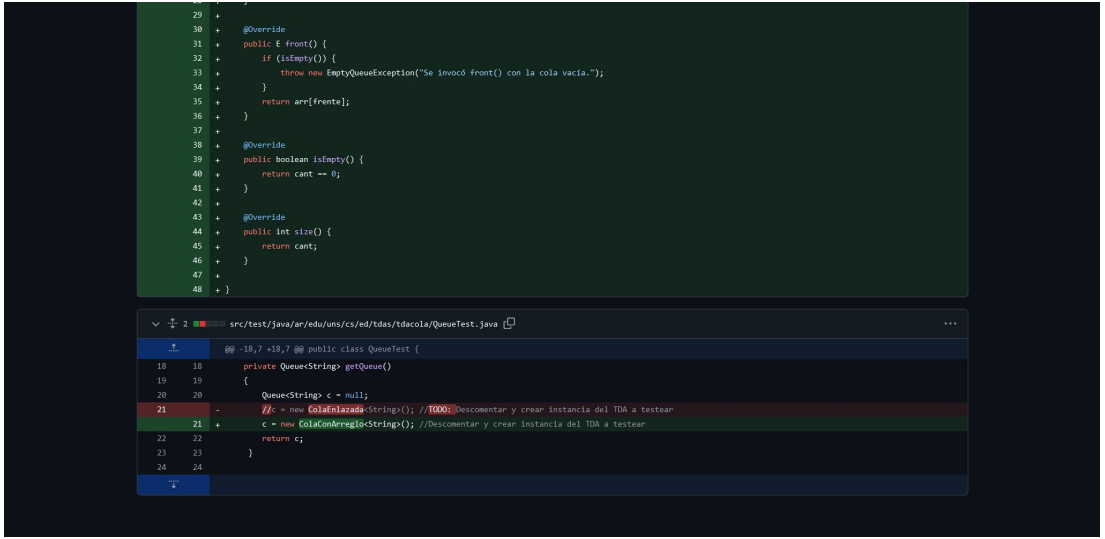
Uno puede explorar brevemente esta información para determinar si está abriendo el PR a partir de los cambios correctos.



GitHub - Pull Requests (PRs)

Más abajo en dicha pantalla se muestran los commits y las diferencias existentes entre la rama a integrar y la base.

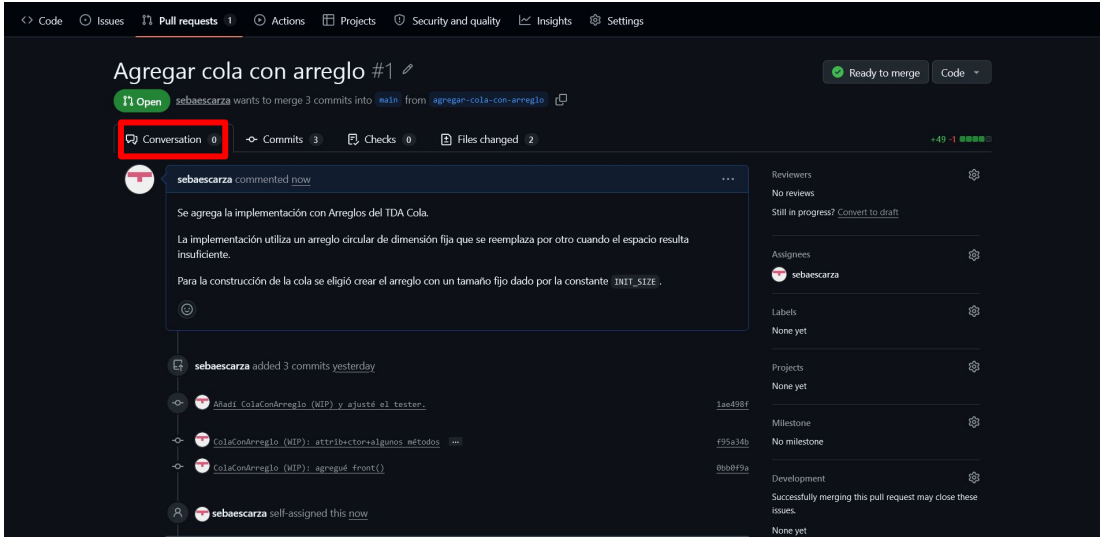
Uno puede explorar brevemente esta información para determinar si está abriendo el PR a partir de los cambios correctos.



GitHub - Pull Requests (PRs)

Al pulsar **Create Pull Request** el PR se crea, queda abierto y lo podemos encontrar en la sección **Pull Requests** del sitio.

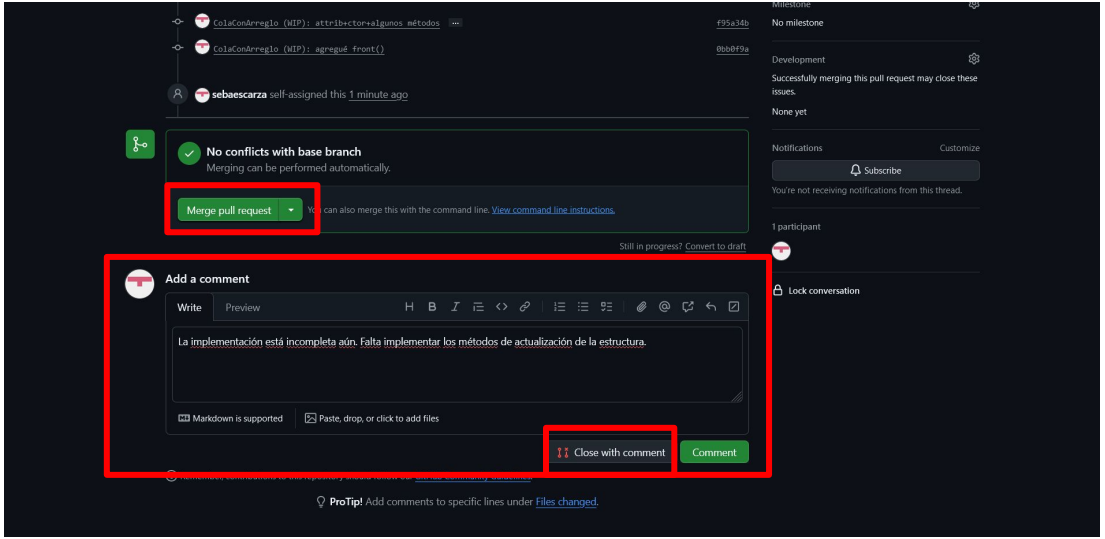
En la solapa **Conversation**, cada Pull Request muestra la información cargada durante su creación y además su evolución en el tiempo (commits publicados en la rama a integrar y comentarios de desarrollo o revisión a cargo de los desarrolladores).



GitHub - Pull Requests (PRs)

Se pueden agregar comentarios para aclarar o consultar cosas sobre el PR y sus cambios, reaccionar, arrobar desarrolladores y otras bondades.

Cuando el trabajo está listo, el botón **Merge pull request** produce la mezcla y cierre del PR. También se puede cerrar un PR con un comentario (**Close with comment**) sin mezclar sus cambios (rechazando los mismos con una razón fundada). Los PR cerrados pueden reabrirse en el futuro.

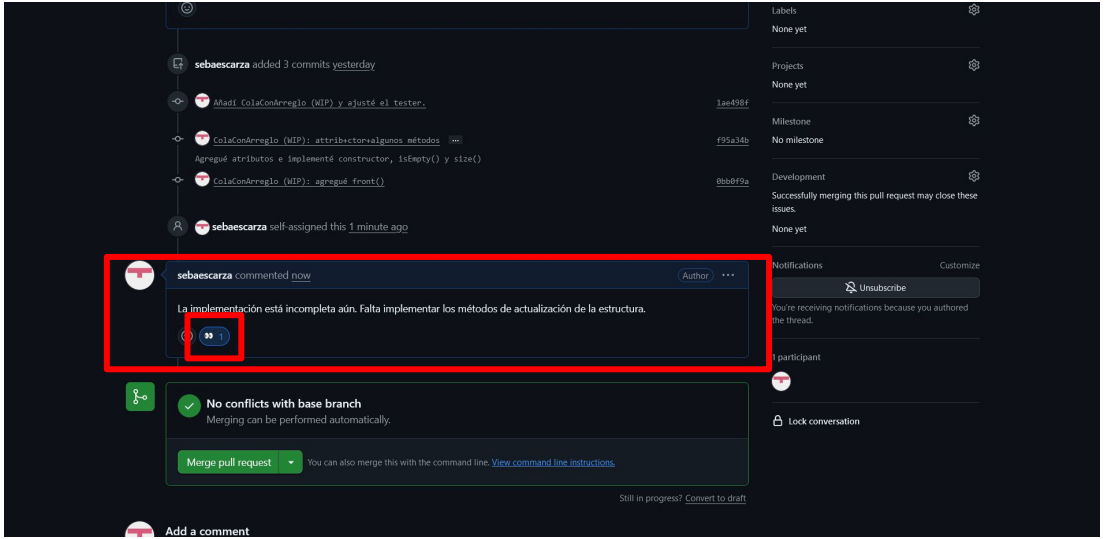


GitHub - Pull Requests (PRs)

A medida que se comenta, el hilo del PR evoluciona.

En el ejemplo se muestra un comentario y una reacción al mismo.

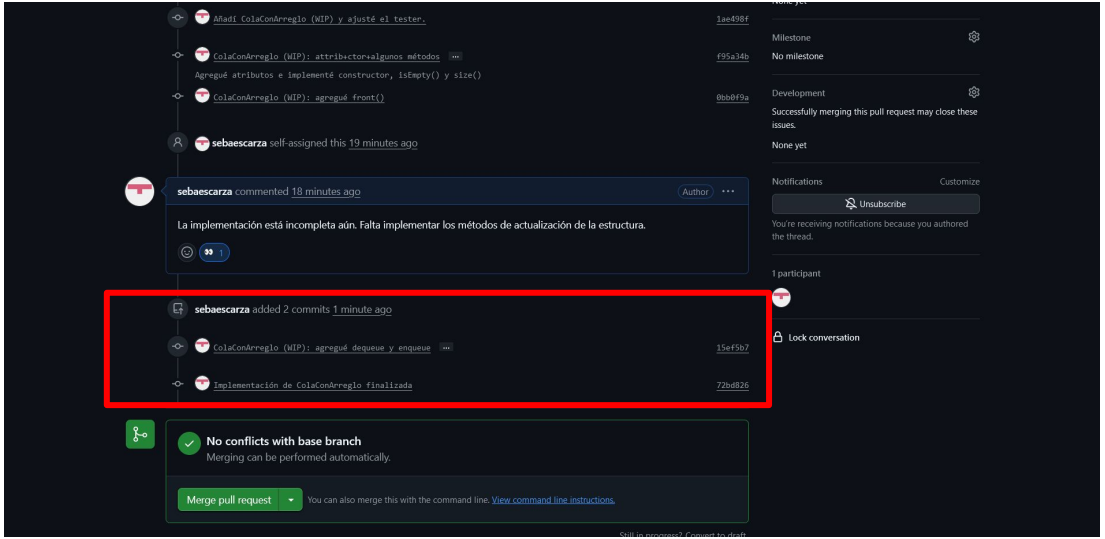
Los desarrolladores pueden debatir sobre los cambios en el código directamente sobre el mismo.



GitHub - Pull Requests (PRs)

Agregar nuevos commits sobre la rama local y hacer push de los mismos para actualizar la rama upstream en GitHub provoca que se actualice el hilo del PR.

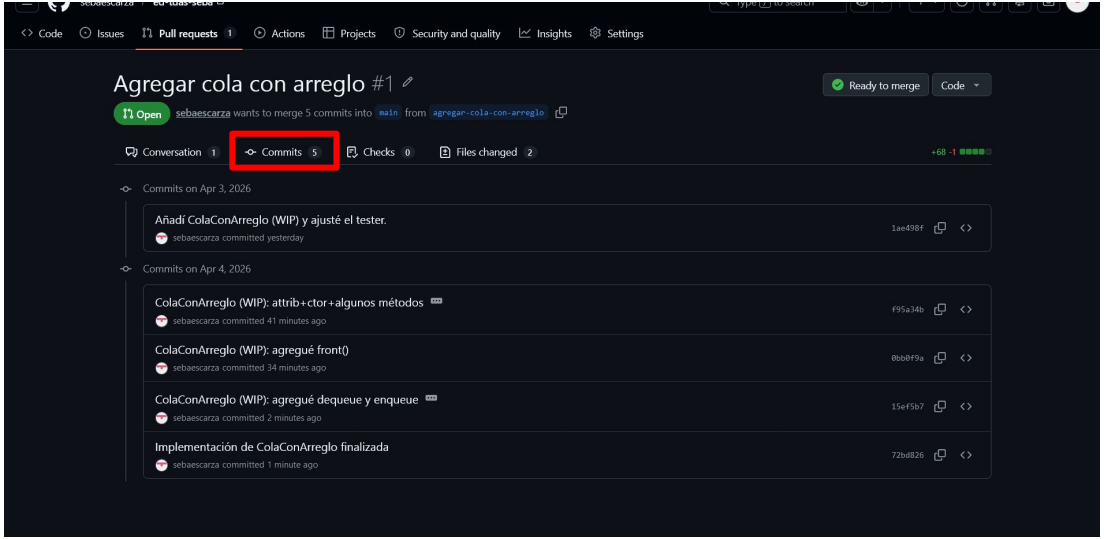
La solapa Conversation muestra en todo momento la evolución en el tiempo del PR.



GitHub - Pull Requests (PRs)

La solapa Commits muestra los commits del PR en orden cronológico.

Sólo se muestran los PRs que aportan cambios entre la rama a mezclar y la rama base.

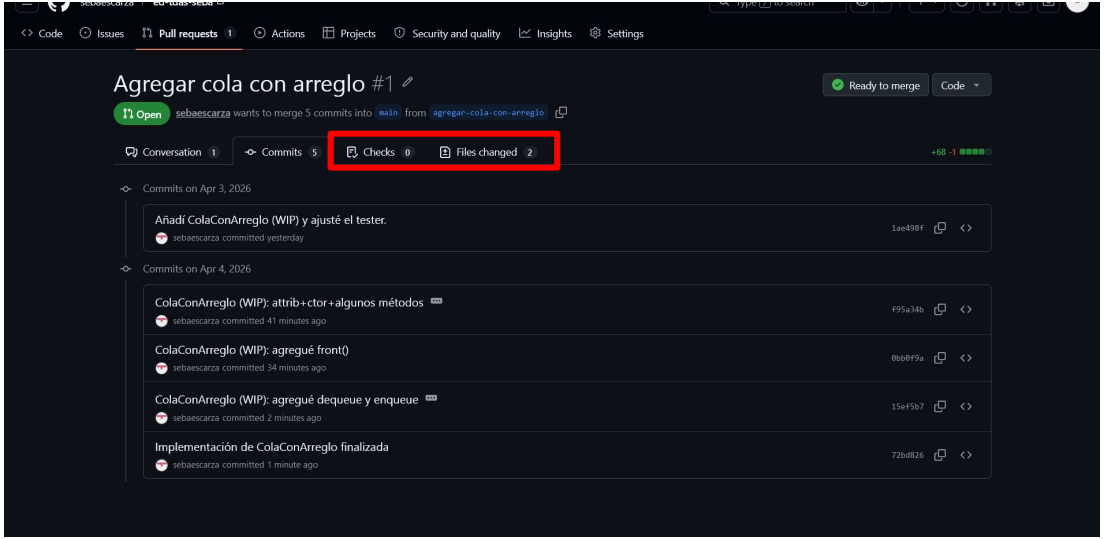


GitHub - Pull Requests (PRs)

La solapa **Checks** muestra chequeos automáticos para el PR que se pueden configurar en el proyecto.

Por ej. se puede bloquear la mezcla del PR si los cambios no pasan test automatizados, o si no se obtuvo aprobación explícita de un número de desarrolladores, etc. No lo utilizaremos en la materia.

Files changed muestra los cambios del PR archivo por archivo.

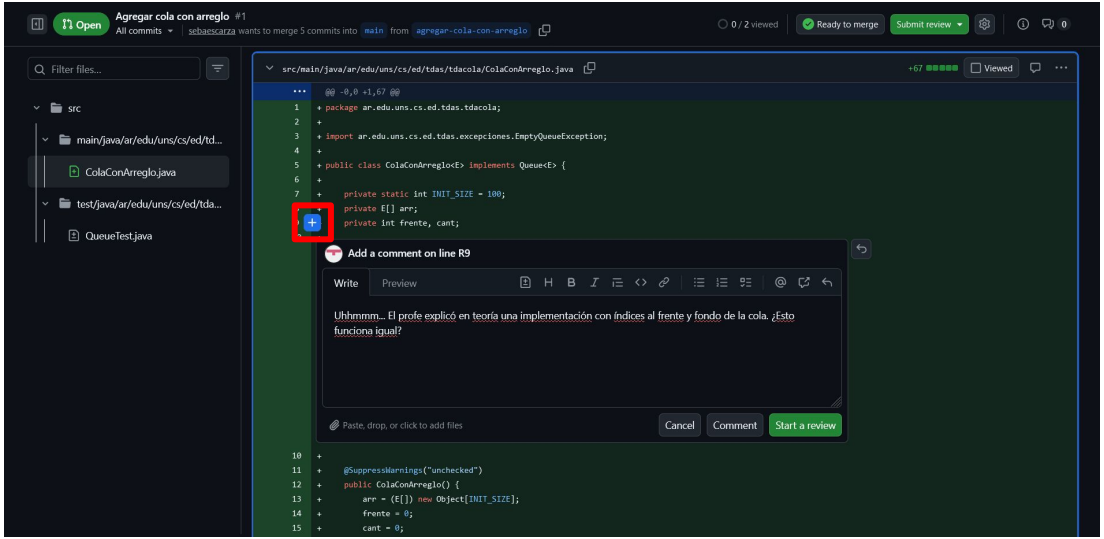


GitHub - Pull Requests (PRs)

La vista **Files changed** permite comentar el código línea a línea.

Haciendo click sobre el + azul del margen se pueden abrir comentarios referidos a la línea marcada.

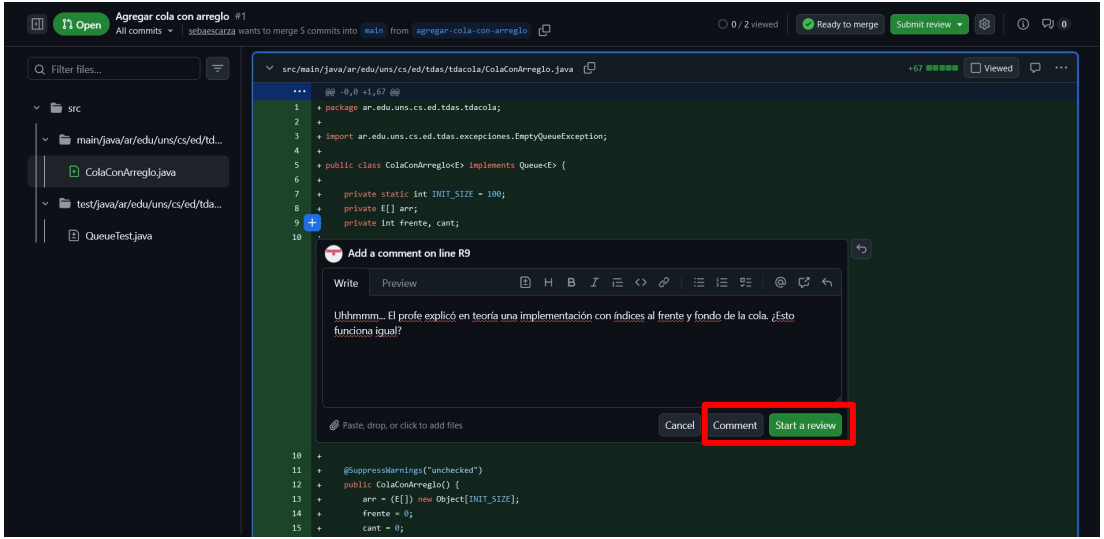
Si en lugar de hacer click, se arrastra sobre el + del margen, se pueden seleccionar varias líneas (un fragmento de código) y comentar sobre él.



GitHub - Pull Requests (PRs)

El comentario puede:

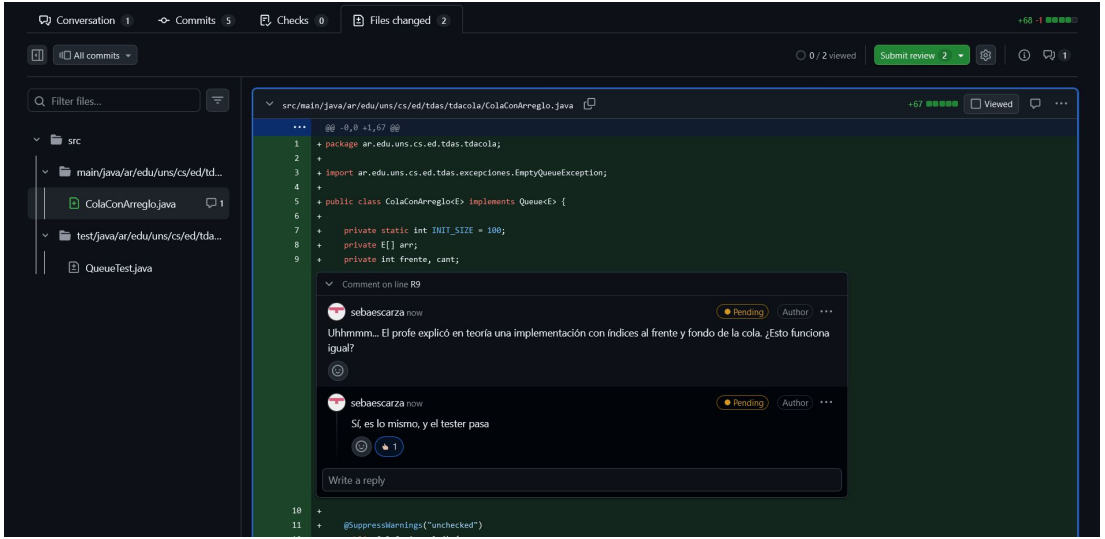
- ser un comentario no vinculante (**Comment**): se trata de un comentario simple, o
- iniciar/continuar un review (**Start a review**): En este caso se puede configurar el PR para que no se pueda mezclar hasta resolver los comentarios de review. Un integrador podría pedirle cambios obligatorios al encargado del PR.



GitHub - Pull Requests (PRs)

Los comentarios en el código sirven para mantener un hilo de discusión ahí mismo, reaccionar, debatir alternativas, etc.

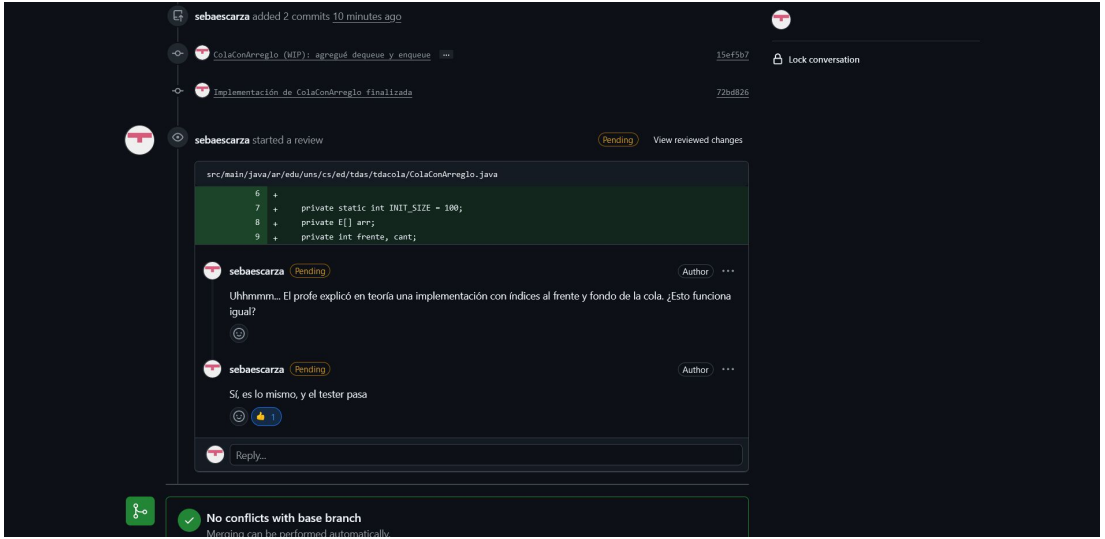
En el ejemplo se marcan los comentarios como Pending porque se inició un Review. Un desarrollador diferente del creador/encargado del PR debería aprobar dichos cambios para continuar (el diálogo es simulado pues no se dispone de otro developer al momento de hacer estas capturas =P).



GitHub - Pull Requests (PRs)

El chat con la porción de código relvante también se muestra automáticamente en la solapa de Conversation.

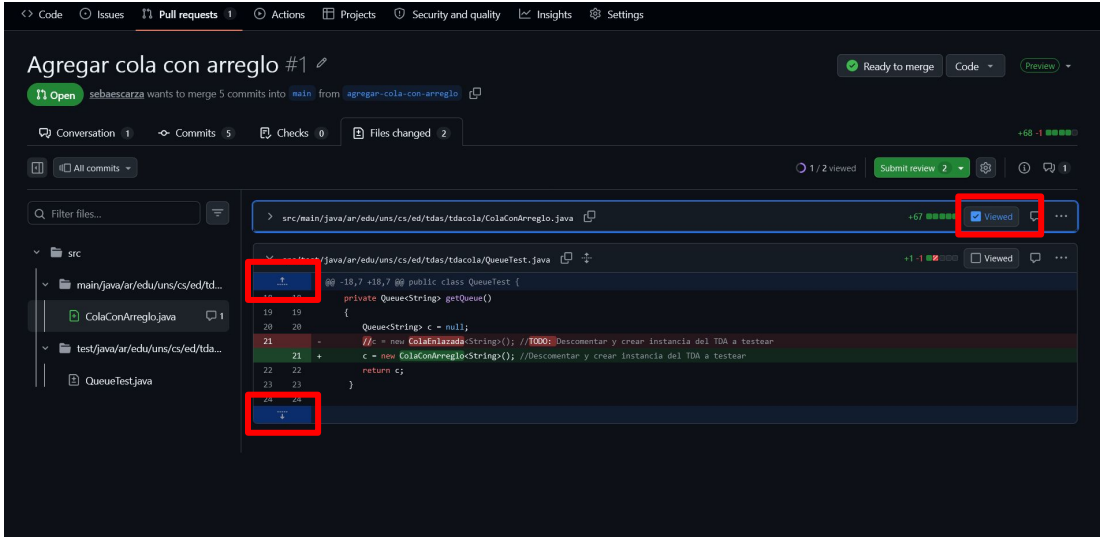
Ahí se agrupan los comentarios y se puede hacer un seguimiento en el tiempo y continuar las diferentes discusiones.



GitHub - Pull Requests (PRs)

Durante una revisión de código GitHub permite marcar los archivos que se hayan revisado para agilizar el control.

También muestra sólo las porciones de cada archivo que han cambiado pudiendo el revisor, si lo necesita, abrir más contexto alrededor del cambio con las flechas azules.



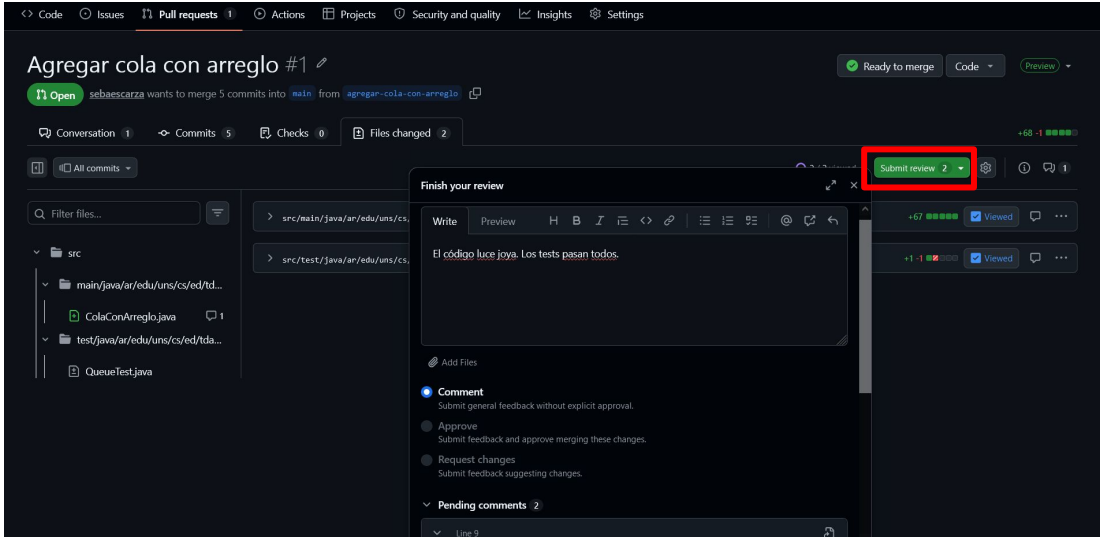
GitHub - Pull Requests (PRs)

La revisión se puede cerrar desde el botón **Submit review**.

Ahí se puede hacer un comentario general sobre el review (Comment), aprobar el review (Approve), o solicitar cambios específicos (Request changes).

Tabmién se muestran los comentarios pendientes para cerrarlos antes de completar el review.

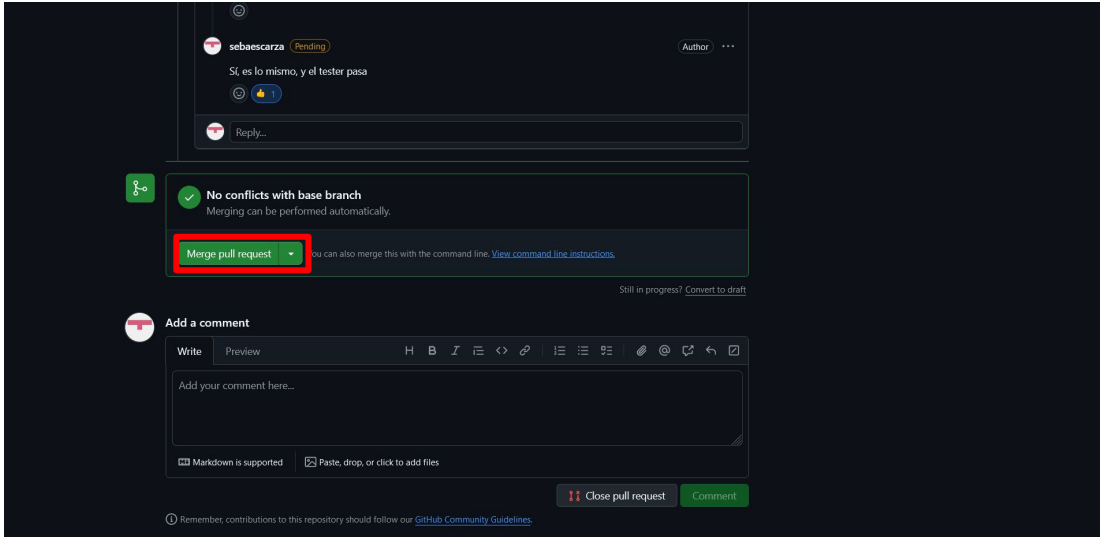
En la captura algunas opciones no están habilitadas pues el creador del PR no puede concluir su revisión.



GitHub - Pull Requests (PRs)

Cuando los cambios están listos, y si no hay conflictos ni checks pendientes, en la solapa de Conversation se puede mezclar el PR (**Merge pull request**).

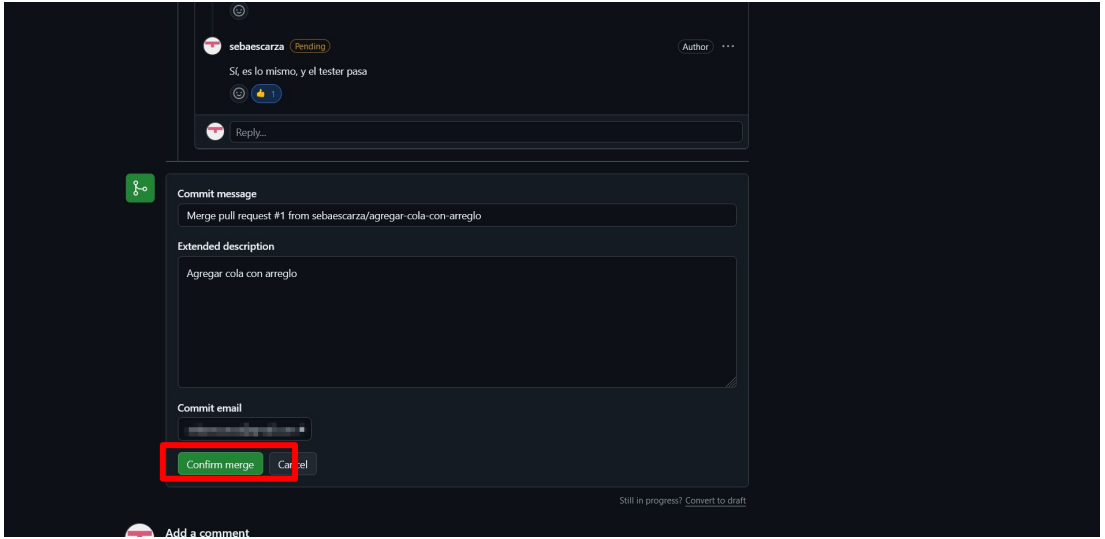
Esta acción le indica a GitHub que haga un **merge** de la rama del PR en la rama base y que cierre el PR.



GitHub - Pull Requests (PRs)

El merge se produce añadiendo un commit en la rama base (main) que condensa todos los cambios de la rama a mezclar.

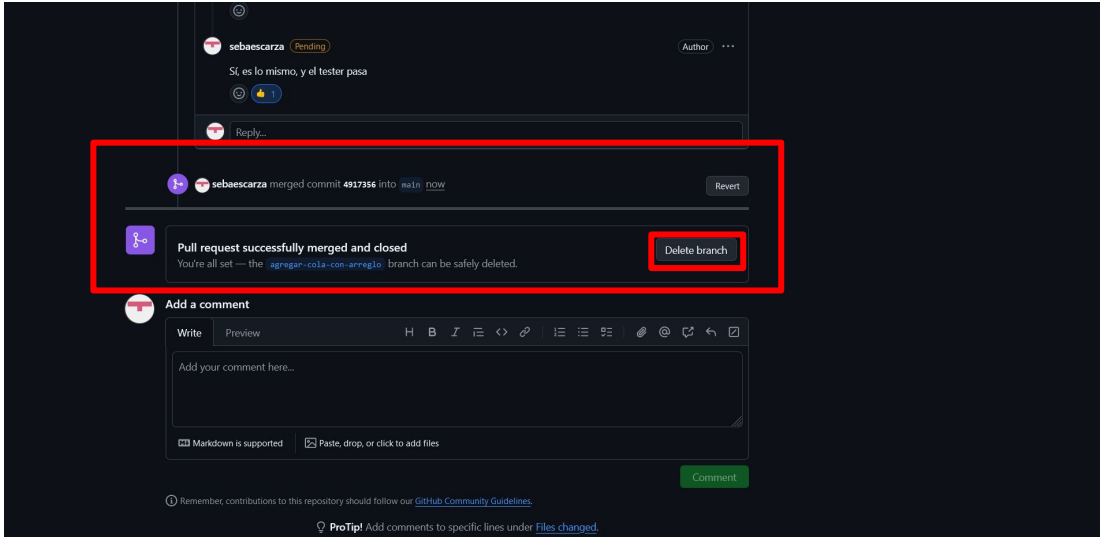
Se revisa el mensaje de commit (encabezado y cuerpo, no suele hacer falta cmabiarlo, pero es posible) y se confirma el merge (**Confirm merge**) para completar la integración (en **origin/main** en el ejemplo).



GitHub - Pull Requests (PRs)

La conversación se actualiza mostrando la mezcla y el cierre del PR.

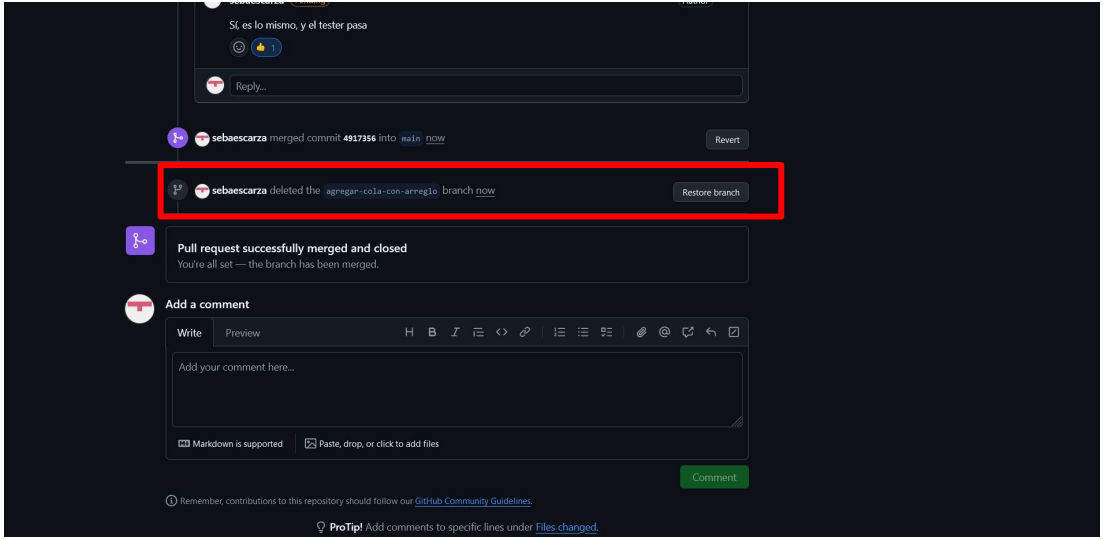
La rama base, a menos que se trate de una rama persitente y multiuso, puede eliminarse ya que no es necesaria. Para hacerlo se puede pulsar **Delete branch**.



GitHub - Pull Requests (PRs)

La eliminación de la rama se muestra en conversation. La rama podría restaurarse si se hubiese eliminado por error.

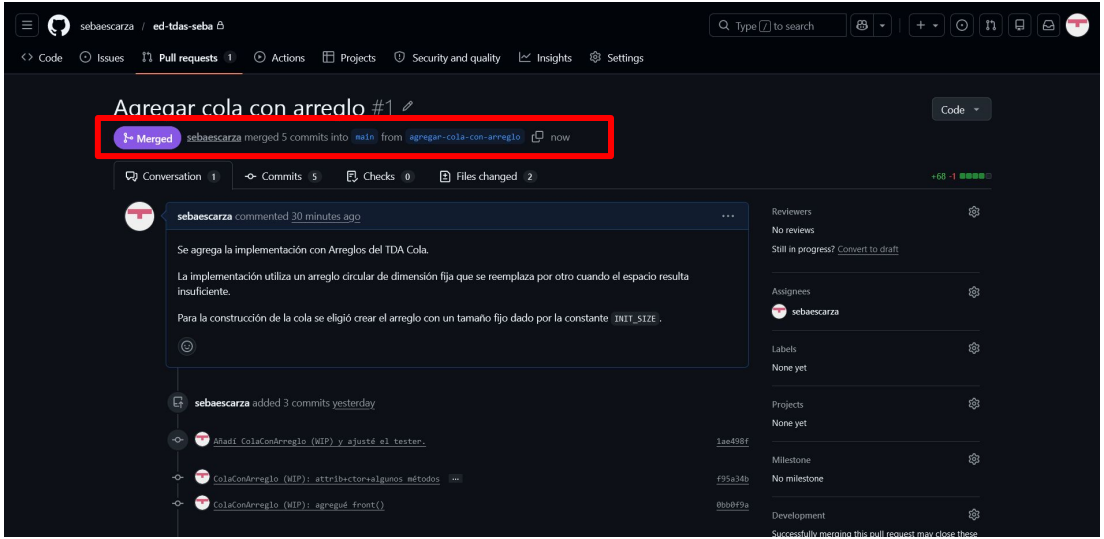
Eliminar las ramas en desuso mantiene el repositorio limpio y más fácil de gestionar.



GitHub - Pull Requests (PRs)

La interface de GitHub muestra el PR correctamente mezclado.

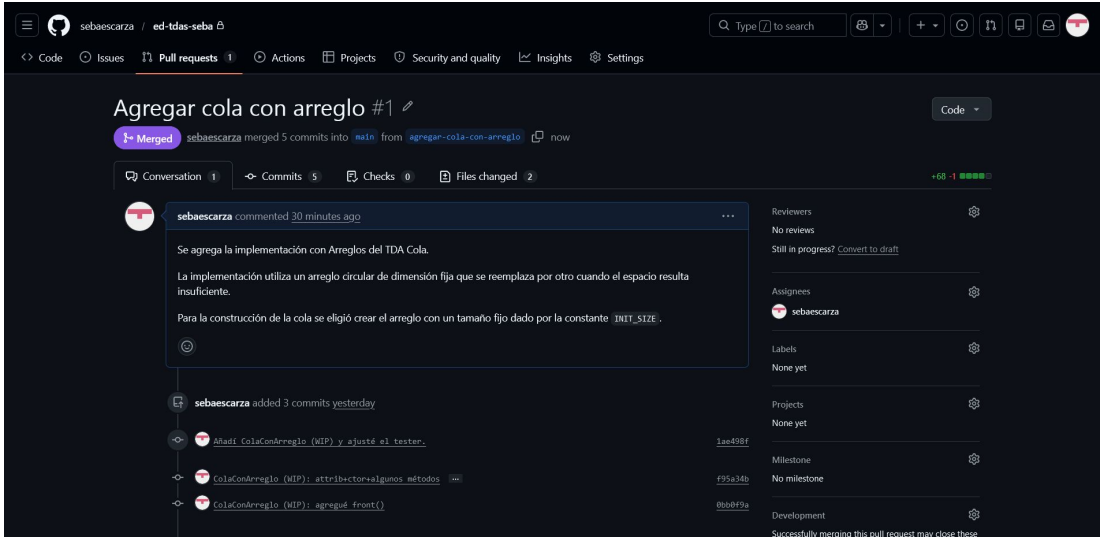
A partir de mezclar un PR, se vuelve al proyecto y el ciclo recomienza: se introducen cambios, se testean, se añaden al repo local, se publican en el remote, y a partir de ahí, en GitHub, se abre un PR con los cambios a integrar, se discuten/evalúan y finalmente se mezclan.



GitHub - Pull Requests (PRs)

Si algo no funciona o no cumple lo requerido en algún paso, se puede descartar sin comprometer el proyecto.

Las ramas con código no funcional pueden quedar sueltas o borrarse, los PR, si se llegaron a abrir, se pueden cerrar sin integrar. Y si algún cambio introdujo problemas, se puede rastrear el punto exacto donde ocurrió y resolverlo. Son las ventajas de usar un sistema de control de versiones como Git/Github.



Resolviendo conflictos

Resolviendo Conflictos en Git

Git trackea el contenido de los archivos línea a línea y aplica el cambio más reciente según el orden de los commits que afectan dicha línea.

En este contexto pueden producirse dos tipos principales de conflictos:

- Git intenta modificar el Working Directory o el Staging Area pero allí hay cambios sin confirmar que se podrían perder, sobre líneas que Git intenta cambiar. Esto puede producirse al hacer **git checkout**, **git pull**, **git merge**, etc. A menos que el working state no esté limpio (**clean** state), Git se negará a completar la operación para proteger los cambios pendientes.
- Git intenta mezclar ramas con commits conflictivos: en cada rama se modifica determinada línea de manera diferente y Git no puede decidir cuál de los cambios es más reciente y/o debería prevalecer. El usuario debe intervenir y decidir la versión que prevalecerá.

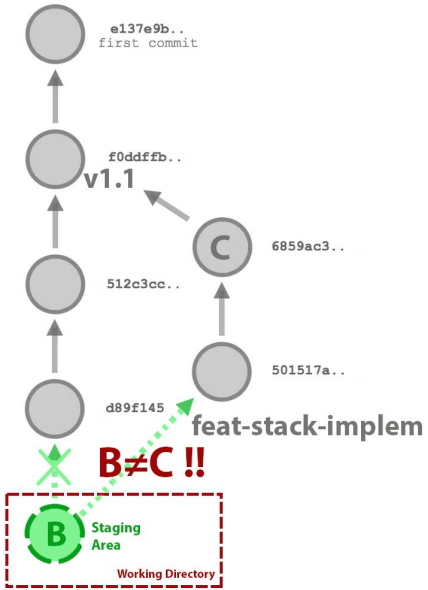
Conflictos en Working Directory/Staging Area

Supongamos que estamos trabajando en una rama arbitraria, y en determinada línea L de un archivo X realizamos un cambio que denominamos B. Dicho cambio queda en el Working Directory o en el Staging Area.

Ahora supongamos que hacemos **git checkout feat-stack-implem**, para cambiar a la rama **feat-stack-implem** para hacer commit allí del cambio B (entre otros), y que en algún punto de esa rama existe un commit que contiene otro cambio C sobre la misma línea L del archivo X.

Dado que Git para completar el checkout debe sobreescribir la línea L para que cambie según C, pero B está en WD/Staging aún sin confirmar y se perdería, Git se rehusa a completar la operación.

Esta situación no se limita a **checkouts**, sino que puede darse con cualquier comando que modifique archivos con líneas en conflicto, como por ej. **pull**, **merge**, etc.



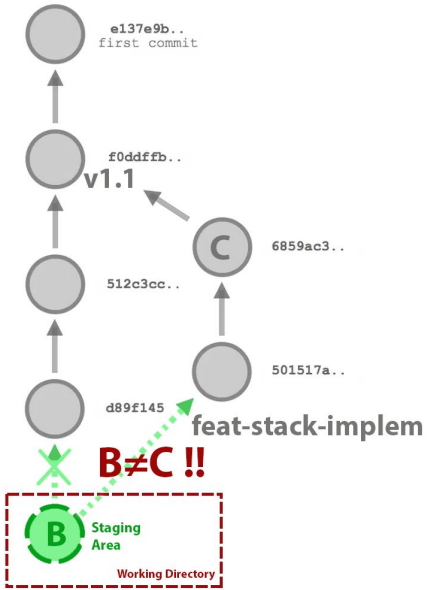
Conflictos en Working Directory/Staging Area

Este tipo de conflictos son de resolución simple ya que dependen íntegramente de la prolijidad del usuario y el manejo a nivel local que hace de Git.

Algunas posibles soluciones una vez presentada la situación son:

- Descartar el cambio B o los cambios de WD/Staging (y perderlos)
- Confirmar o hacer commit de B sobre la rama actual antes de cambiar/mezclar/traer la rama de C.
- Poner los cambios en algún archivo temporal fuera del repositorio momentáneamente hasta cambiar/mezclar/traer la rama de C, y luego restaurarlos y continuar trabjando, de manera que B se vea como algo posterior a C. Git tiene el Stash para hacer esto dentro de la herramienta.

Sin embargo, lo mejor es ser ordenado y prevenir la situación haciendo commits regularmente sin dejar cambios pendientes antes de trabajar en otra cosa.

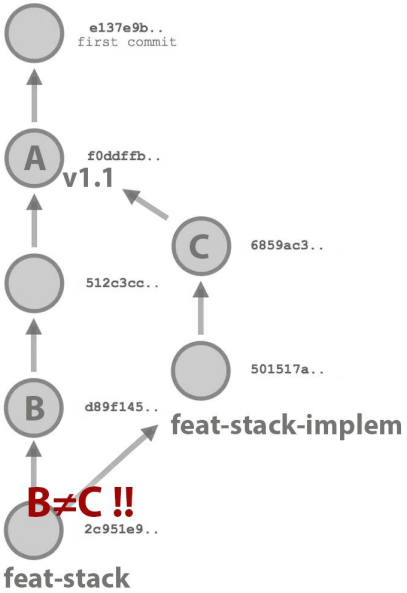


Conflictos de mezcla (merge conflicts)

Supongamos que estamos trabajando en una rama (por ej. feat-stack) tal que algún commit previo de dicha rama hace un cambio B sobre una línea L de un archivo X, y queremos mezclar otra rama (por ej. feat-stack-implem) tal que algún commit previo de dicha rama hace otro cambio C sobre la misma línea L del archivo X.

Git no tiene forma de saber si B ocurre antes que C o visceversa de manera de determinar cuál debería prevalecer. Entonces detiene el **merge** para que el usuario decida entre aplicar B, aplicar C, aplicar A (la versión de la línea L en X tal y como estaba en el commit ancestro común más reciente entre ambas ramas - equivale a ignorar tanto B como C), o producir un nuevo cambio a partir de combinarlos o uno completamente diferente de lo existente en el historial.

Lo mismo puede ocurrir al hacer un **pull** de una rama remota y recibir cambios hechos por otro developer en la misma rama.



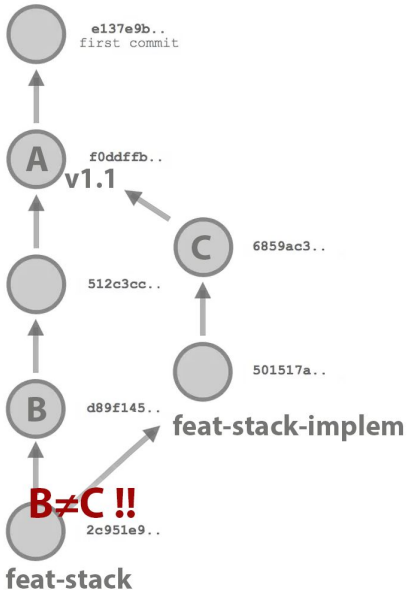
Conflictos de mezcla (merge conflicts)

Git manifiesta los conflictos de mezcla añadiendo unos marcadores especiales en texto plano en el archivo X (en lugar de la línea L):

- Lo que está entre <<<<<<< y ======= es la versión de la rama actual
- Lo que está entre ======= y >>>>>>> es la versión de la rama a mezclar

La solución del conflicto implica eliminar esos marcadores y dejar texto sin los mismos. Se puede realizar con cualquier editor de texto básico, y lo que quede es la versión que prevalecerá como solución.

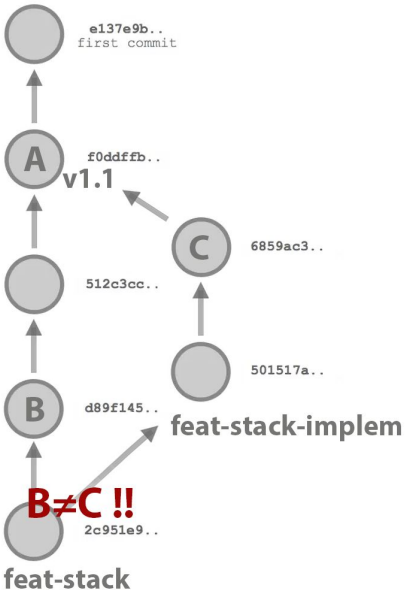
```
<<<<<<< agregar-cola-con-arreglo-circular (Current change)
    c = new ColaConArregloCircular<String>(); //Descomentar y crear instancia del TDA a testear
=======
    c = new ColaConArreglo<String>(); //Descomentar y crear instancia del TDA a testear
>>>>>>> main (Incoming change)
```



Conflictos de mezcla (merge conflicts)

Este tipo de conflictos son más complejos de resolver ya que involucran código confirmado (commited) en el historial, y no dependen de un único desarrollador ya que hacen al timing y la dinámica propia del trabajo en equipo: mientras un desarrollador trabajaba en B, otro estuvo trabajando en C.

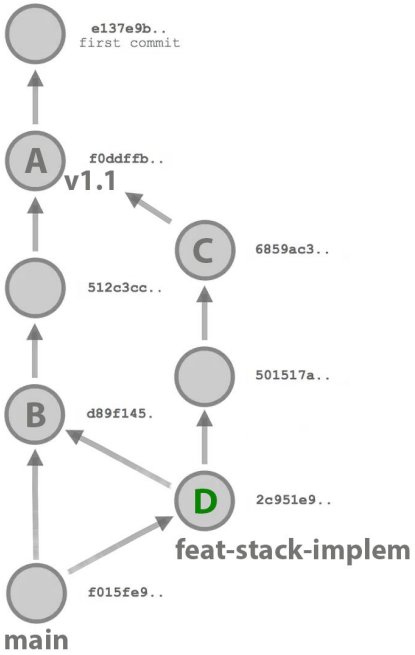
Si puedo hacer commits o mezclar en la rama de B, la solución más simple es, habiendo hecho **checkout** de la rama de B (por ej. feat-stack), mezclar (**git merge**) ahí la rama de C (por ej. feat-stack-implem) y resolver qué dejar en la línea L de X.



Conflictos de mezcla (merge conflicts)

Si, por el contrario, no puedo hacer commits o mezclar en la rama de B de manera directa por estar protegida, por ser una rama remota, o por ser una rama de GitHub en la que deba mezclar vía PRs (por ej si es origin/main), una forma de resolverlos implica:

1. Cambiar (**git checkout**) a la rama de C (por ej. feat-stack-implem)
2. En esa rama mezclar (**git merge**) la rama de B (por ej. main) para actualizarla (y que aparezca el conflicto). Editar la línea L de X con algún cambio (B, C, A, o uno diferente D) y completar el merge. Ahora la rama de C ya es “compatible” con la de B.
3. Cambiar (**git checkout**) a la rama de B (main) y mezclar la rama de C actualizada (feat-stack-implem) si puedo, o pushear y abrir PR para la rama de C y D, según el caso.



Mitigando Conflictos en Git

Hay un conjunto de buenas prácticas para minimizar la aparición de conflictos en Git/GitHub:

- Hacer **pull/fetch** de la rama local antes de empezar a cambiar cosas. Esto hace que nuestros cambios sean posteriores a los cambios obtenidos, evitando conflictos con los mismos.
- Hacer **pull/fetch** periódicos. Mientras más actualizado esté nuestro código, menor probabilidad hay de que otros desarrolladores introduzcan cambios conflictivos.
- Coordinar el desarrolo y de ser posible trabajar en archivos diferentes, (o en partes diferentes de un mismo archivo). Si no se tocan las mismas líneas, no puede haber conflictos.
- Mantener las vida de las ramas lo más corta posible. Definir ramas de propósito acotado y específico e integrarlas rápidamente. Esto reduce la ventana de probabilidad de conflictos.

Mitigando Conflictos en Git

Hay un conjunto de buenas prácticas para minimizar la aparición de conflictos en Git/GitHub:

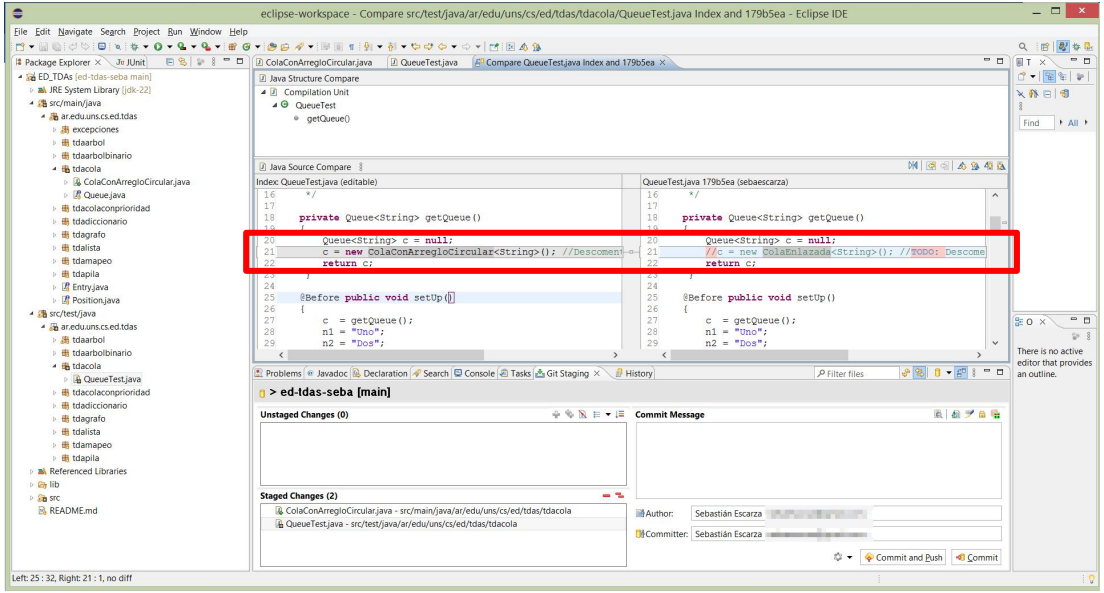
- Usar ramas y PRs (para que distintos programadores trabajen en distintas ramas). Es más simple resolver conflictos al mezclar ramas que hacerlo al querer añadir commits a una rama compartida.
- No trabajar con ramas y tirar todos los commits a main es simple y tentador. Si trabajan solos es perfectamente viable y práctico. Pero si hay dos o más programadores trabajando en el mismo repositorio, evitarlo. Varios desarrolladores tirando commits a la misma rama tienden a generar una batalla de versiones que produce conflictos constantemente con cada **push**/**pull**.

Resolviendo conflictos en Eclipse y GitHub

Resolviendo conflictos en WD/Staging

Para ilustrar la resolución de este tipo de conflictos consideremos el siguiente ejemplo.

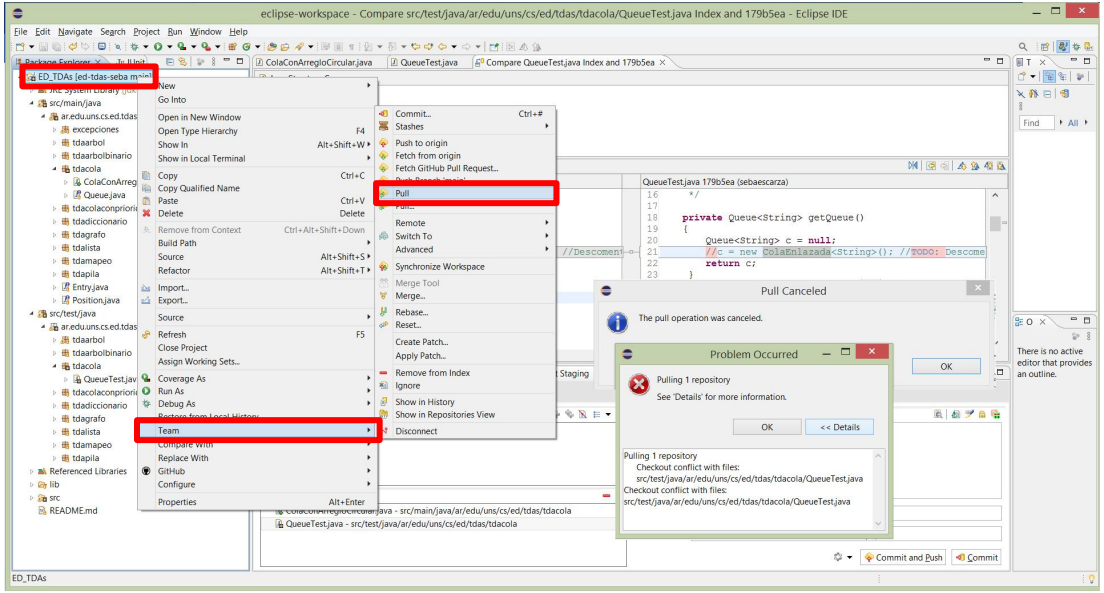
Estando en la rama main, el ejemplo muestra un cambio en la línea 21 de QueueTest.java. Se habilita el testeo de ColaConArregloCircular y se pasan esos cambios a staging.



Resolviendo conflictos en WD/Staging

El desarrollador decide ver si hubo actualizaciones de main (en origin/main) y traerlas.

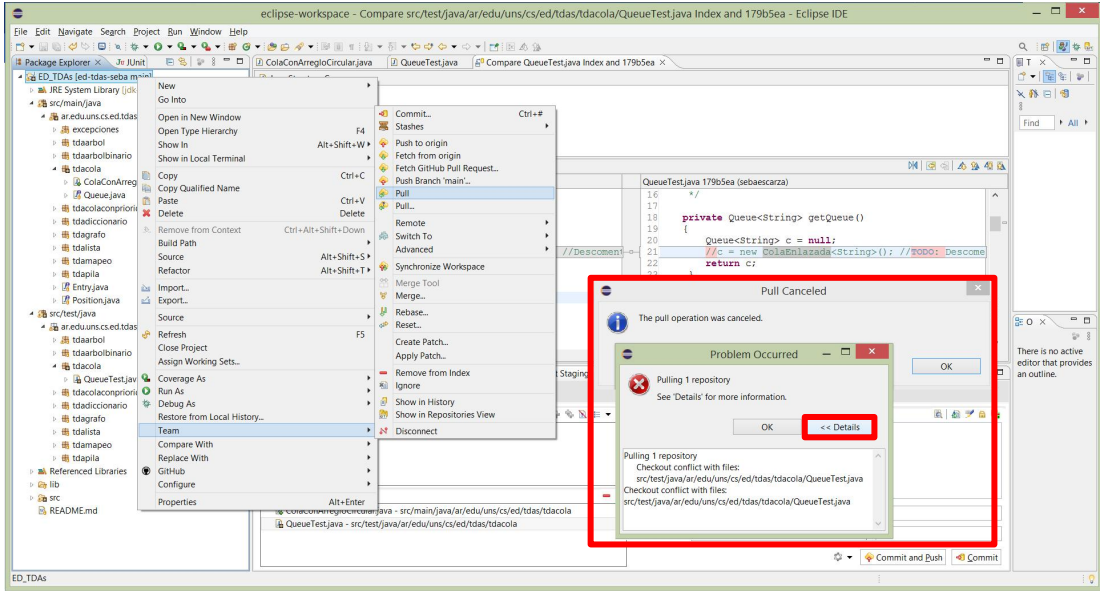
Para eso hace **git pull**.



Resolviendo conflictos en WD/Staging

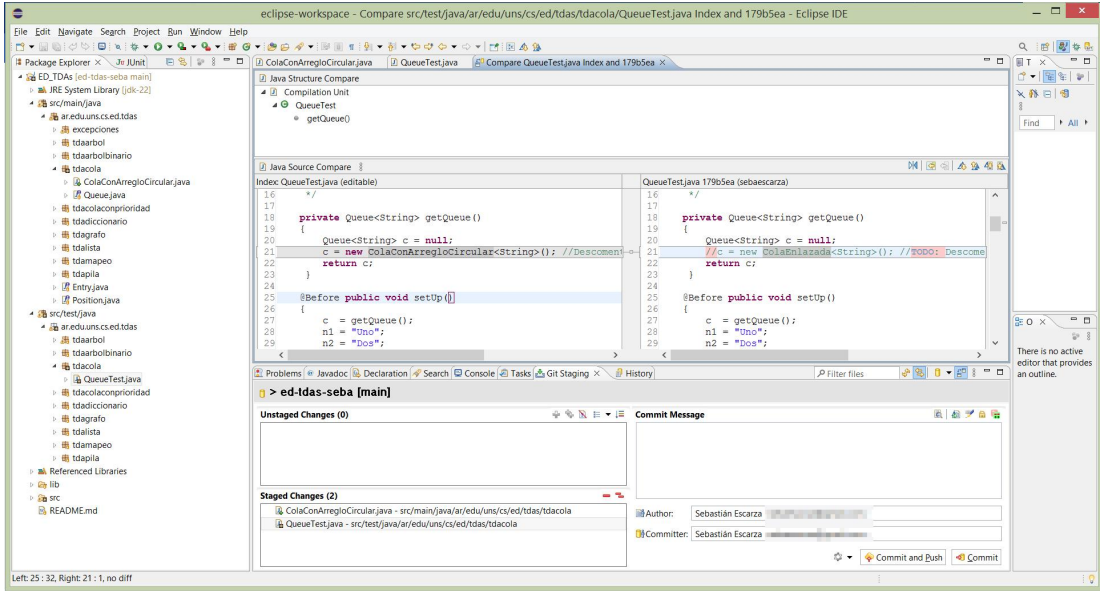
Existen conflictos entre las actualizaciones en staging y las que vienen de origin/main a través del pull. Git aborta la operación y lo reporta con un error.

Para evitar perder los cambios en staging debemos hacer algo con dichos cambios (descartarlos, confirmarlos, o guardarlos temporalmente en otro lado) antes de intentar alguna operación que modifique los archivos de WD/Staging.



Resolviendo conflictos en WD/Staging

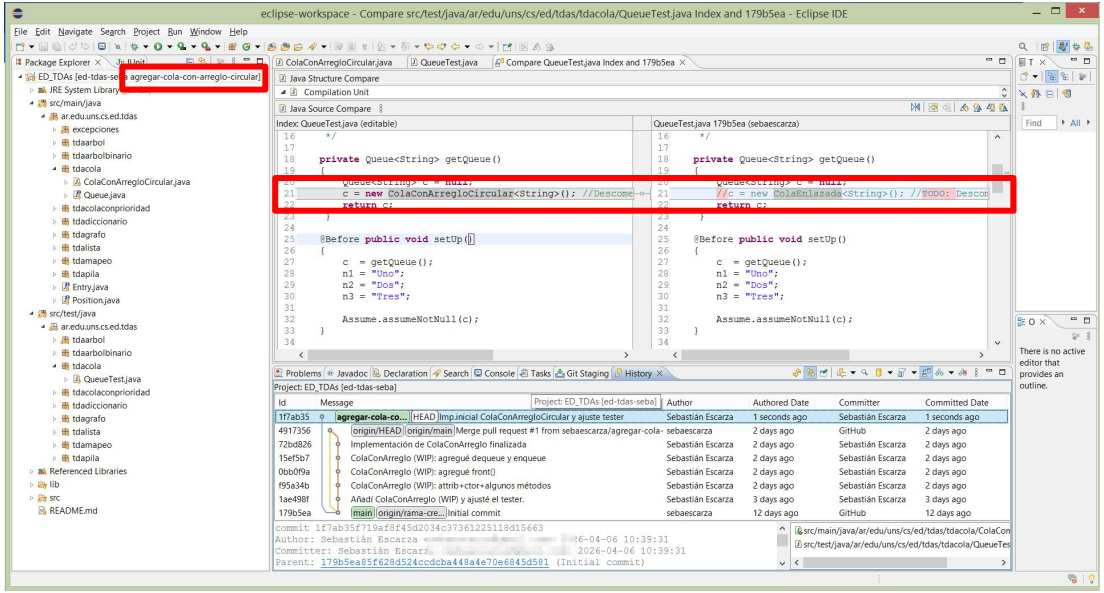
Lo recomendable en este caso hubiese sido hacer el **pull** primero para actualizar la rama y recién luego modificar los archivos en WD/Staging para trabajar a partir de versiones actualizadas de main.



Resolviendo merge conflicts

Consideremos ahora un ejemplo en el que se introducen cambios entre los que está la línea 21 de QueueTest.java en una rama llamada agregar-cola-con-arreglo-circular.

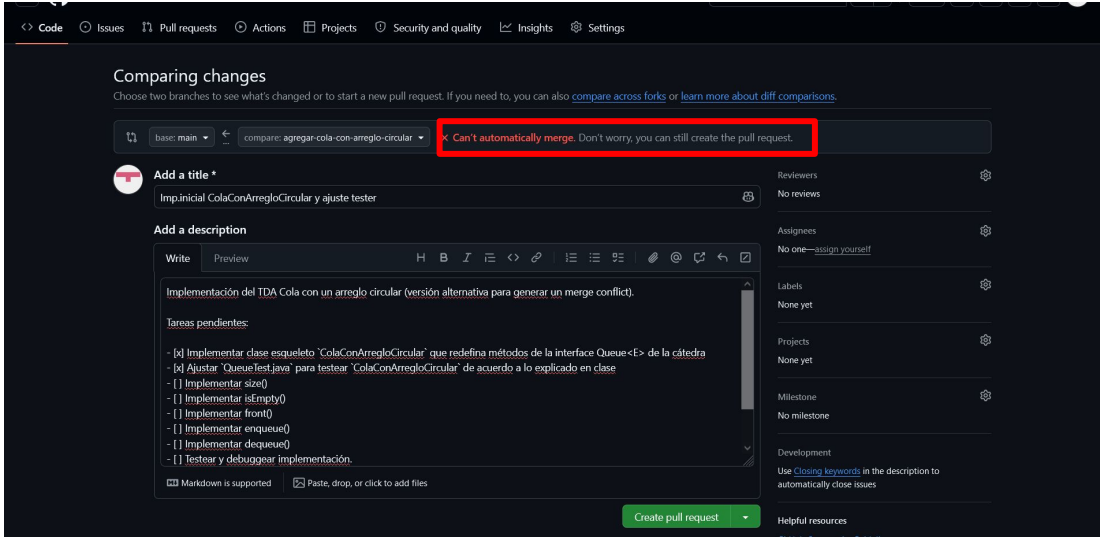
Los cambios se confirman y luego se pushean a GitHub (lo que publica la rama y hace visible los cambios en la plataforma).



Resolviendo merge conflicts

Sin embargo, al crear/abrir el PR en GitHub, la plataforma nos informa que no puede mezclar los cambios de manera automática (es decir, existen conflictos).

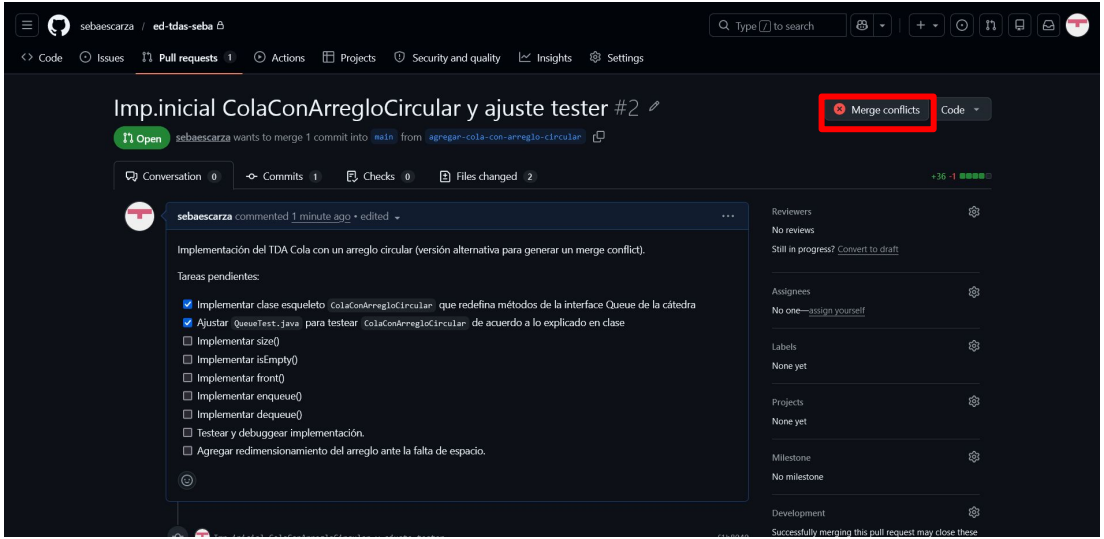
El PR se puede crear igual, por lo que se completa la info y se procede.



Resolviendo merge conflicts

La indicación de un PR con conflictos de mezcla aparece arriba a la derecha.

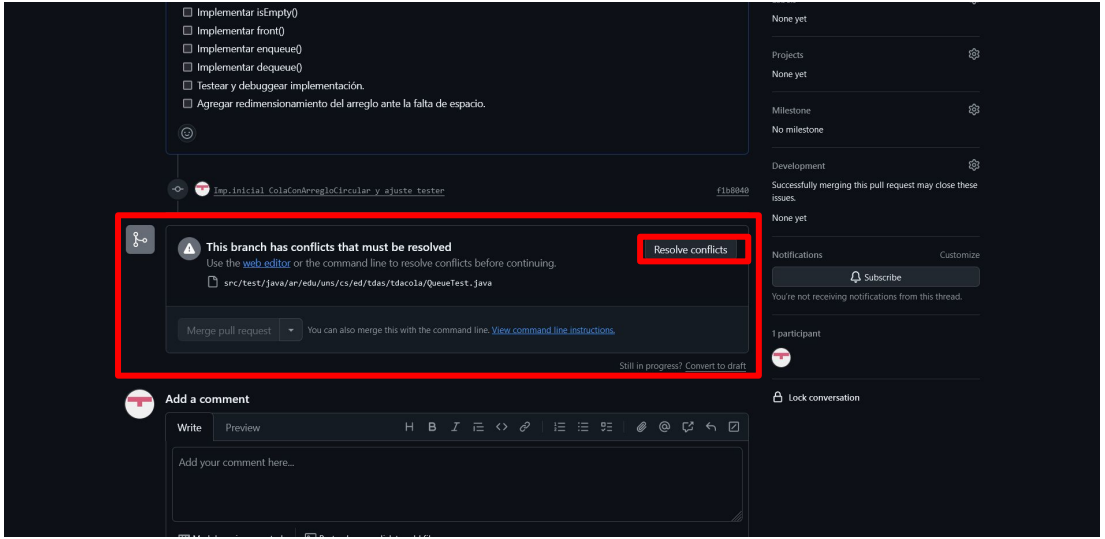
Esto también puede ocurrir al añadir (pushear) un commit con posterioridad a haber creado el PR.



Resolviendo merge conflicts

El PR no puede mezclarse hasta que no se resuelvan los conflictos. En el sector de mezcla del PR se indican los archivos conflictivos y hay un botón **Resolve conflicts**.

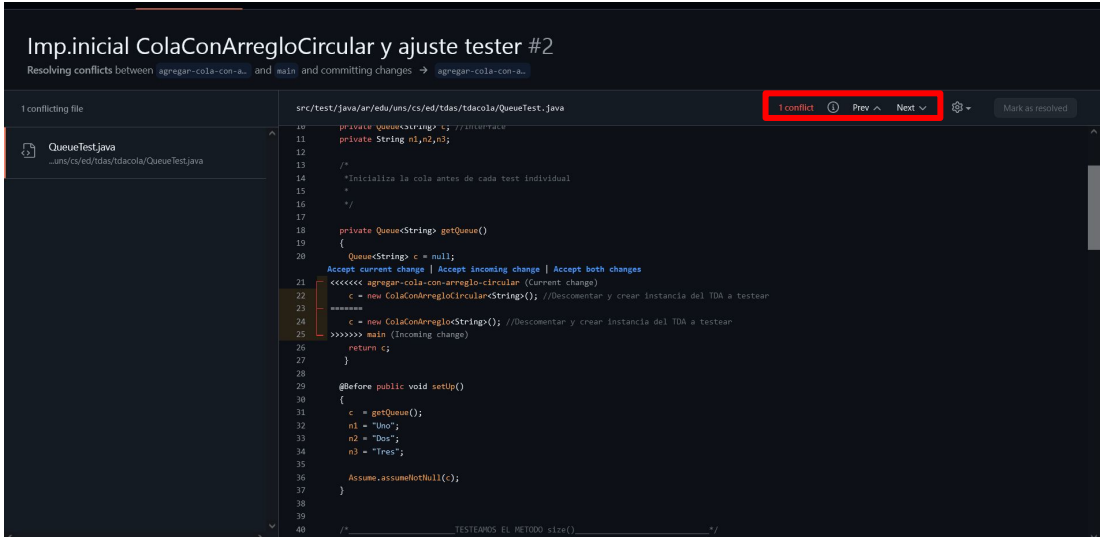
Esta opción permite resolver los conflictos directamente en GitHub (la opción más simple si usamos ramas y PRs, lo más recomendado).



Resolviendo merge conflicts

Se abre la vista de resolución de conflictos donde se muestran todos los conflictos existentes (en el ejemplo sólo hay uno en un único archivo).

Se indican cuántos conflictos hay pendientes de resolución y se ofrece una barra de navegación para analizarlos uno por uno.

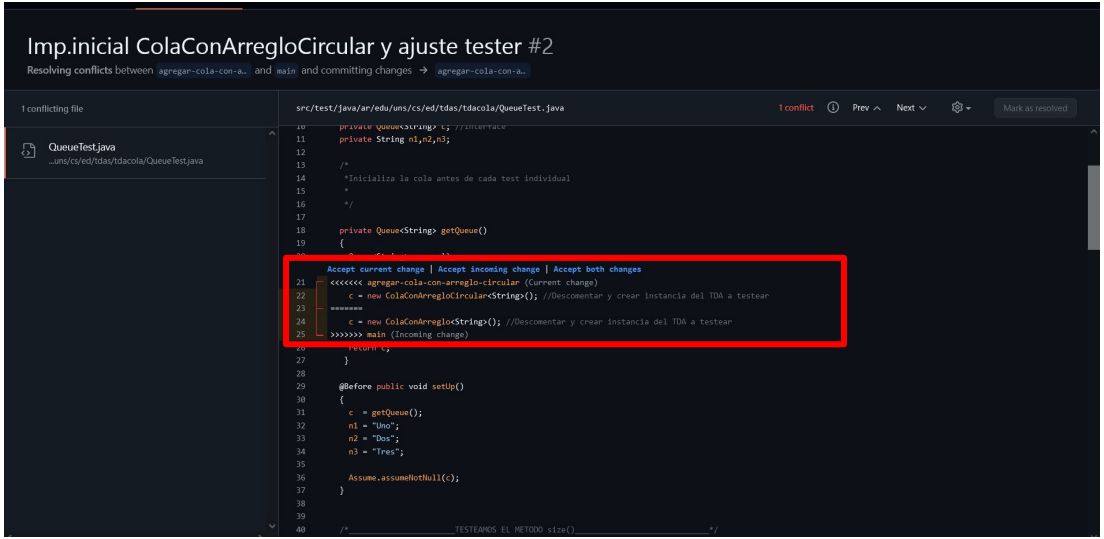


Resolviendo merge conflicts

El conflicto se presenta con los marcadores de git:

<<<<<<<
Línea 21 versión del PR
=======
Línea 21 versión de main (target)
>>>>>>>

Git está tratando de mezclar los commits de origin/main en nuestra rama agregar-cola-con-arreglo-circular para prepararla y luego mezclarla definitivamente en origin/main.

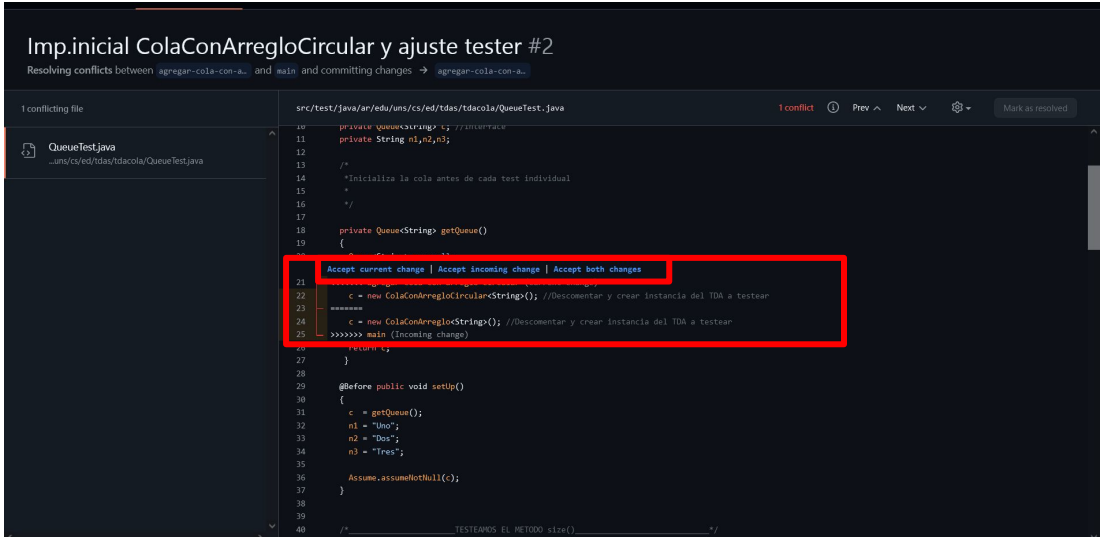


Resolviendo merge conflicts

Mientras el desarrolador de la ColaConArregloCircular programaba, otro programador hizo y ajustó el tester para trabajar con una instancia de ColaConArreglo.

Hay que decidir con qué versión nos quedamos.

GitHub y los IDEs agregan botones para facilitar la elección de opciones y no tener que tiepar a mano el archivo borrando los marcadores para resolver el conflicto.



Resolviendo merge conflicts

Si por ej. pulsamos **Accept current change** para aceptar la versión del PR, GitHub automáticamente acomoda el archivo resolviendo el conflicto en ese sentido (en un click).

Y se habilita el botón **Mark as resolved** para dar por finalizado dicho conflicto.

De esta manera se resuelven todos los conflictos existentes.



Resolviendo merge conflicts

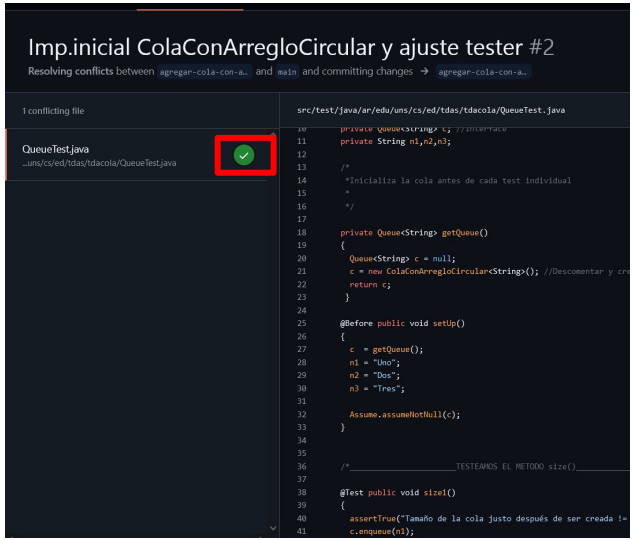
A medida que resolvermos los conflictos de cada archivo, éstos se van tildando en verde.

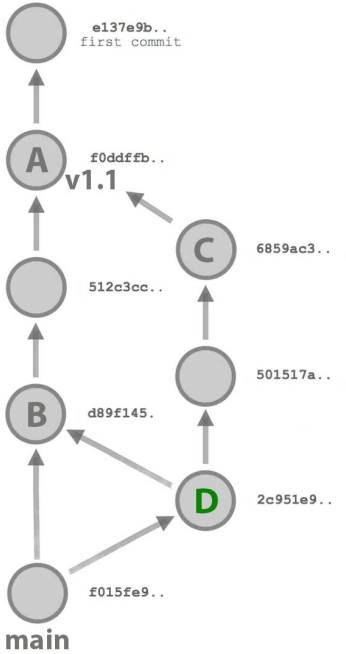
Al completar la resolución de todos los conflictos, se habilita el botón **Commit merge** para mezclar origin/main en la rama del PR y dejarla lista para mezclar el PR.



Resolviendo merge conflicts

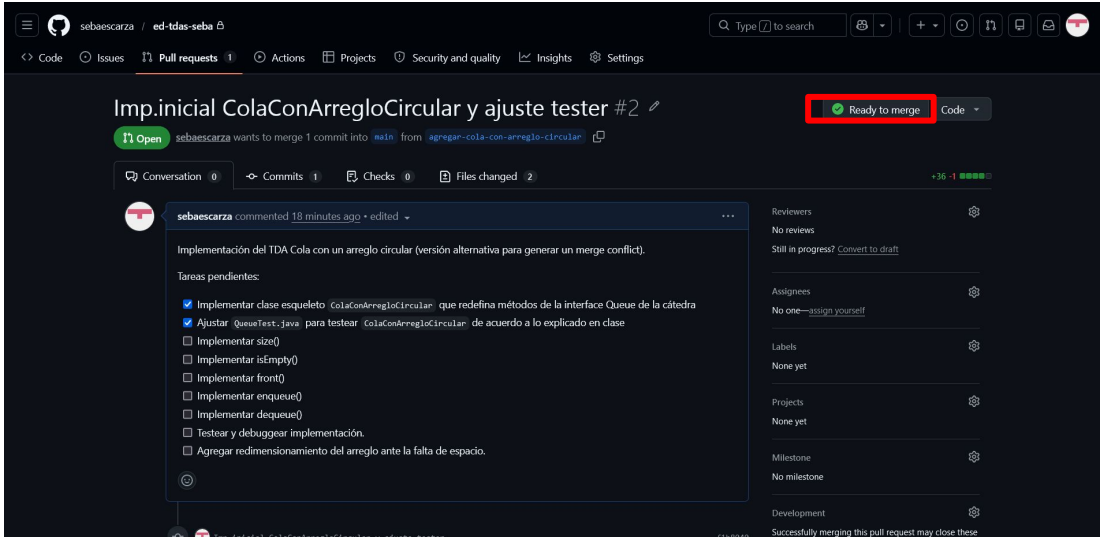
Al completar la resolución de todos los conflictos, se habilita el botón **Commit merge** para mezclar origin/main en la rama del PR y dejarla lista para mezclar el PR.





Resolviendo merge conflicts

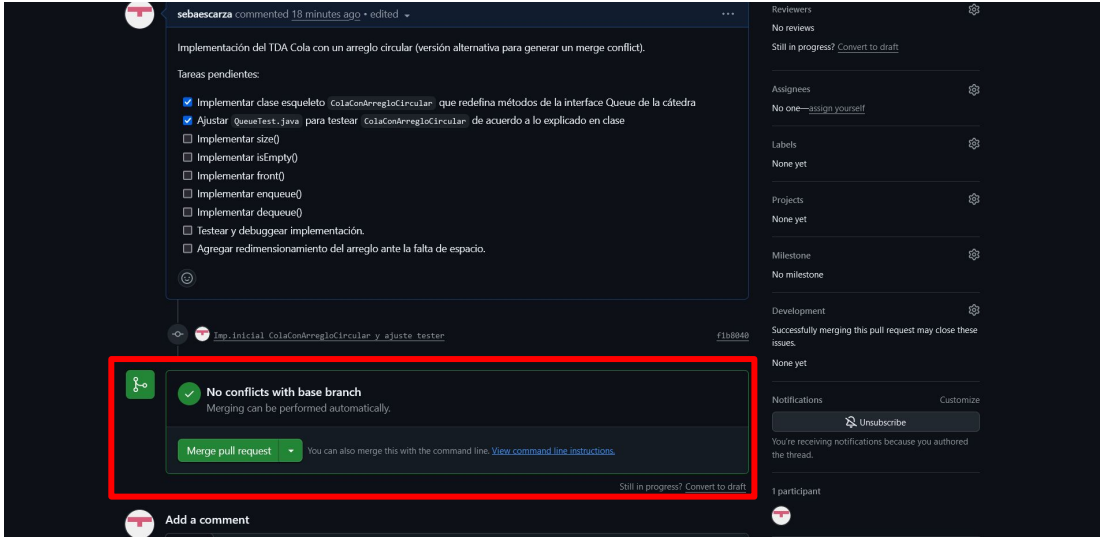
Luego de resolver los conflictos, el PR (y su rama asociada) están listos para mezclarse automáticamente.



Resolviendo merge conflicts

A partir de acá se puede trabajar con el PR normalmente, agregar cambios, discutirlos, comentar, etc, y si surgen nuevos conflictos resolverlos.

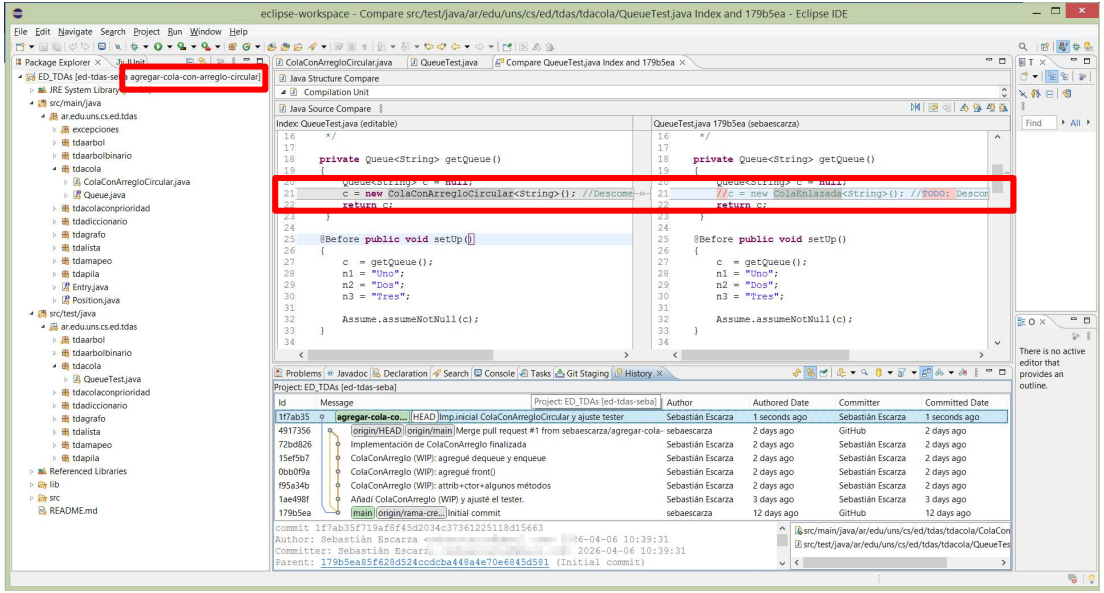
Al pulsar **Merge pull request** (ahora habilitado) GitHub mezclará la rama origin/agregar-cola-con-arreglo-circular en origin/main y cerrará el PR.



Resolviendo merge conflicts

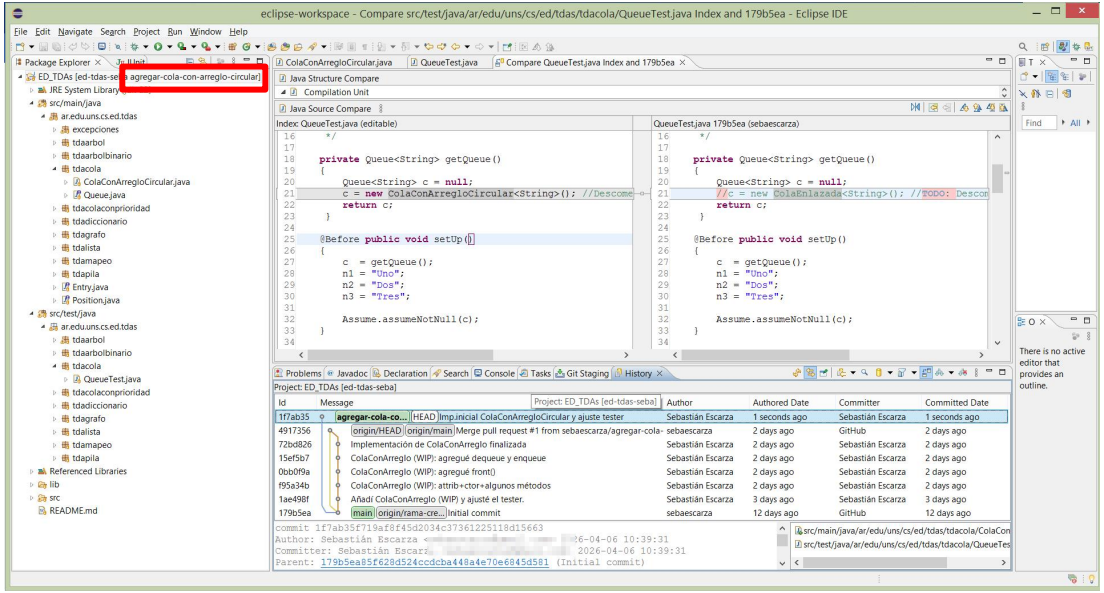
Si se usan ramas y PRs, los conflictos de mezcla se pueden resolver en GitHub. Sin embargo, a veces pueden aparecer conflictos de mezcla utilizando el IDE localmente.

Para ilustrarlo consideremos el mismo ejemplo en el que se introducen cambios en la línea 21 de QueueTest.java en la rama llamada agregar-cola-con-arreglo-circular.



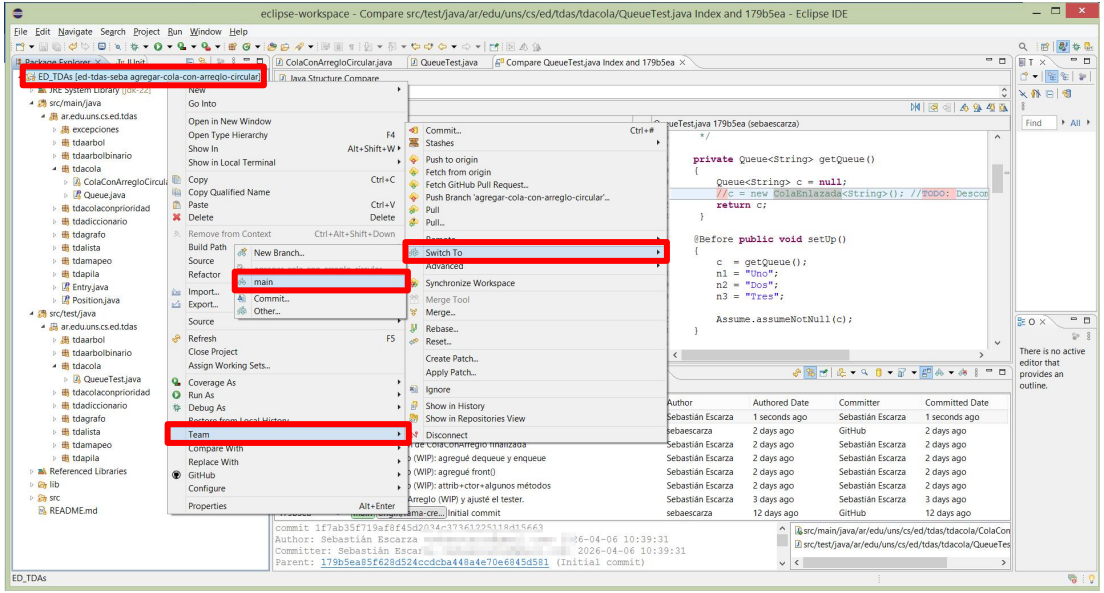
Resolviendo merge conflicts

Pero esta vez vamos a actualizar la versión local de main (trayendo cambios conflictivos) y vamos a intentar mezclar main en agregar-cola-con-arreglo-circular para prepararla para publicarla en GitHub e integrarla.



Resolviendo merge conflicts

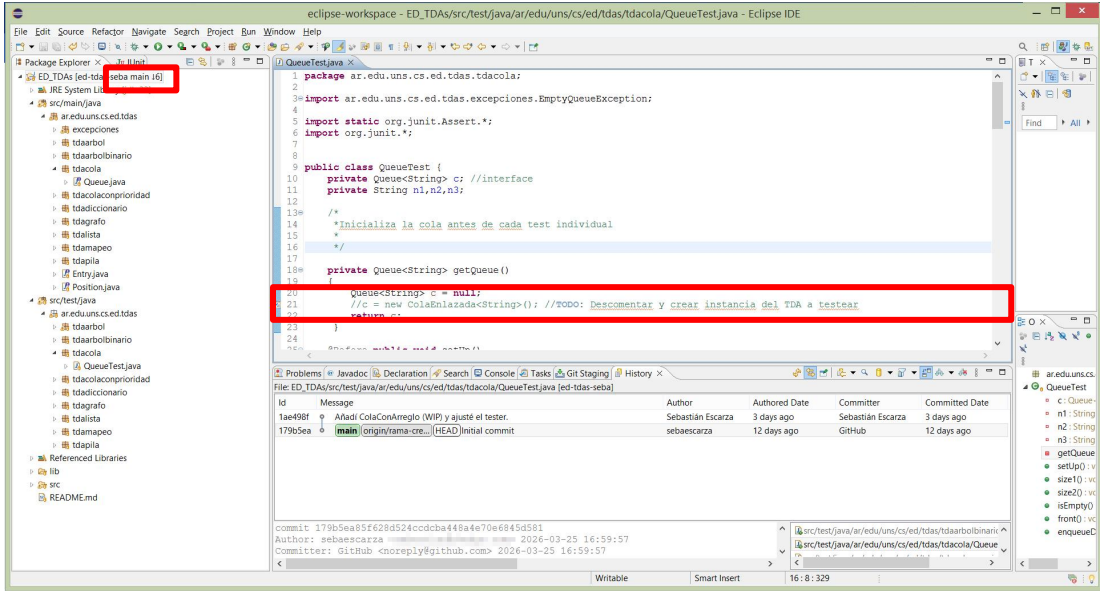
Hacemos **checkout** de main para traer los cambios que otros desarrolladores pudieron haber hecho en origin/main.



Resolviendo merge conflicts

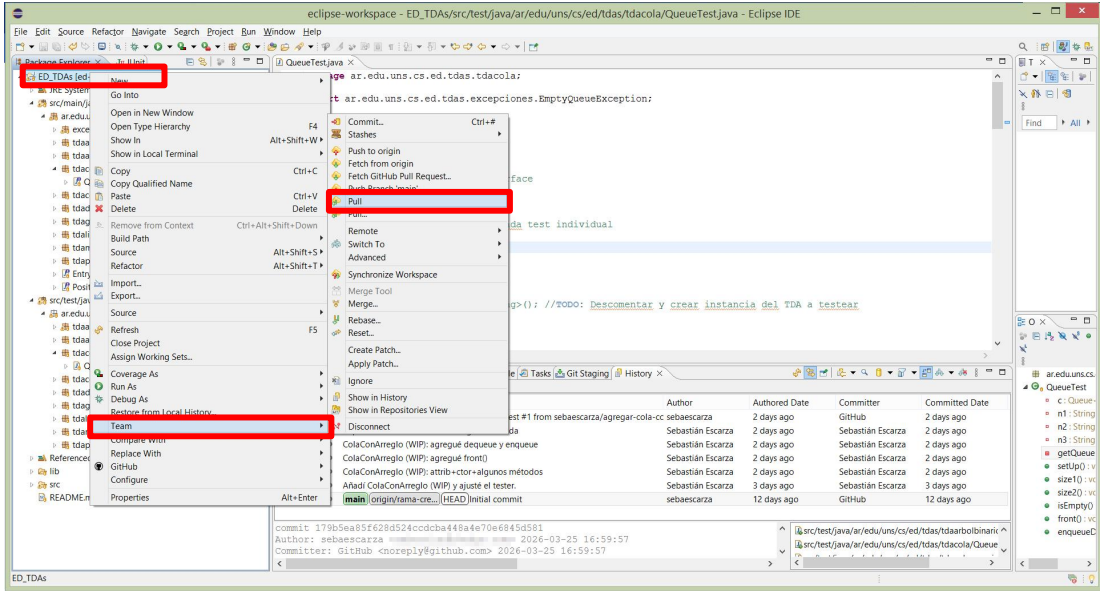
Eclipse indica que hay 6 commits para descargar que están en GitHub pero aún no están reflejados localmente en main.

Notar que la línea 21 de QueueTest.java está comentada en la versión local.



Resolviendo merge conflicts

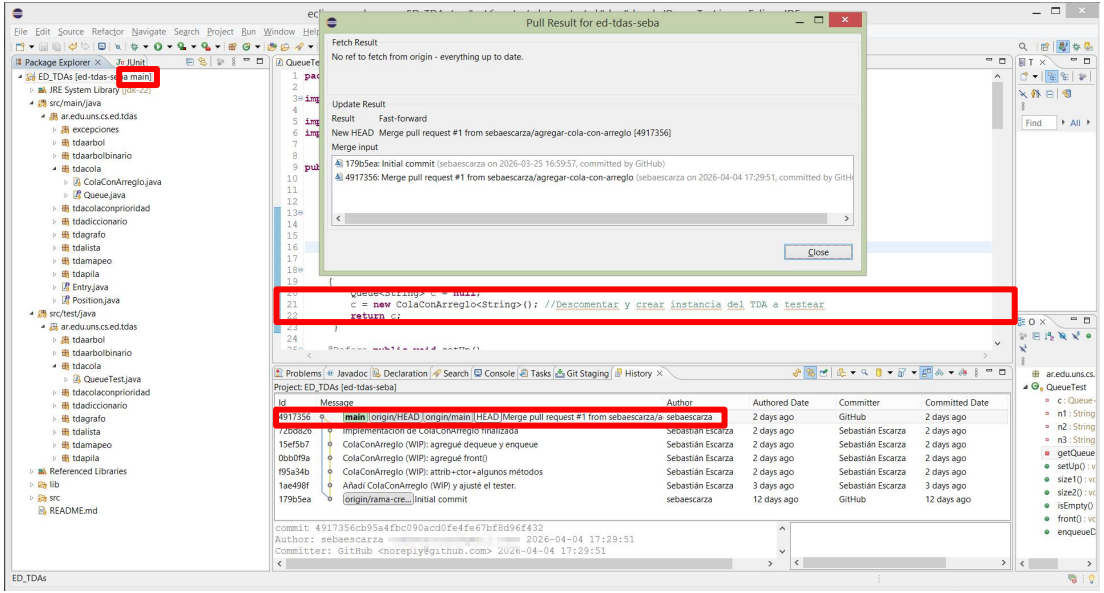
Hacemos un **Pull** para descargar los cambios.



Resolviendo merge conflicts

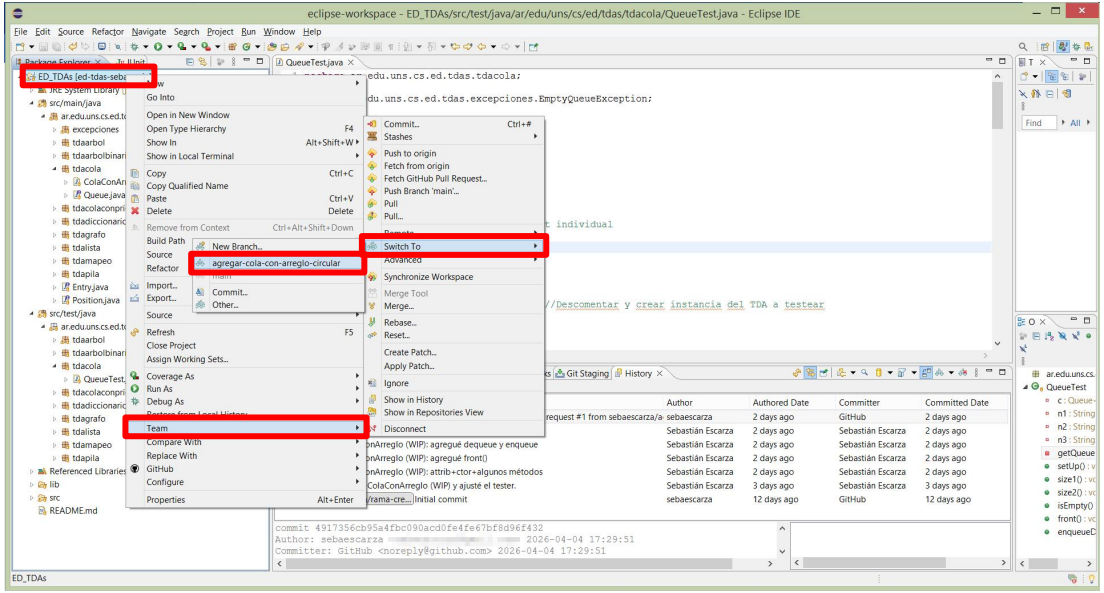
Luego de la actualización ya tenemos los últimos cambios del remote (GitHub) de origin/main en main local (main avanzó en la **History** y alcanzó a origin/main).

Notar que la línea 21 de QueueTest.java crea una instancia de ColaConArreglo, un cambio que pudo haber hecho otro programador mientras trabajábamos.



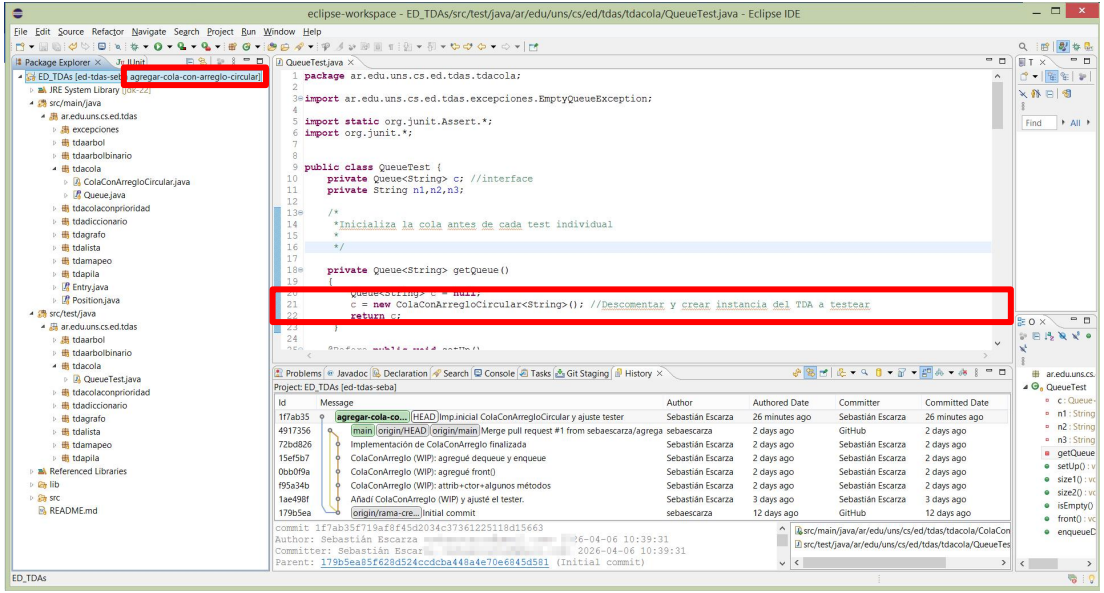
Resolviendo merge conflicts

Hacemos **checkout** de agregar-cola-con-arreglo-circular para volver a la rama de trabajo.



Resolviendo merge conflicts

En nuestro código, la línea 21 de QueueTest.java crea una instancia de ColaConArregloCircular, algo que provocará un conflicto con el código de main.

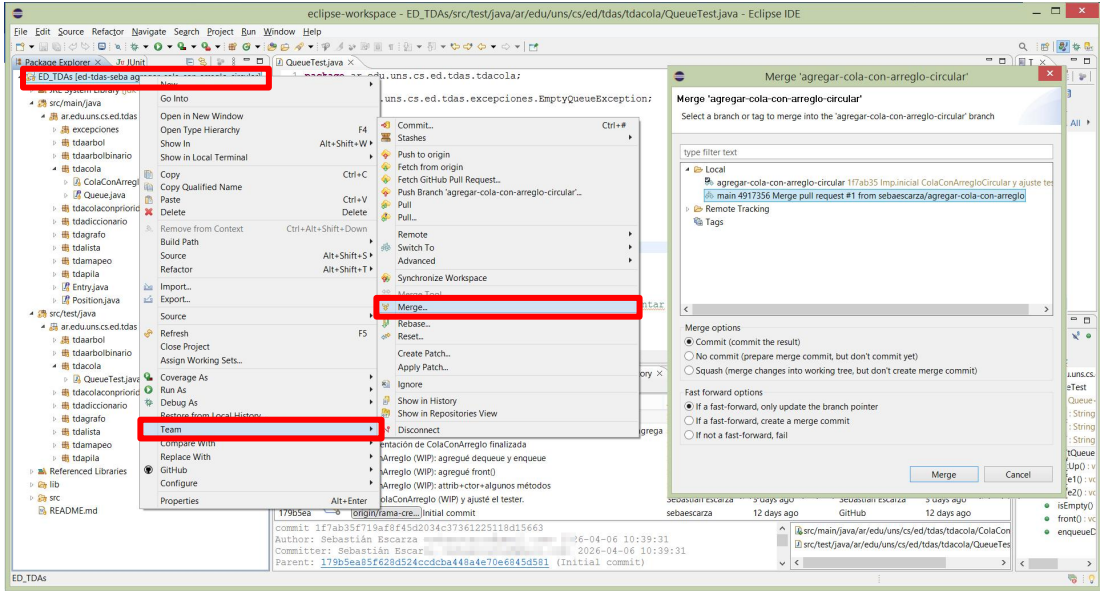


Resolviendo merge conflicts

Para preparar la rama local para mezclarse con origin/main, vamos a mezclar main en ella. Esto hará que los cambios de main pasen a ser parte también de la rama local achicando sus diferencias.

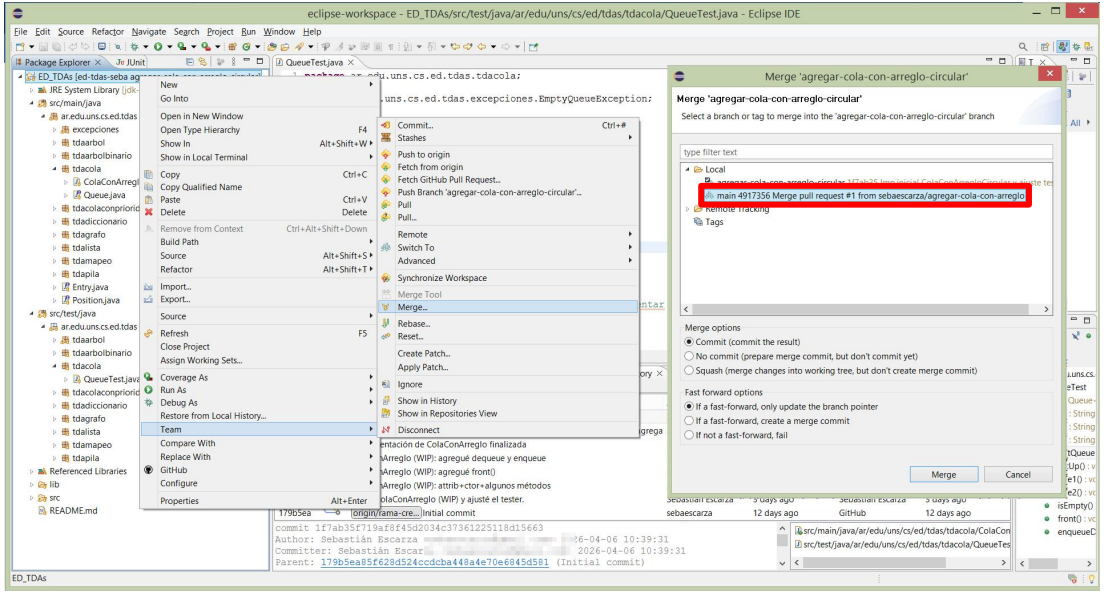
Para eso, estando en la rama local (agregar-cola-con-arreglo-circular) vamos a:

... > Team > Merge...



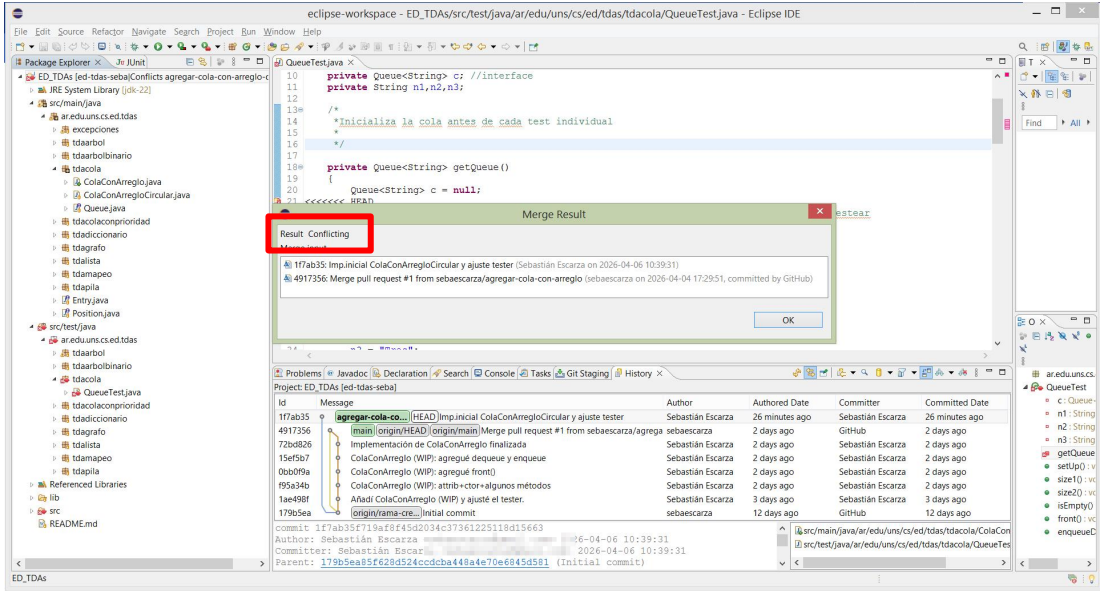
Resolviendo merge conflicts

Elegimos **main** para hacer la mezcla, y presionamos **Merge**.



Resolviendo merge conflicts

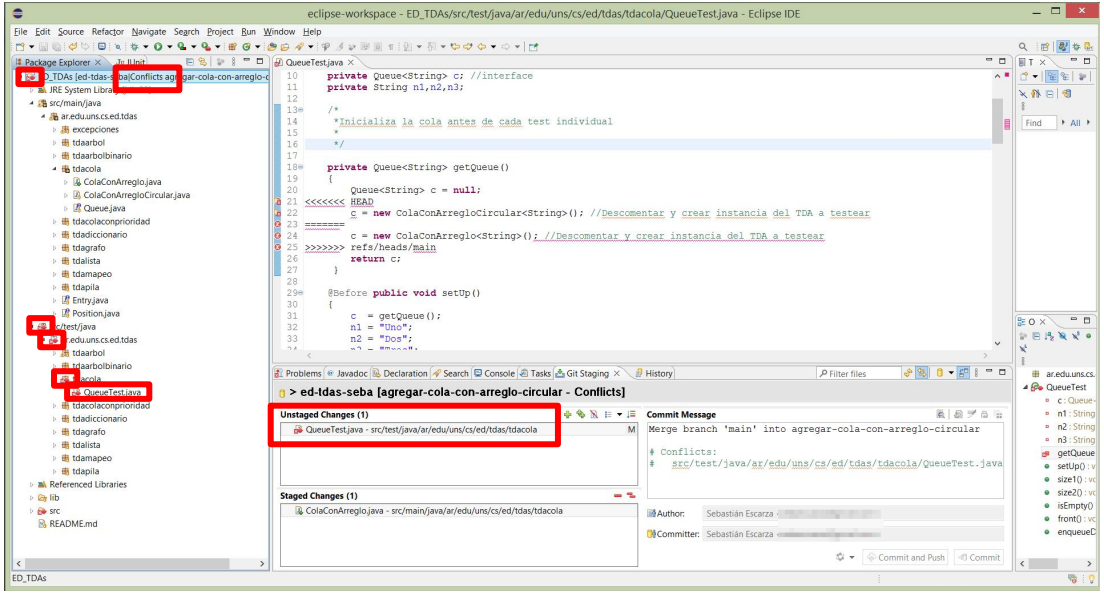
La mezcla no se completa por la existencia de conflictos (el commit queda configurado a la espera de resolución).



Resolviendo merge conflicts

Los conflictos se muestran:

- en el panel **Git Staging** de Eclipse, como Unstaged changes
- en la rama actual se indica la existencia de conflictos
- en el **Package Explorer** mediante la propagación de la sub-iconografía inferior derecha que indica su existencia

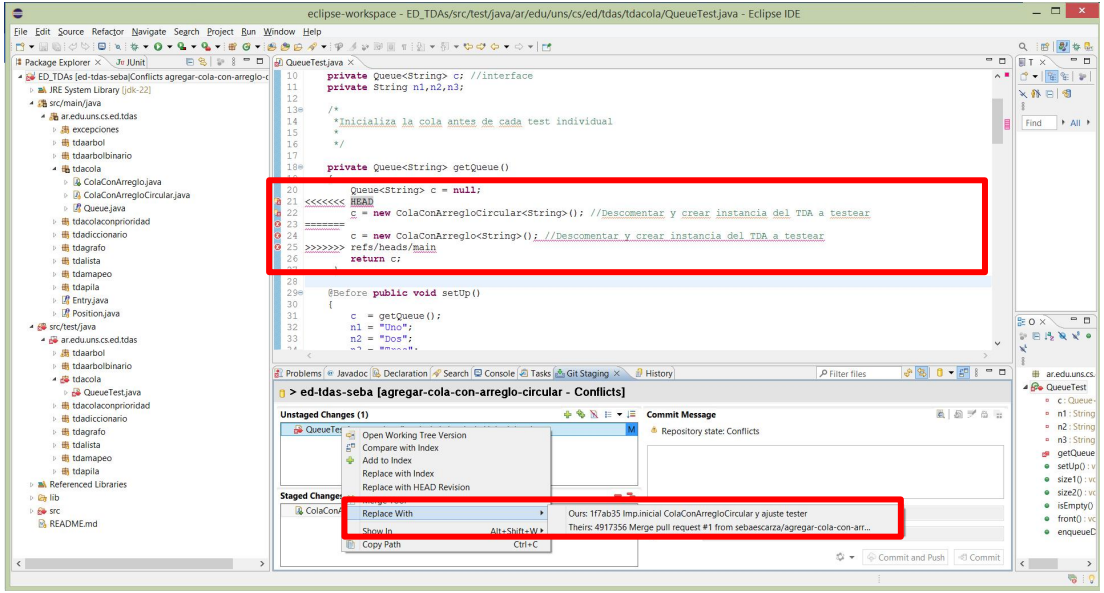


Resolviendo merge conflicts

Los conflictos también se muestran en el archivo con los marcadores típicos de git:

<<<<<<<
Línea 21 versión rama local
=======
Línea 21 versión de main
>>>>>>>

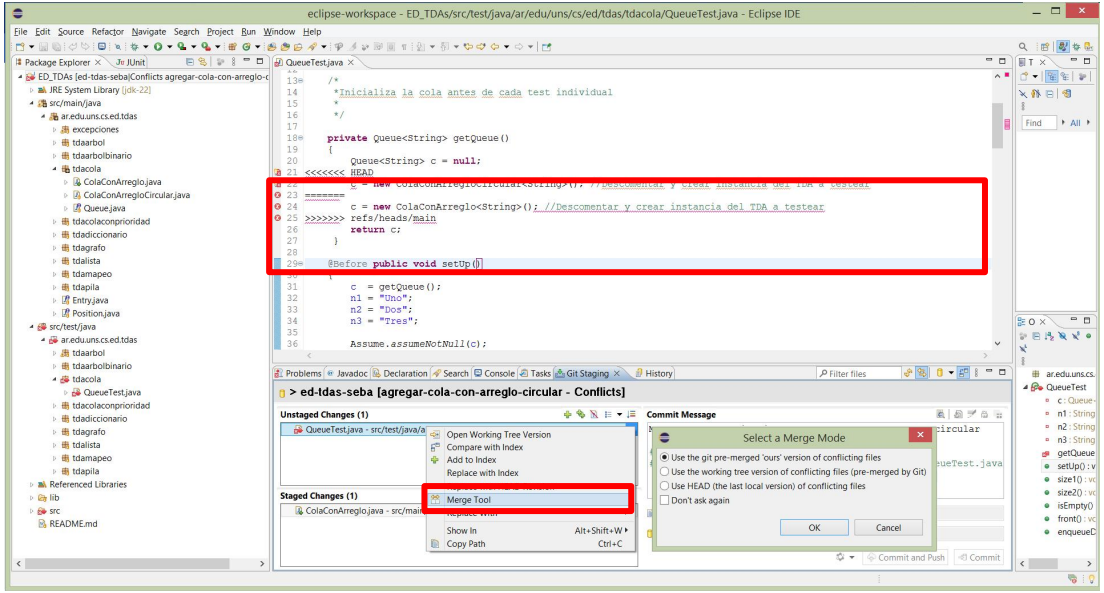
Además, el panel **Git Staging** ofrece opciones para tomar todos los cambios propios (Ours) o entrantes (Theirs) pero a nivel de archivo completo.



Resolviendo merge conflicts

La resolución básica implica editar el archivo, eliminar los marcadores dejando el contenido como se desea, y luego pasar el archivo a Staging para hacer el Commit del merge (cuyo mensaje está preconfigurado).

Eclipse también tiene una **Merge Tool** algo rústica en el menú contextual, para resolver los conflictos uno por uno.

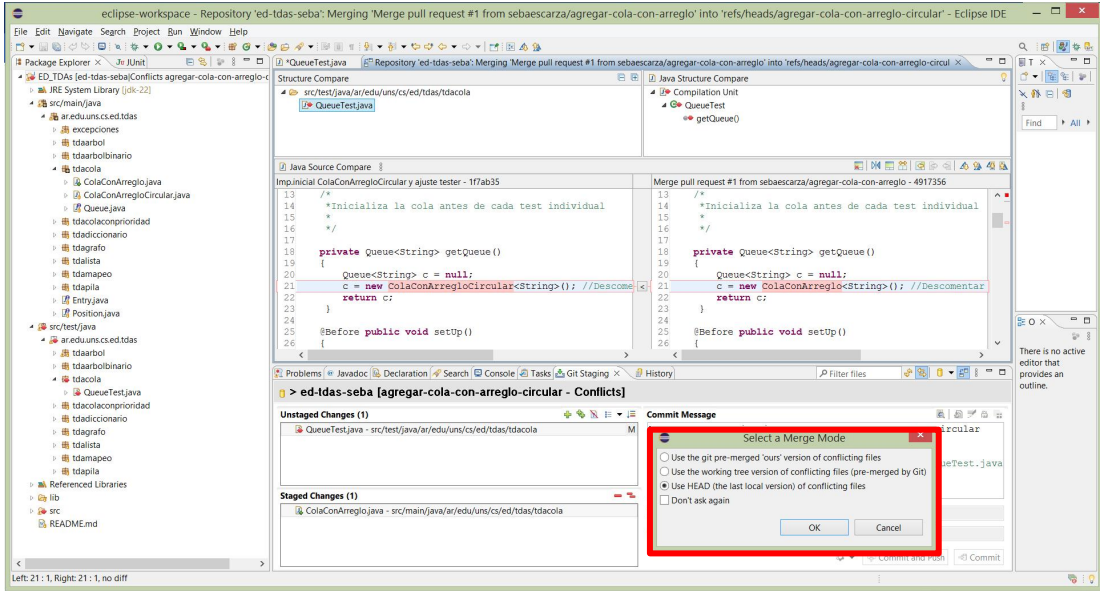


Resolviendo merge conflicts

La herramienta de merge muestra los cambios en dos paneles. En el lado derecho muestra los cambios entrantes de la rama a mezclar.

Lo que muestra en el lado izquierdo depende del Merge Mode elegido:

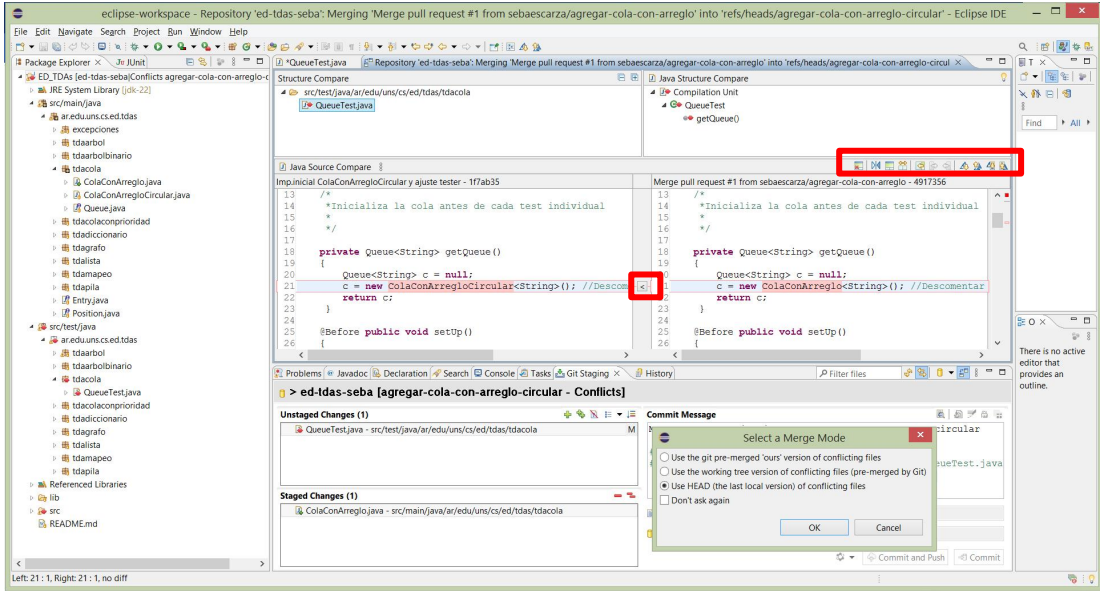
- Use Git pre-merged...: la versión entrante (quedan ambos lados iguales, útil si tenemos tendencia a preferir los cambios entrantes.
- Use the working tree...: el archivo con los marcadores de git para editar a mano.
- Use HEAD...: la versión de la rama actual (la opción normalmente más útil, y que se muestra en el ejemplo)



Resolviendo merge conflicts

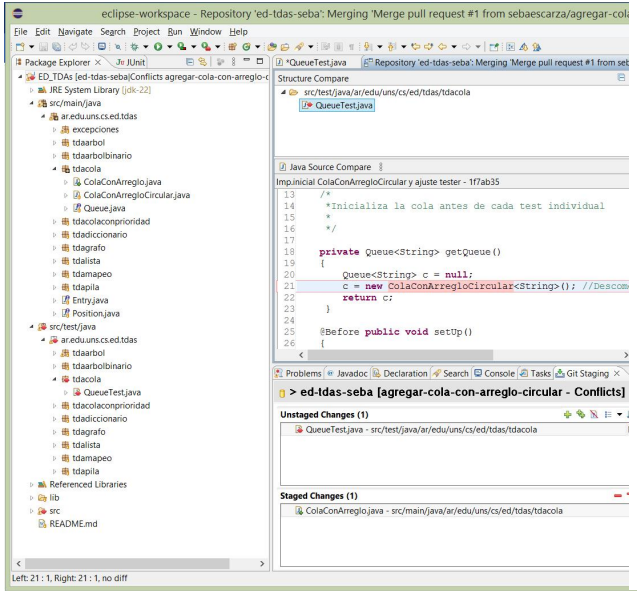
La herramienta de merge tiene una barra para navegar los cambios y para copiar con un click las versiones de cada conflicto del lado derecho al izquierdo.

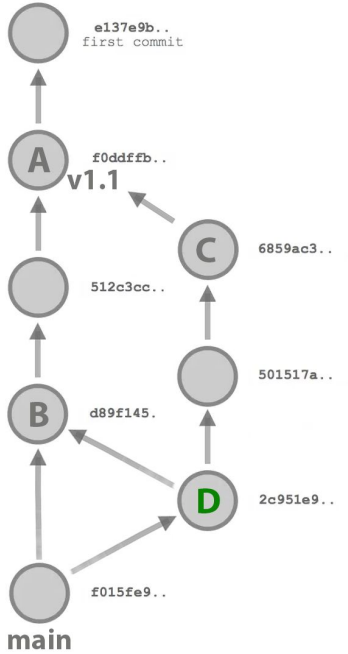
No es una herramienta muy intuitiva y suele ser más práctico editar el archivo directamente borrando los marcadores de conflicto de git.



Resolviendo merge conflicts

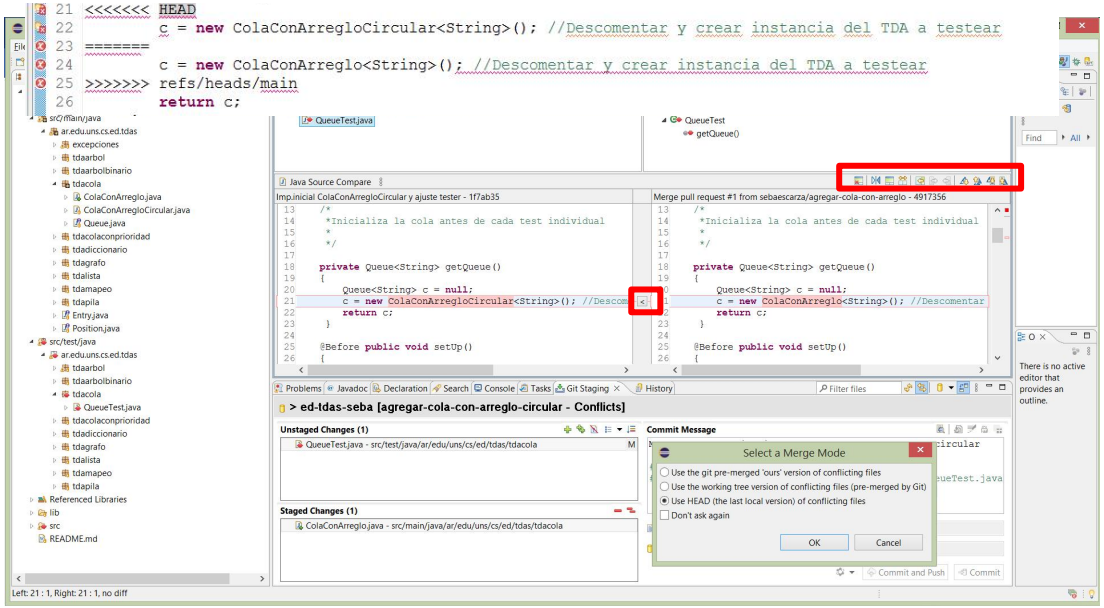
El Merge Tool nos muestra la versión de main (incoming = B), la versión de la rama local (agregar-cola-con-arreglo-circular = C) y tiene un control para ver la versión del commit ancestro común más reciente para tener mayor contexto de los cambios (A).





Resolviendo merge conflicts

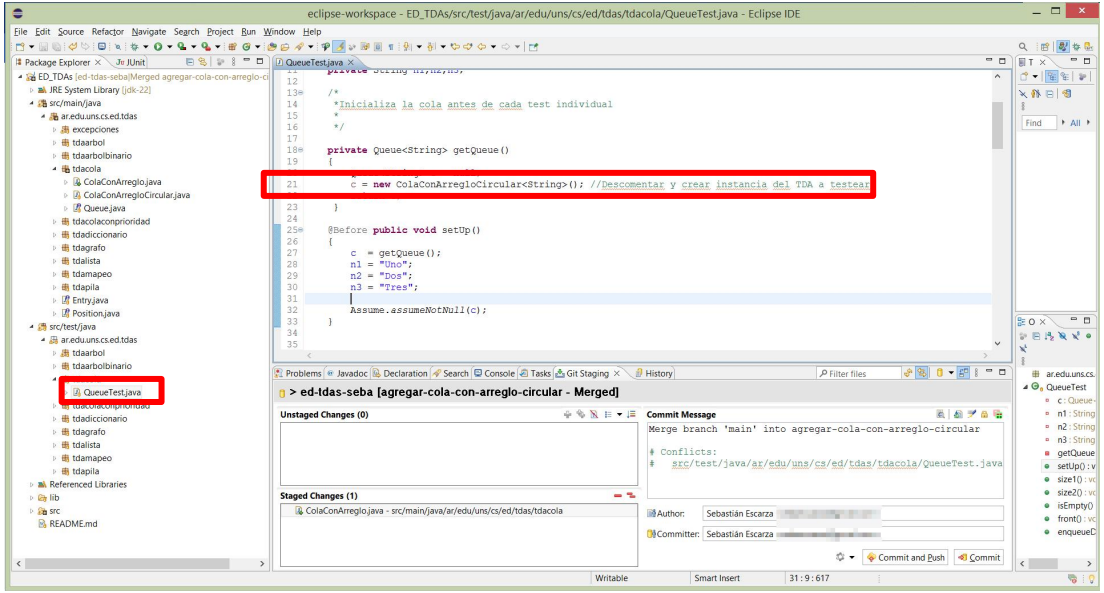
Tanto si se usa el Merge Tool como si se resuelven editando a mano, hay que ir conflicto por conflicto determinando cuál será la versión que prevalece en la mezcla.



Resolviendo merge conflicts

Una vez resueltos todos los conflictos hay que pasar los cambios a Staging para unirlos con los cambios no conflictivos que ya estaban ahí y proceder al **Commit**.

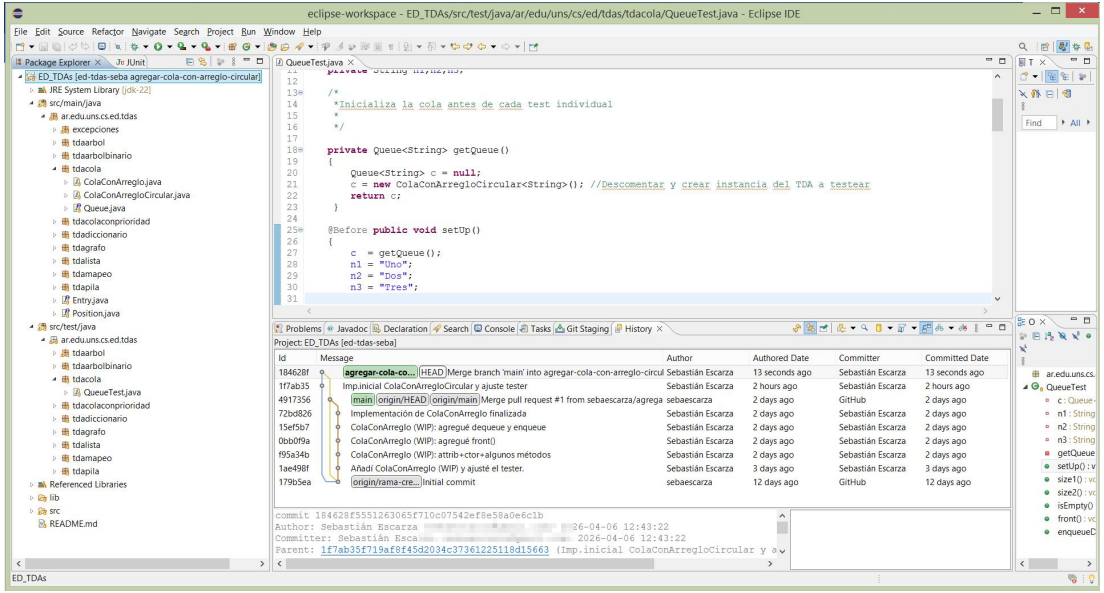
En el ejemplo QueueTest.java ya no está en conflicto, pero como hemos decidido resolverlo con la versión de la rama local (HEAD), no hay cambios que hacer. Por eso QueueTest.java desapareció al pasarlo a Staging.



Resolviendo merge conflicts

Luego del **Commit**, el historial muestra la fusión de ambas ramas, y el código inicializando una instancia de ColaConArregloCircular.

A partir de este punto main quedó mezclada localmente en agregar-cola-con-arreglo-circular y ya se pueden sincronizar los cambios (**git push**) para publicarla y luego continuar trabjando, o ir a GitHub a abrir el PR e intentar mezclarla.



Utilizando Git Bash y Git GUI

Git Bash y Git GUI

Un repositorio de Git puede gestionarse con cualquier herramienta que le de soporte. El estado del repositorio se matiene en la subcarpeta **.git** del mismo y cualquier herramienta que lo acceda puede mostrarnos el repositorio y ayudarnos a gestionarlo.

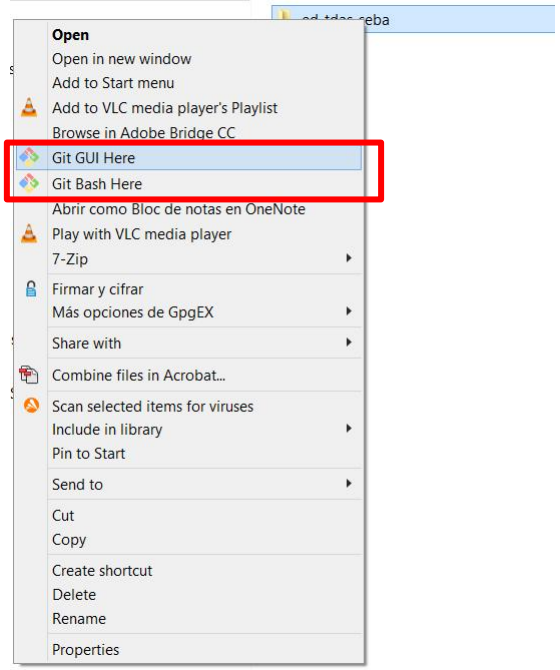
A continuación presentamos dos herramientas complementarias que incluye la **instalación de Git para Windows** y que, aún sin describirlas de manera completa, en ciertos contextos pueden resultar de utilidad como complemento de la funcionalidad que nos da Eclipse.

Estas herramientas son:

- **Git Bash**: una interface por línea de comandos (CLI)
- **Git GUI**: una herramienta con interface gráfica de usuario (GUI)

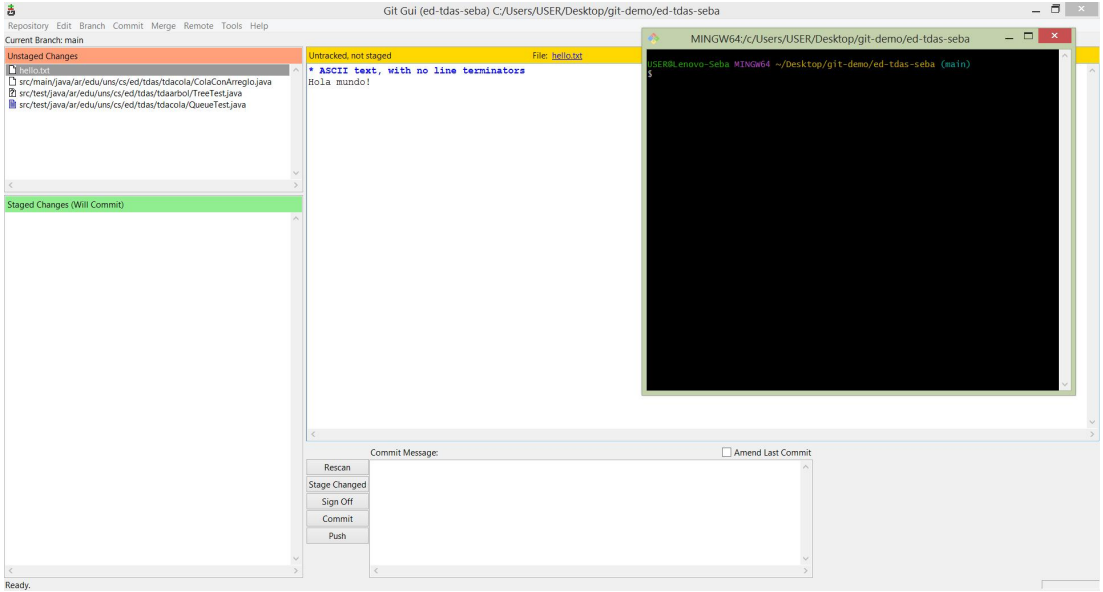
Git Bash y Git GUI

Se puede acceder a estas herramientas desde el Explorador de Archivos, yendo a la carpeta que contiene el repositorio y haciendo click derecho sobre la carpeta que es el repositorio de Git sobre el que queremos trabajar.



Git Bash y Git GUI

La captura muestra ambas herramientas abiertas sobre el mismo repositorio.

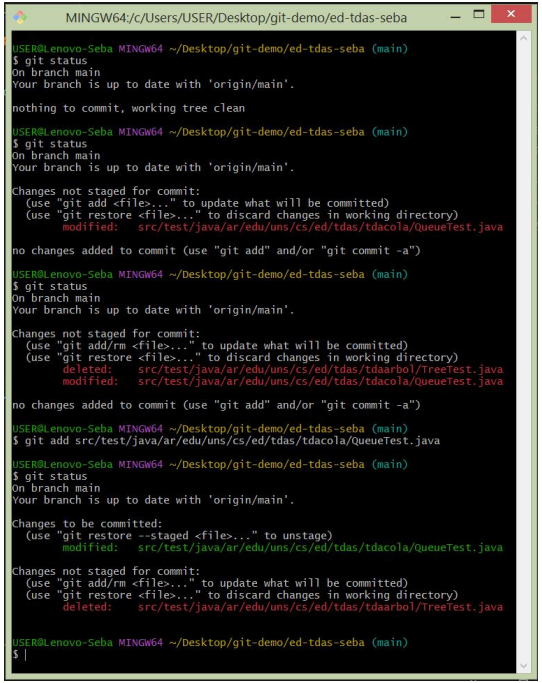


Git Bash

Git Bash es una CLI basada en Bash, el intérprete de comandos de Linux, ejecutando en Windows mediante MINGW64.

Muestra la carpeta y rama actual y permite escribir los comandos de Git de manera directa y sin ornamentos (los comandos que el IDE ejecuta por nosotros). Proporciona la máxima flexibilidad pero **hay que saber lo que se hace**.

En el ejemplo se usa **git status** para ver el estado del Working Directory respecto del último commit de la rama actual, antes y después de algunas modificaciones (se muestran en rojo). También se usa **git add** para agregar un archivo al Staging Area, archivos que git status muestra en verde.

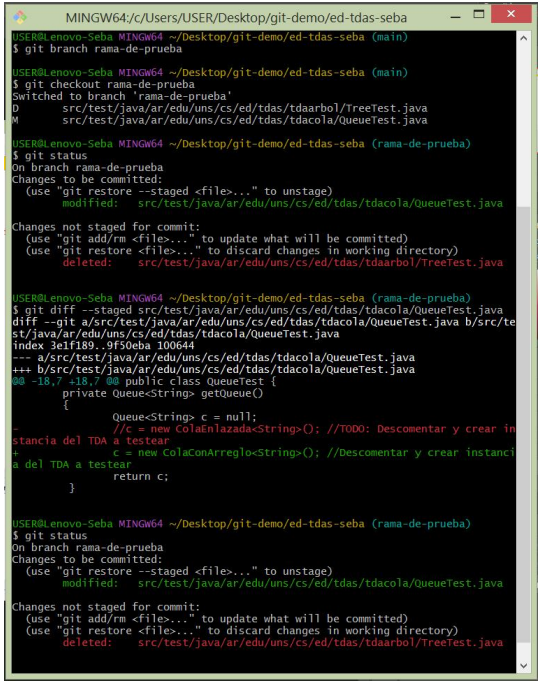


Git Bash

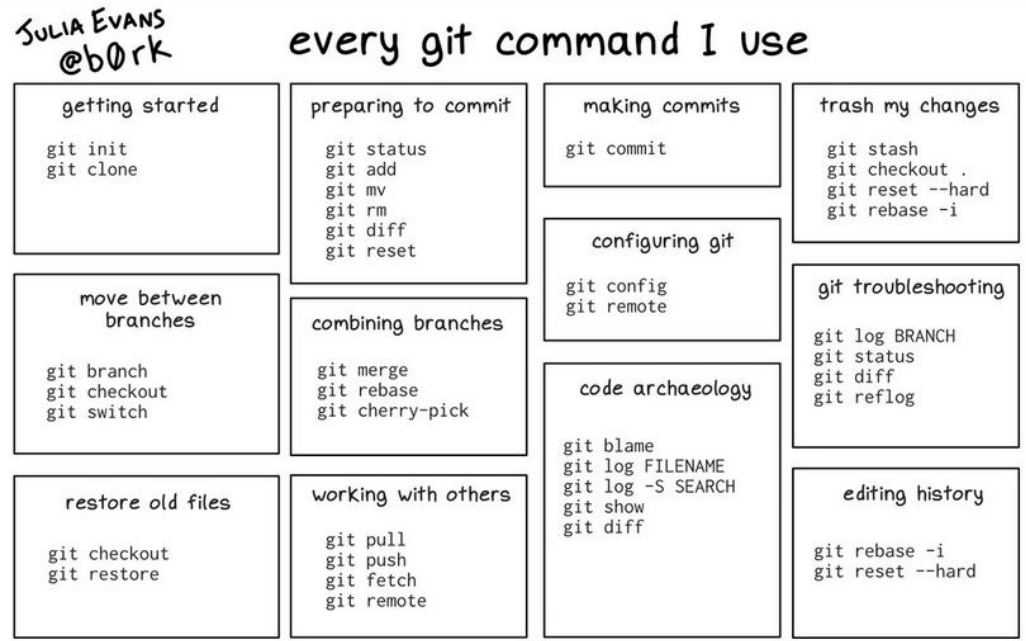
Más comandos de ejemplo:

- **git branch**: para crear una rama local nueva
- **git checkout**: para cambiar a la rama creada
- **git diff --staged**: para comparar los cambios de un archivo entre la versión en staging y la versión del último commit de la rama actual
- **git status**: para examinar el estado del Working Directory y del Staging Area entre comandos.

Hay una innumerable cantidad de comandos (y parámetros) de Git. Uno no llega a dominarlos todos y se maneja con un subconjunto.



Algunos Comandos de Git



Git Cheat Sheet

Git: configurations

```
$ git config --global user.name "FirstName LastName"
$ git config --global user.email "your-email@email-provider.com"
$ git config --global color.ui true
$ git config --list
```

Git: starting a repository

```
$ git init
$ git status
```

Git: staging files

```
$ git add <file-name>
$ git add <file-name> <another-file-name> <yet-another-file-name>
$ git add .
$ git add --all
$ git add -A
$ git rm --cached <file-name>
$ git reset <file-name>
```

Git: committing to a repository

```
$ git commit -m "Add three files"
$ git reset --soft HEAD^
$ git commit --amend -m <enter your message>
```

Git: pulling and pushing from and to repositories

```
$ git remote add origin <link>
$ git push -u origin master
$ git clone <clone>
$ git pull
```

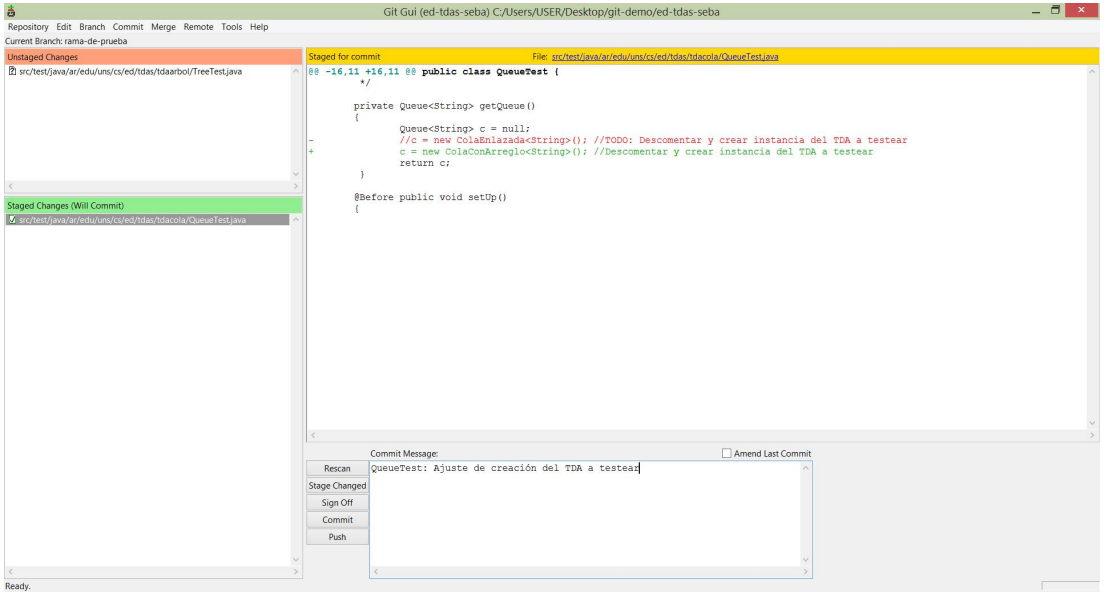
Git: branching

```
$ git branch
$ git branch <branch-name>
$ git checkout <branch-name>
$ git merge <branch-name>
$ git checkout -b <branch-name>
```

Git GUI

La aplicación Git GUI muestra el estado del repositorio de manera gráfica.

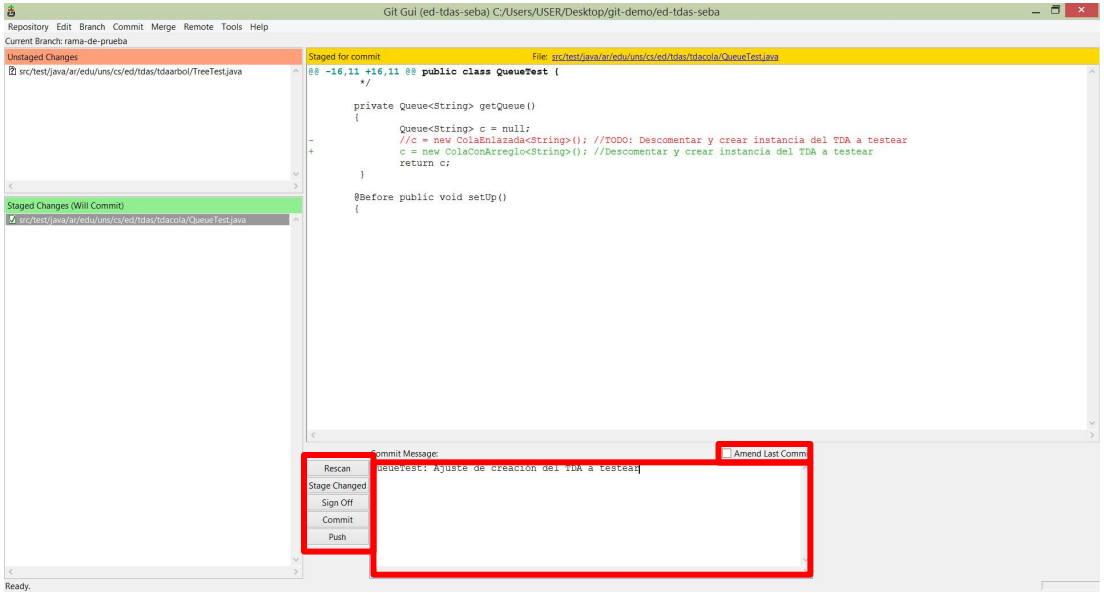
Se muestra la rama actual (debajo del menú), el Working Directory (**Unstaged Changes** en rojo), el Staging area (**Staged Changes** en verde), y para el cambio seleccionado se muestra un **git diff** con el detalle (panel amarillo).



Git GUI

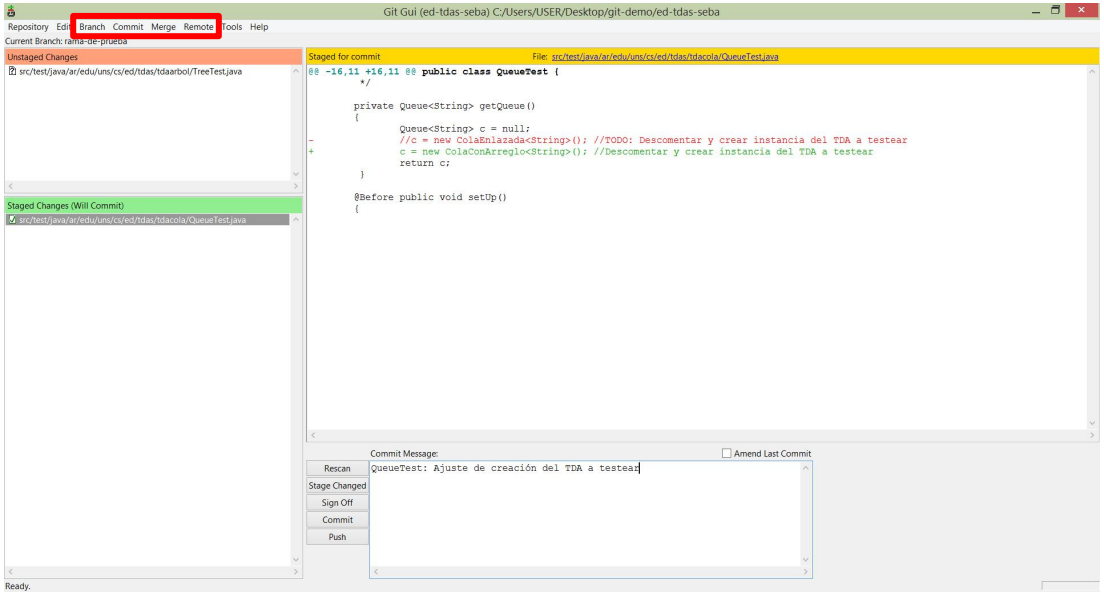
En la parte inferior hay un panel cuyos botones más usados son:

- **Rescan**: refresca la vista del repositorio (por si algo cambió externamente a la herramienta).
- **Stage Changed**: pasa a Staging todos los Unstaged Changes.
- **Commit**: Realiza el commit con el mensaje indicado. Se puede enmendar el mensaje del último commit (Ammend Last Commit)
- **Push**: Para enviar los commits al remote.



Git GUI

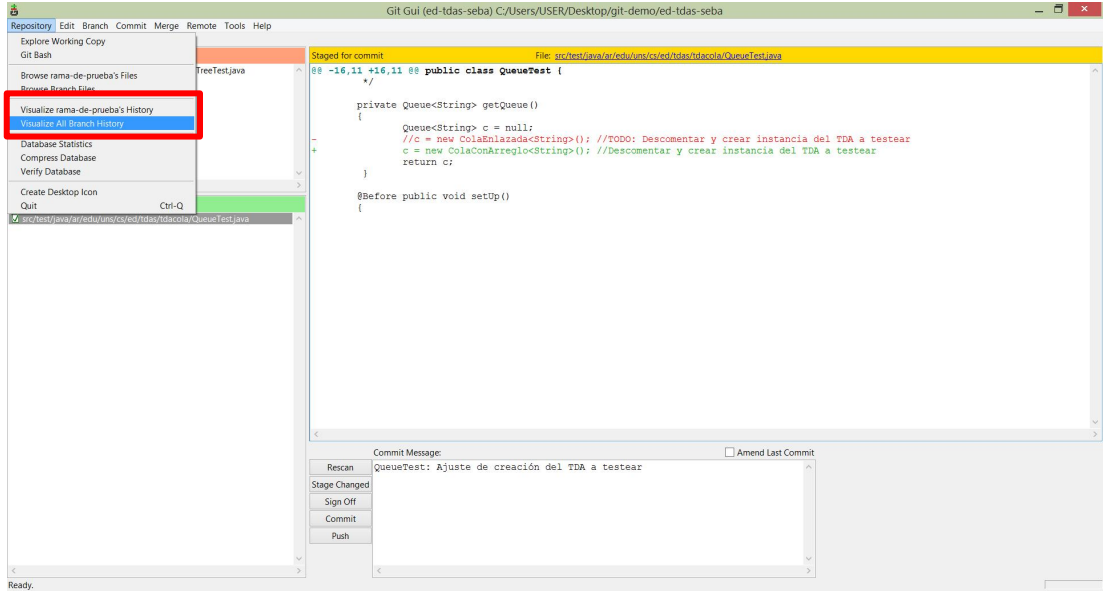
Si la situación requiere salirse del flujo normal, el menú dispone de una gran cantidad de opciones y comandos para gestionar el repositorio.



Git GUI

Una opción muy interesante es visualizar el historial de Git, mediante las opciones del menú Repository para:

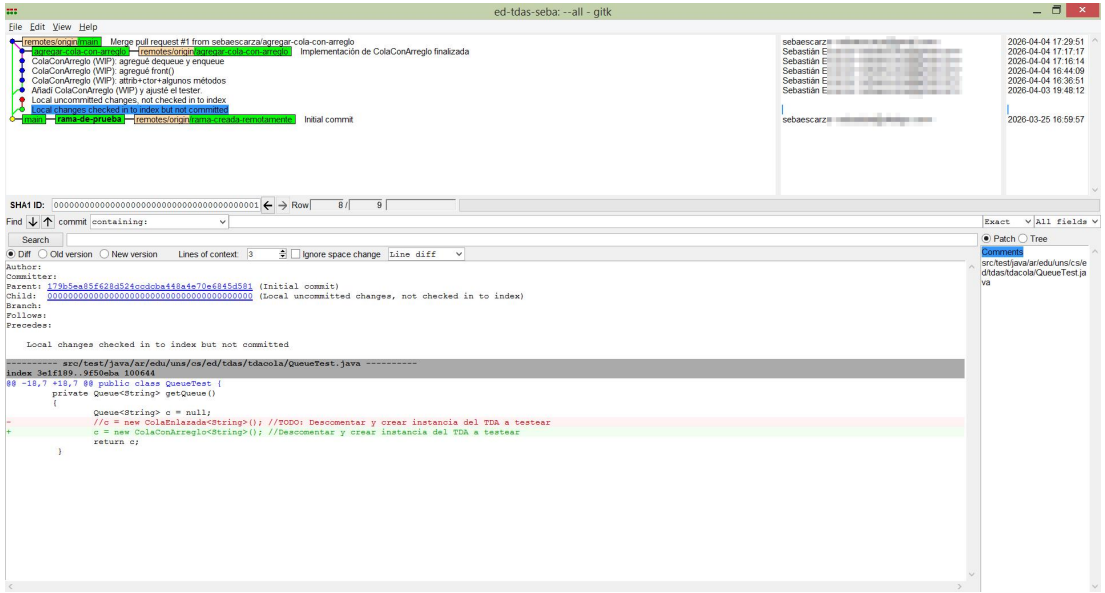
- visualizar el historial de la rama actual, o
- visualizar el historial de todas las ramas.



Git GUI

En la pantalla se muestra el historial con las ramas y commits, y la interface permite seleccionar un commit y examinar los cambios (git diff de cada archivo modificado).

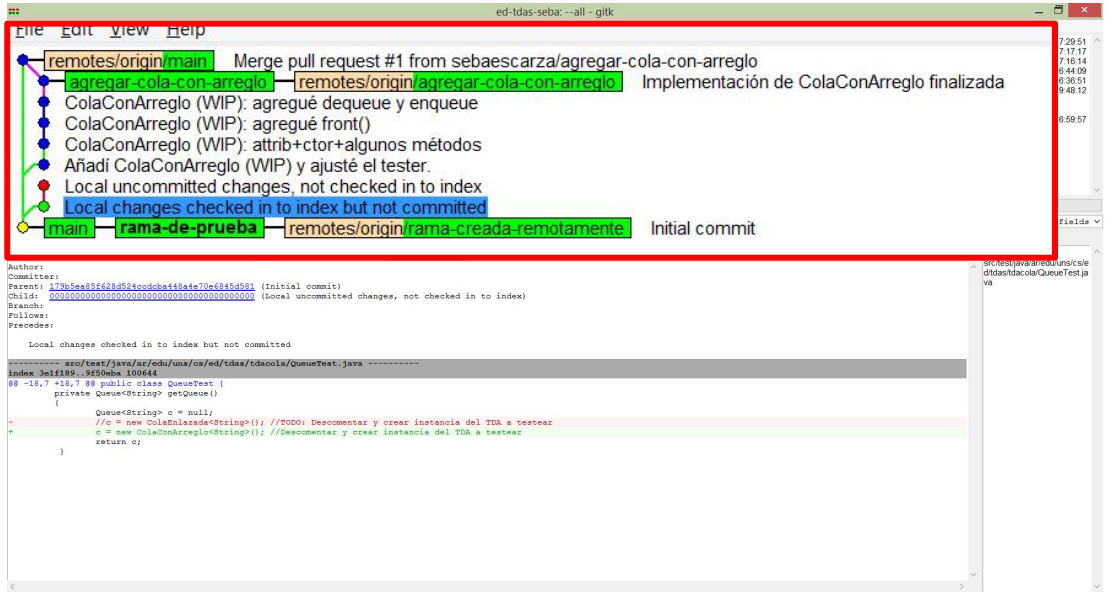
También se muestra con un punto verde, el Staging Area, y con un punto rojo, el Working Directory



Git GUI

En el ejemplo se puede ver:

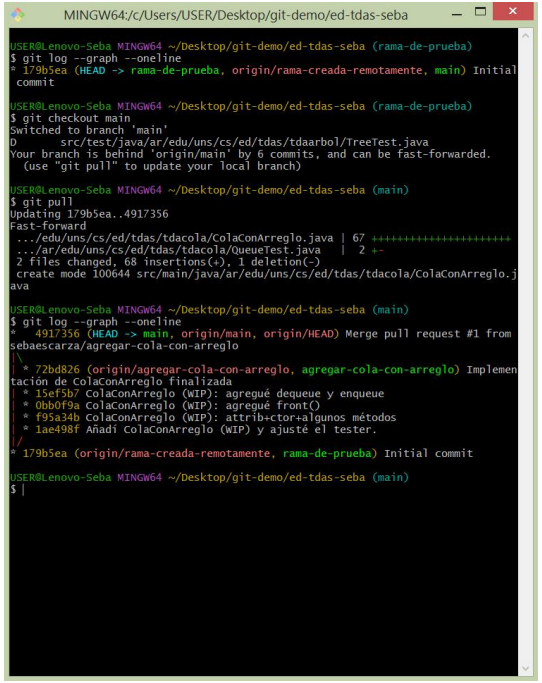
- que **origin/main** está más adelantada que **main** (faltaría hacer **fetch/pull** para traer esos cambios),
- que **rama-de-prueba** es la rama actual y coincide con main (WD y Staging se muestran a continuación como una futura rama),
- **origin/rama-creada-remotamente** es una rama remota que también coincide con el commit inicial,
- la secuencia de commits resultantes de agregar ColaConArreglo al código y su posterior merge con **origin/main**.



Git Bash

Git Bash también permite ver el historial, aunque de manera más rudimentaria. En el ejemplo se usaron:

- **git log --graph --oneline**: para mostrar el historial de la rama actual de manera “gráfica” y compacta
- **git checkout**: para cambiar a main
- **git pull**: para actualizar main trayendo los commits de su rama upstream origin/main (tal y como sugirió Git)
- **git log --graph --oneline**: en main y con la rama actualizada, para ver el historial completo del trabajo de ejemplo sobre ColaConArreglo.



Git Bash y Git GUI

Git Bash y Git GUI (al igual que toda otra herramienta que maneje reqpositorios Git) resultan un complemento del IDE que en determinadas circunstancias puede ser valioso.

Ambas ofrecen soporte de Git de nivel más cercano a la herramienta: su uso es más complejo pero más también más flexible.

Git Bash y Git GUI no resultan esenciales para trabajar con Git en la materia, ya que con el IDE podemos cubrir las operaciones de un workflow básico. Sin embargo es bueno conocer su existencia porque en ciertos casos nos permiten hacer cosas que con el IDE no resulta posible, y eventualmente solucionar algunos problemas.

Recomendaciones y comentarios finales

Git - Aspectos avanzados



Existen numerosos aspectos de uso avanzado de Git que no vimos y difícilmente usaremos:

- distintos tipos de merges (squash, rebase), cherrypicking, y mecanismos de reescritura de la historia,
- configuración de múltiples remotes
- stashing y almacenamiento de cambios temporales,
- herramientas de análisis y como git blame/bisect entre otras,
- branching models personalizados (adoptaremos las alternativas más simples y utilizadas)

Salvo casos puntuales (o de fuerza mayor), en general utilizaremos las GUI de los IDEs para gestionar el control de cambios, en lugar de los comandos de git de manera directa.

Recomendaciones finales



- Siempre se trabaja desde un punto de vista local: en el nodo actual (al recibir y enviar cambios desde/hacia los remotes), en la rama actual (cuando mezclo otras ramas), en el directorio de trabajo actual (al comparar con el último commit), etc. Tener siempre presente dónde estamos trabajando en el repositorio.
- Usar buenos nombres y descripciones para ramas, commits y PRs. Mientras más limpia y clara escriban la historia de desarrollo, más simple será encontrar problemas y solucionarlos.
- Prestar atención a qué cambios meten en cada commit: no permitan nada no deseado.
- Mitigar los conflictos con las recomendaciones dadas y mantener sincronizado el repo local.
- Y finalmente, no tener pereza o miedo de meter mano, experimentar con la herramienta y practicar. Estamos aprendiendo, aprovechen la oportunidad.

Links:

Git: https://git-scm.com/

Github: https://github.com/

Tutorial visual interactivo de Git (comandos): https://learngitbranching.js.org

Repositorio GitHub de ED/TDAs (público): https://github.com/ed-dcic-uns/tdas

¿Preguntas?